

UNIVERSITY OF WATERLOO

Naming

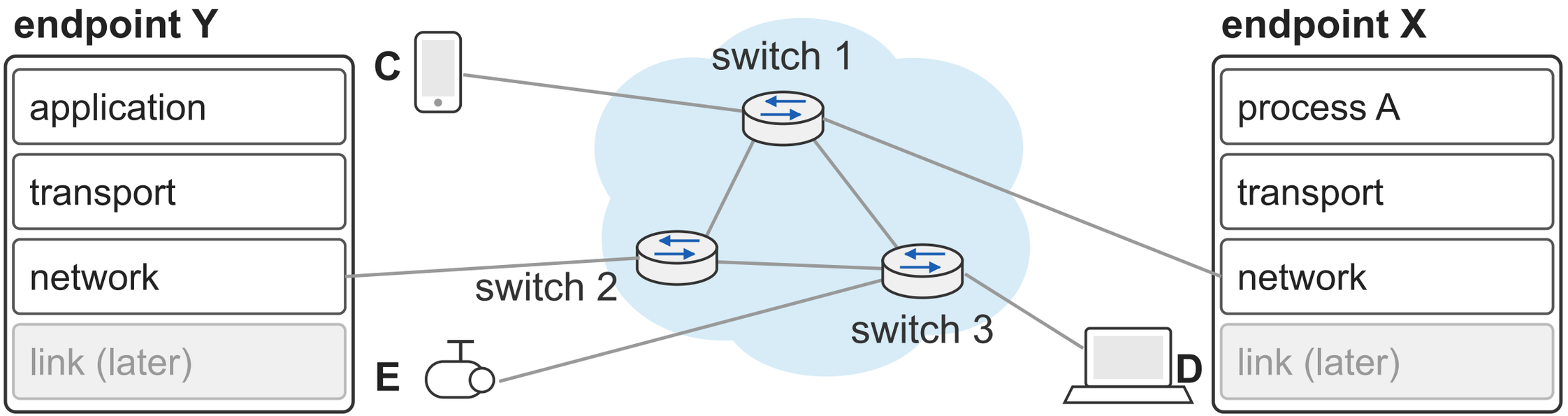
CS 456/656, Fall 2026

Mina Tahmasbi Arashloo

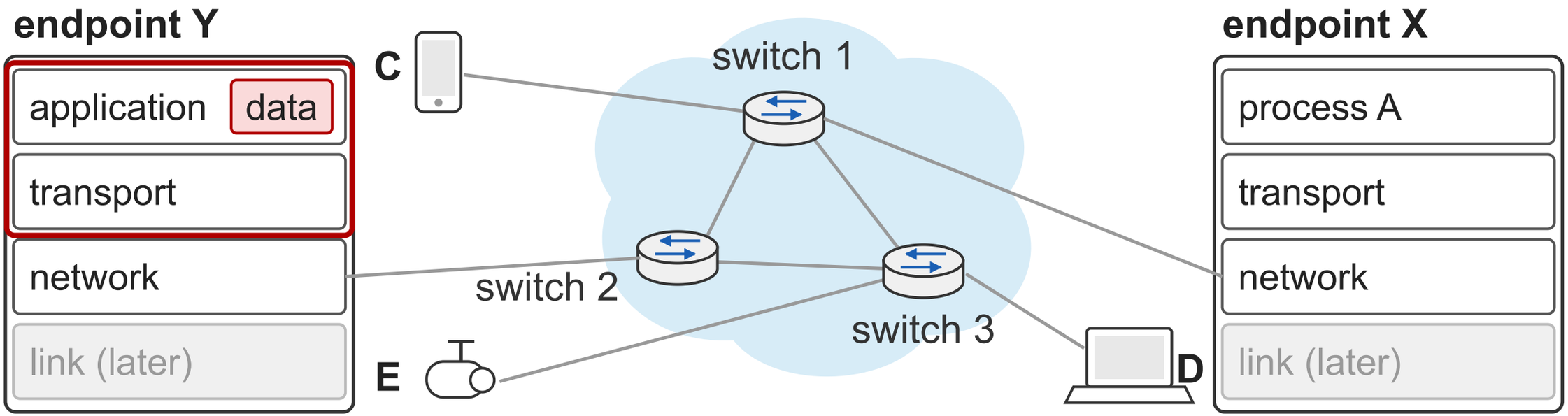
Part III: Scaling to the whole planet

- Part I: the network as a pipe between two endpoints.
- Part II: the network is a shared infrastructure: finding paths, congestion control.
- Part III: scaling to the whole planet.
    - Billions of endpoints, on a network run by tens of thousands of organizations.

Sending data to endpoint X

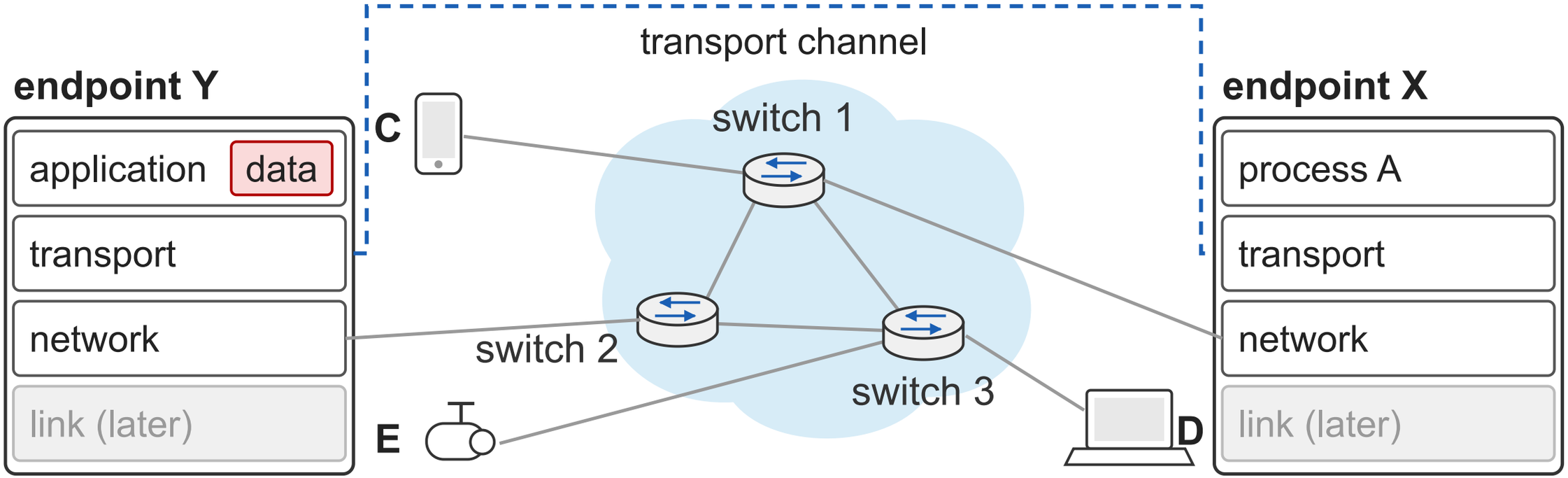


Sending data to endpoint X



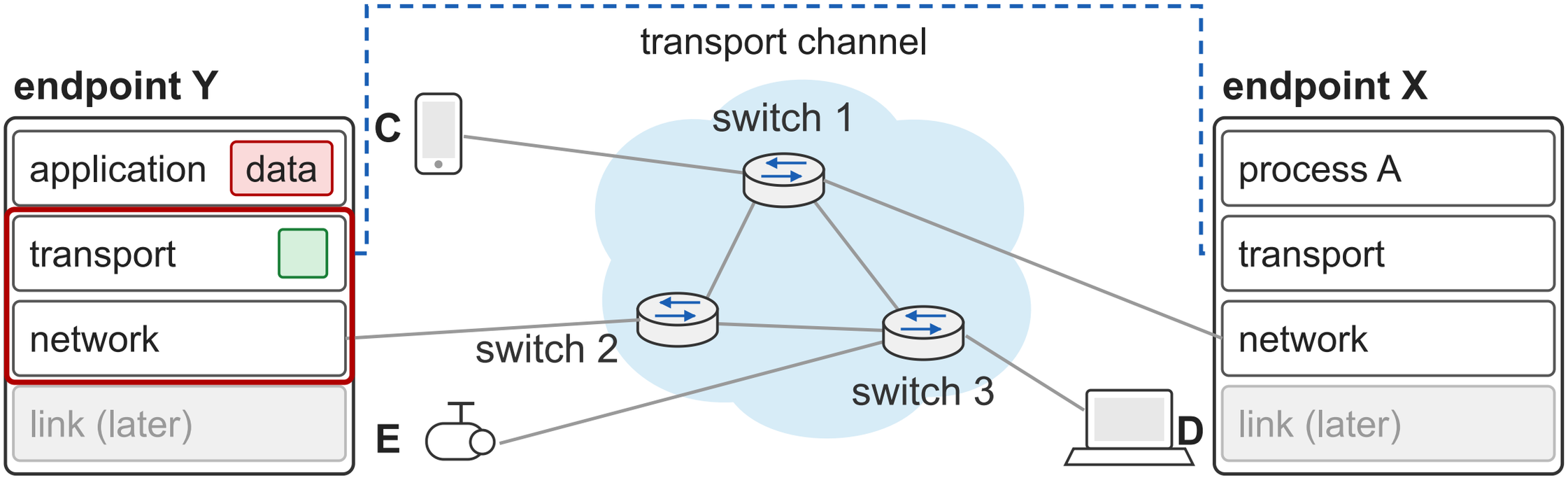
Application → transport: “I want to send data to process A at endpoint X.”

Sending data to endpoint X



Application → transport: “I want to send data to process A at endpoint X.”

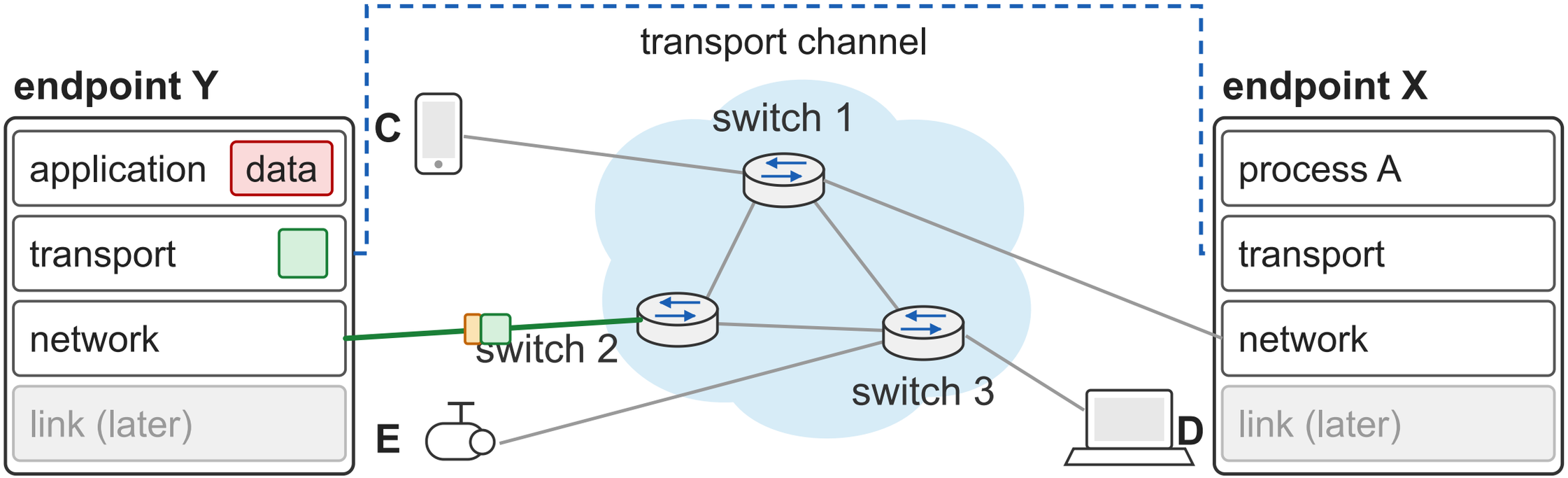
Sending data to endpoint X



Application → transport: "I want to send data to process A at endpoint X."

Transport → network: "Deliver this segment/ack to endpoint X."

Sending data to endpoint X

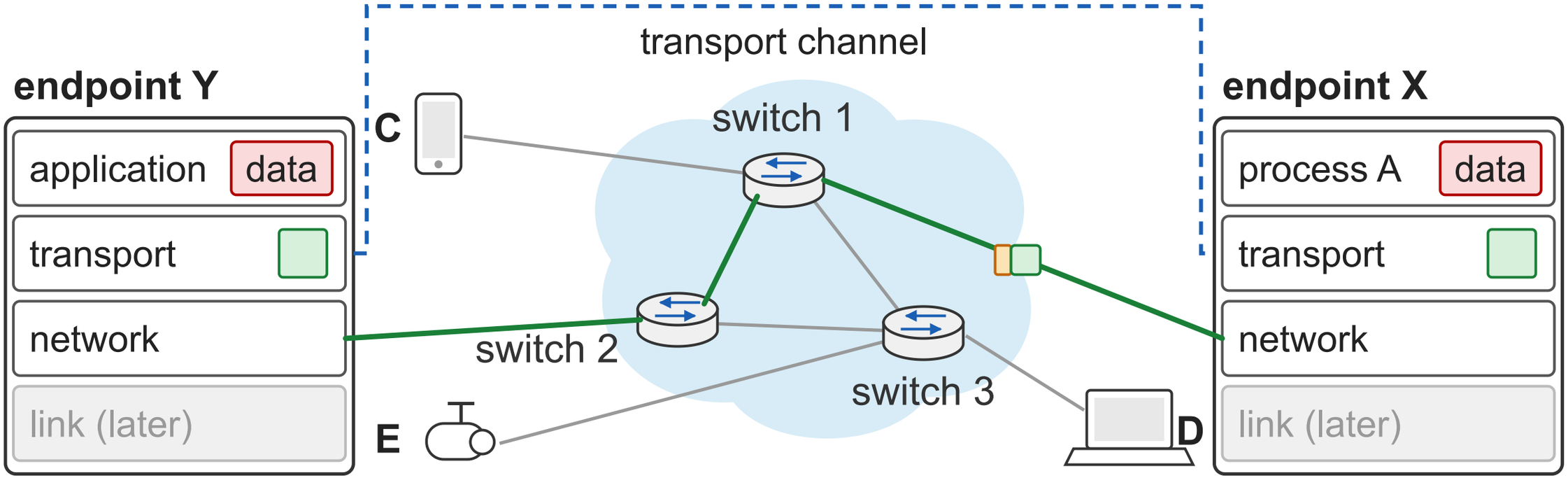


Application → transport: "I want to send data to process A at endpoint X."

Transport → network: "Deliver this segment/ack to endpoint X."

switch 2's forwarding table
endpoint X: the link to switch 1

Sending data to endpoint X



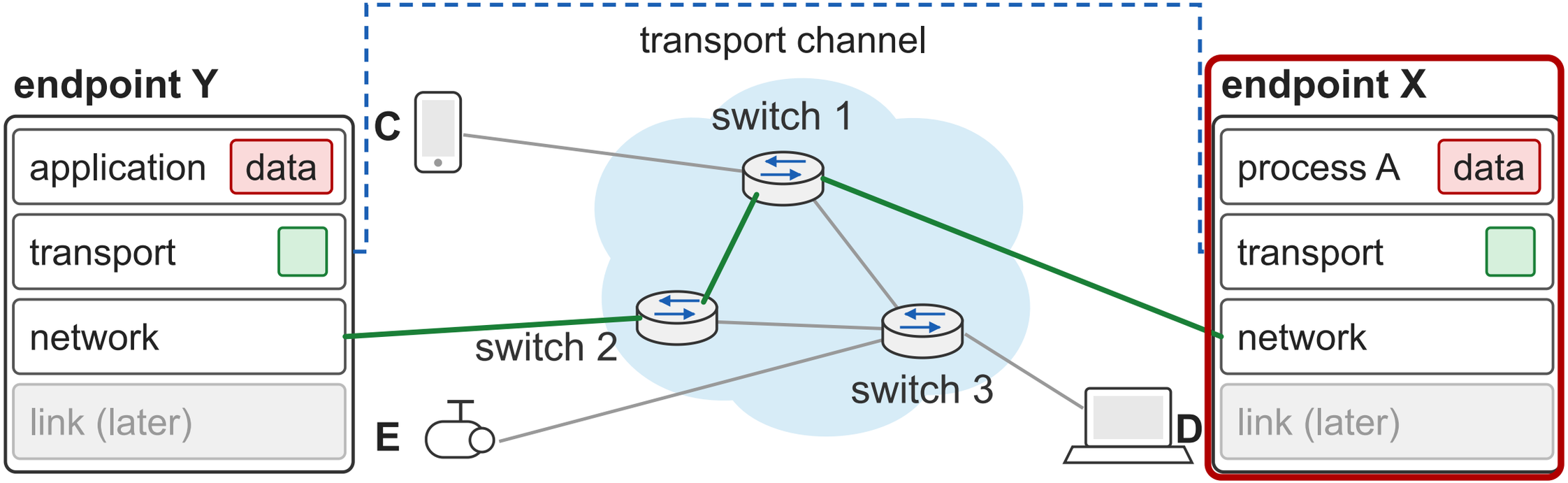
Application → transport: "I want to send data to process A at endpoint X."

Transport → network: "Deliver this segment/ack to endpoint X."

switch 2's forwarding table
endpoint X: the link to switch 1

switch 1's forwarding table
endpoint X: the link to endpoint X

Sending data to endpoint X



Application → transport: "I want to send data to process A at **endpoint X**."

Transport → network: "Deliver this segment/ack to **endpoint X**."

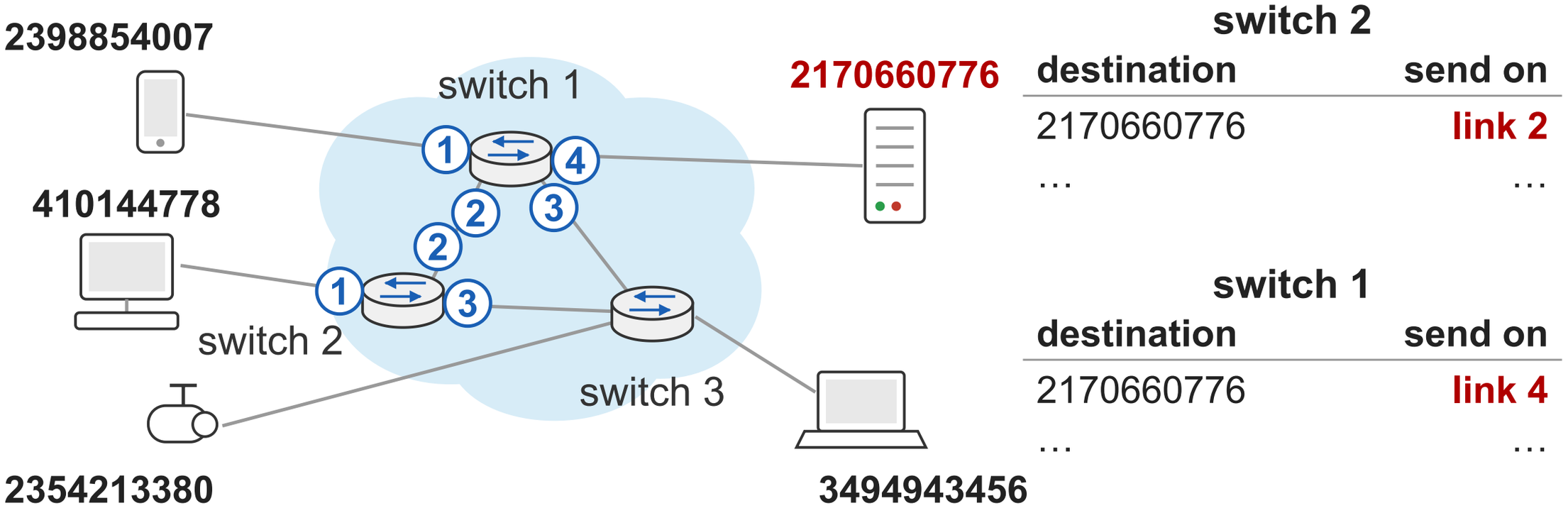
switch 2's forwarding table
endpoint X: the link to switch 1

switch 1's forwarding table
endpoint X: the link to **endpoint X**

We need a globally unique identifier for each endpoint.

A unique number per endpoint

- A reasonable choice: give every endpoint a 32-bit integer.



switch 2

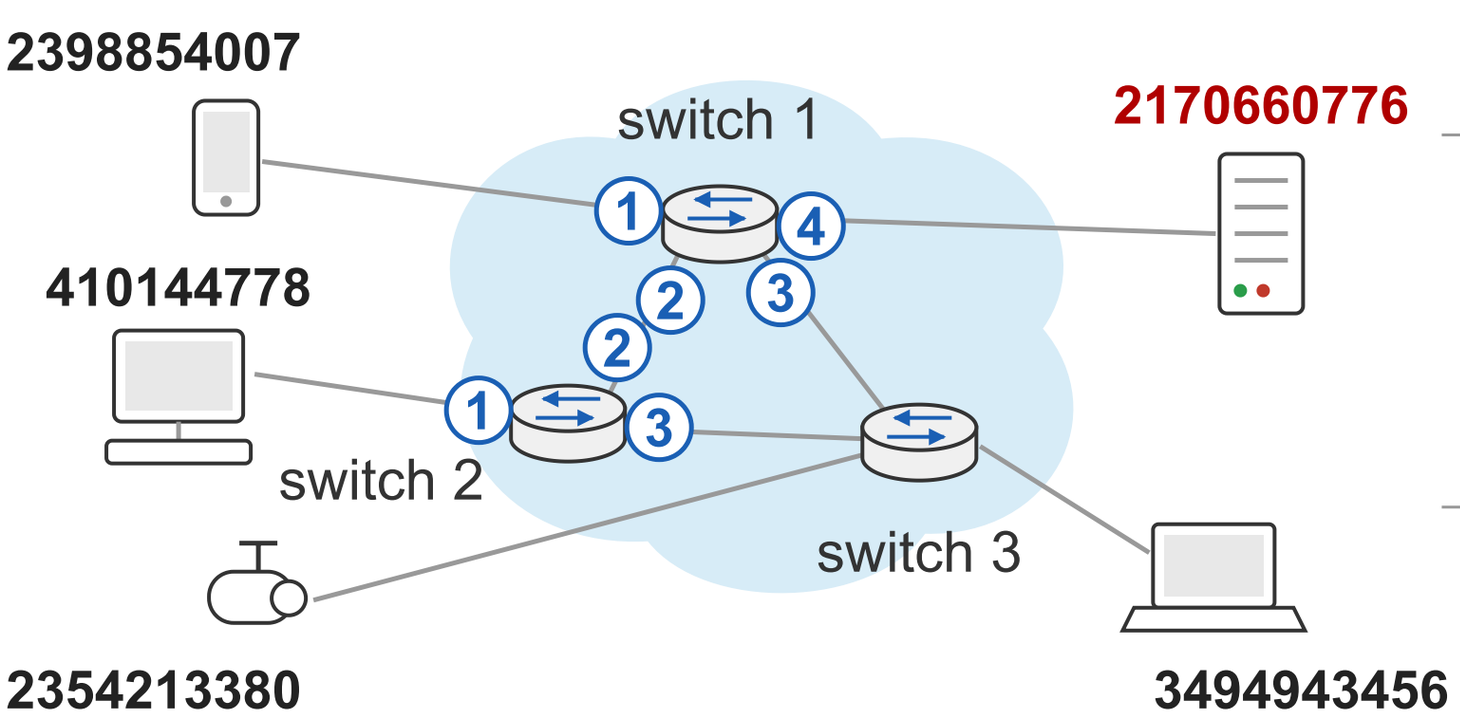
destination	send on
2170660776	link 2
…	…

switch 1

destination	send on
2170660776	link 4
…	…

A unique number per endpoint

- A reasonable choice: give every endpoint a 32-bit integer. This is an IPv4 address.



switch 2

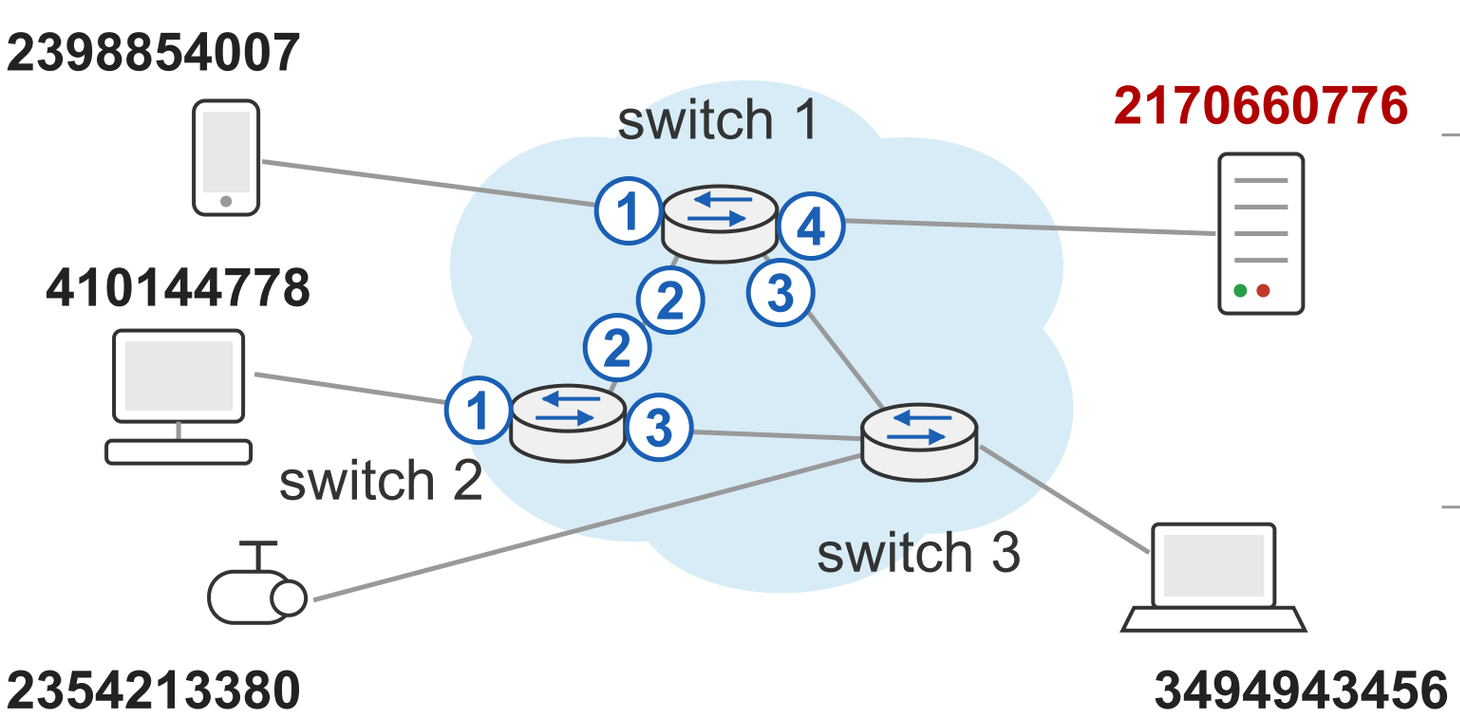
destination	send on
2170660776	link 2
…	…

switch 1

destination	send on
2170660776	link 4
…	…

A unique number per endpoint

- A reasonable choice: give every endpoint a 32-bit integer. This is an IPv4 address.
- Who assigns these numbers, and how: later, when we look at how to scale routing.



switch 2

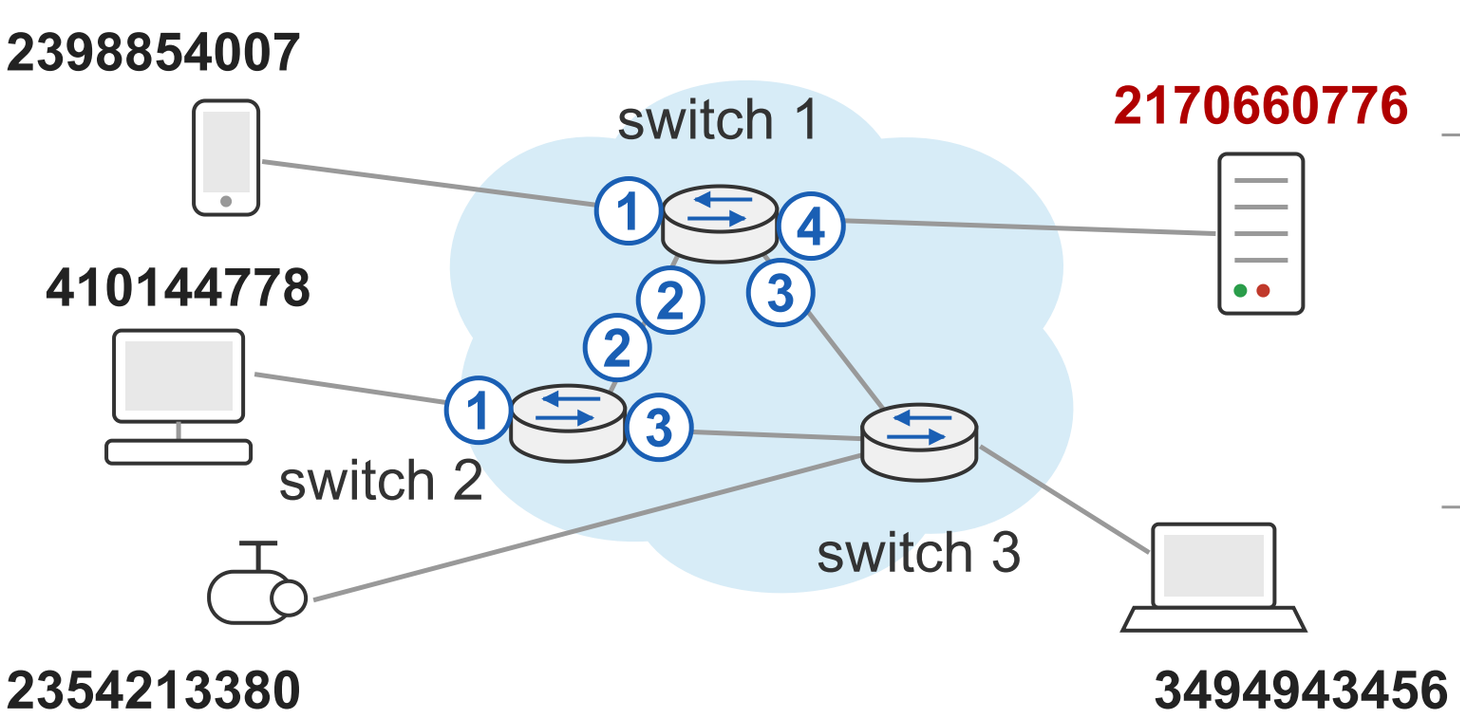
destination	send on
2170660776	link 2
…	…

switch 1

destination	send on
2170660776	link 4
…	…

A unique number per endpoint

- A reasonable choice: give every endpoint a 32-bit integer. This is an IPv4 address.
- Who assigns these numbers, and how: later, when we look at how to scale routing.
- For now, suppose every endpoint has one.



switch 2

destination	send on
2170660776	link 2
…	…

switch 1

destination	send on
2170660776	link 4
…	…

IPv4 addresses

32-bit integer	**`2170660776`**

IPv4 addresses

32-bit integer **2170660776**

in binary 10000001 01100001 10100111 10101000

IPv4 addresses

32-bit integer	**2170660776**			
in binary	10000001	01100001	10100111	10101000
each byte in decimal	129	97	167	168

IPv4 addresses

32-bit integer	**2170660776**			
in binary	10000001	01100001	10100111	10101000
each byte in decimal	129	97	167	168
dotted decimal	**129.97.167.168**			

IPv4 addresses

32-bit integer	**2170660776**			
in binary	10000001	01100001	10100111	10101000
each byte in decimal	129	97	167	168
dotted decimal	**129.97.167.168**			

- Each byte is between 0 and 255: from 0.0.0.0 to 255.255.255.255: about 4.3 billion addresses.

IPv4 addresses

32-bit integer	**2170660776**			
in binary	10000001	01100001	10100111	10101000
each byte in decimal	129	97	167	168
dotted decimal	**129.97.167.168**			

- Each byte is between 0 and 255: from 0.0.0.0 to 255.255.255.255: about 4.3 billion addresses.
- Turns out, 4.3 billion was not enough. IPv6 addresses, as such, are 128 bits. More on that later.

How do you find the address you want?



How do you find the address you want?



Application → transport:
“Send this to the web server (port 443) at **???**”

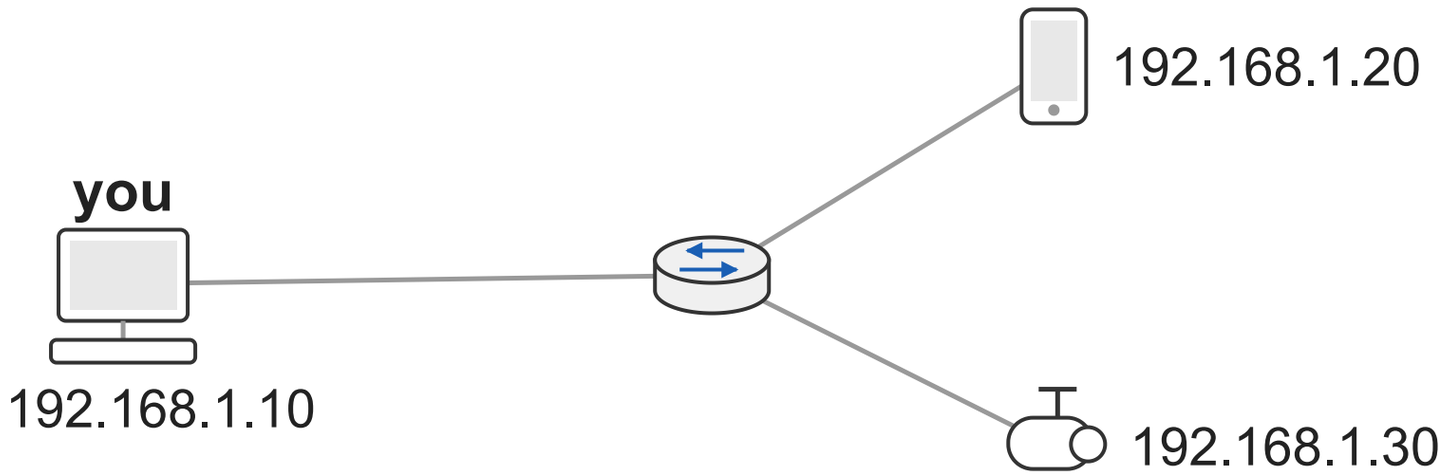
How do you find the address you want?



Application → transport:
"Send this to the web server (port 443) at **???**"

How do you find the address of the endpoint you want?

Remember the addresses?



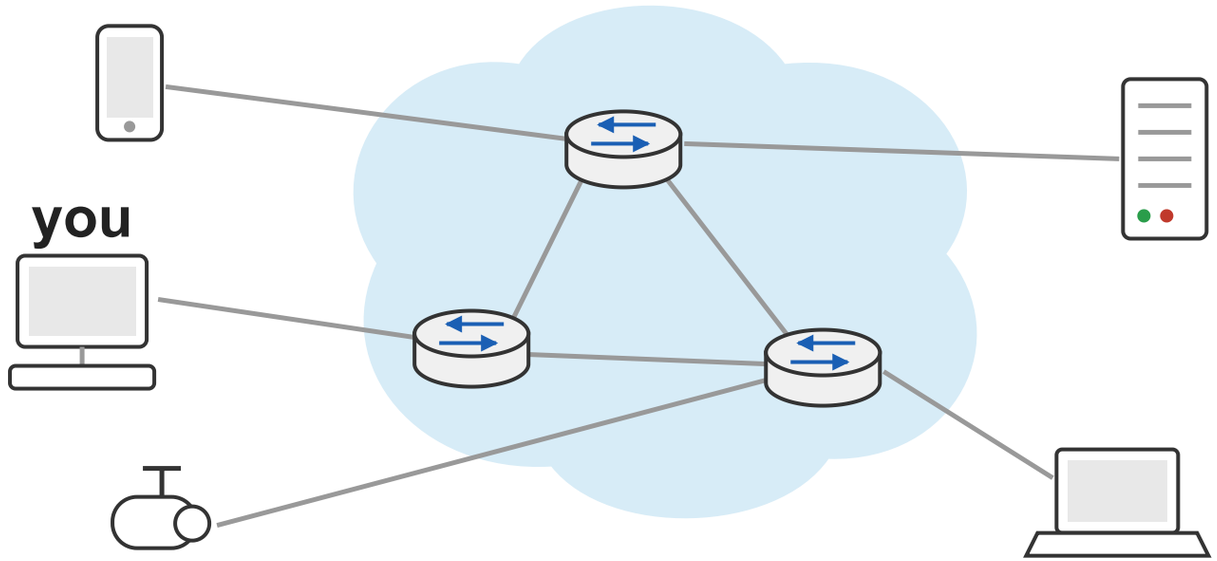
A small network at home

in your head:

phone: …1.20

camera: …1.30

Remember the addresses?



… and billions more endpoints

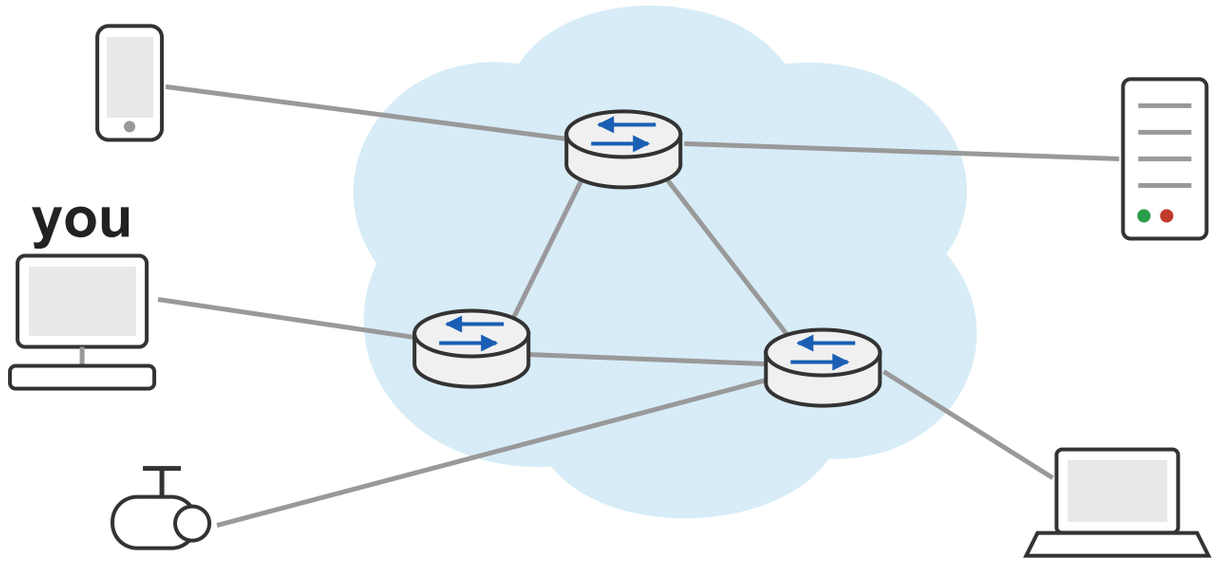
http://129.97.167.168

Pages you visited today:

search	142.251.155.119
encyclopedia	208.80.154.224
course page	129.97.167.168
code hosting	140.82.114.4

… hundreds more

Remember the addresses?



… and billions more endpoints

http://129.97.167.168

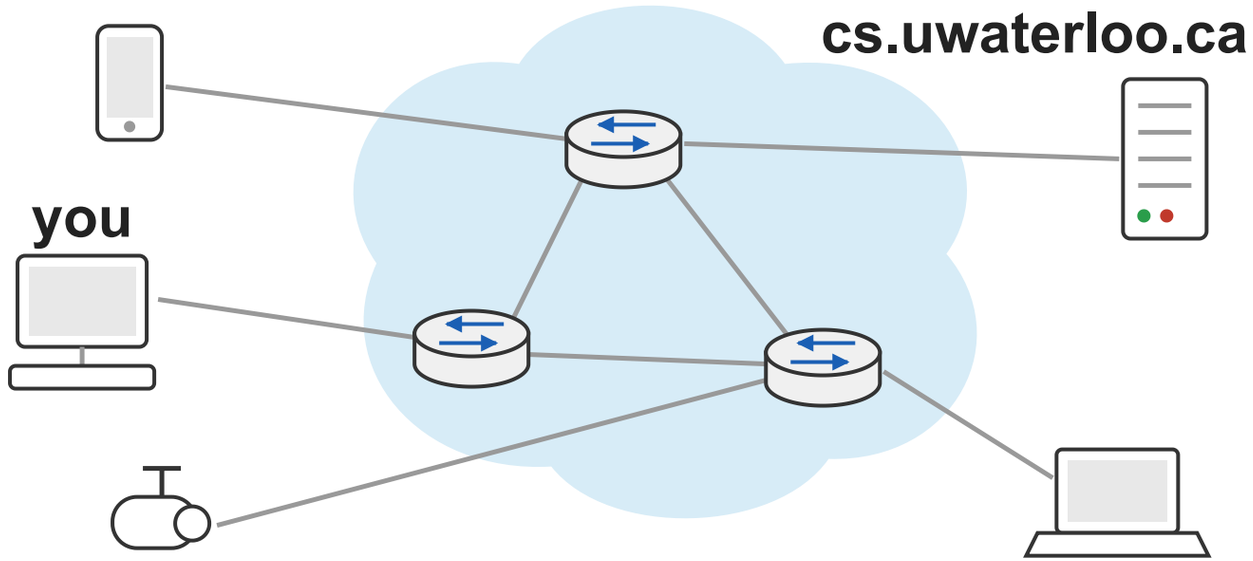
Pages you visited today:

search	142.251.155.119
encyclopedia	208.80.154.224
course page	129.97.167.168
code hosting	140.82.114.4

… hundreds more

Billions of endpoints: remembering their addresses does not scale.

A registry that maps names to addresses

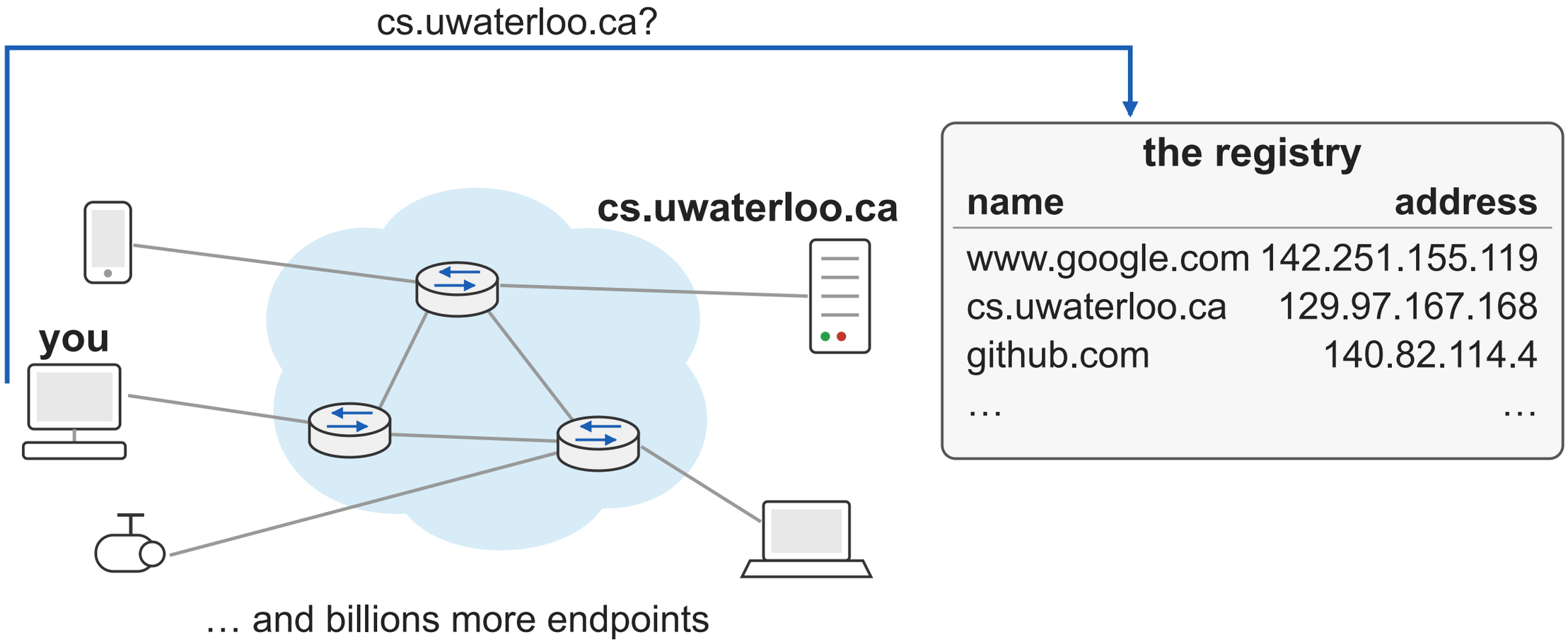


… and billions more endpoints

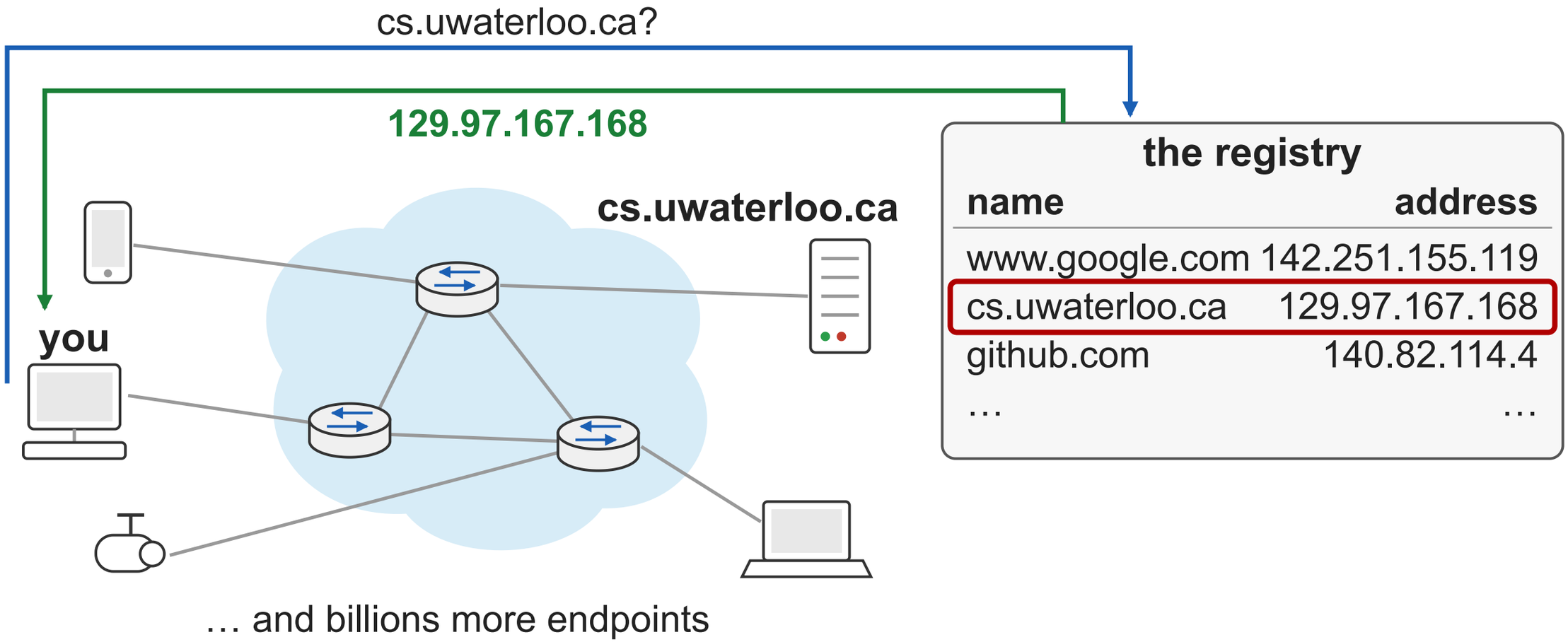
the registry

name	address
www.google.com	142.251.155.119
cs.uwaterloo.ca	129.97.167.168
github.com	140.82.114.4
…	…

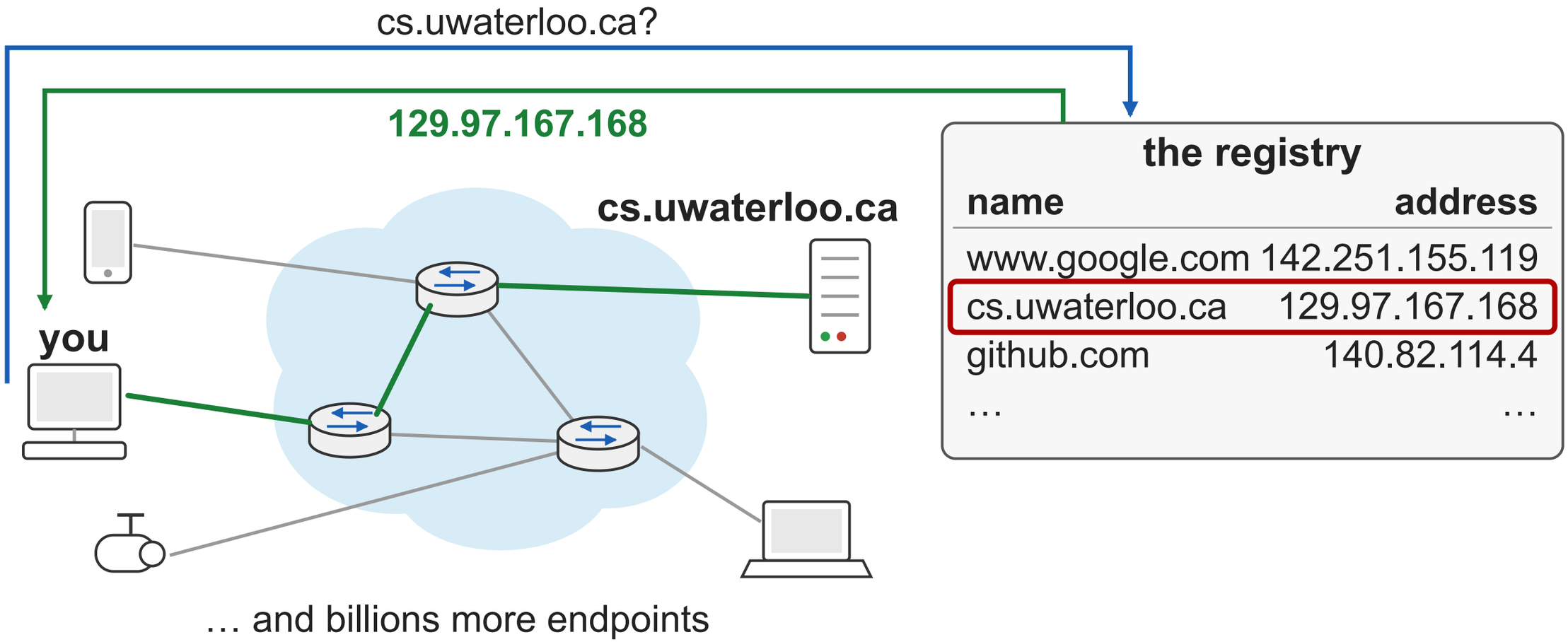
A registry that maps names to addresses



A registry that maps names to addresses



A registry that maps names to addresses



Building the registry

- Fun fact: the registry used to be a file you could download.
  - Until the mid-1980s, every name on the ARPANET (the network the Internet started from) was in one file, HOSTS.TXT, kept at SRI in California.
  - Every host downloaded a fresh copy regularly.
- But the Internet has become so big that this registry is a whole system of its own: the Domain Name System (DNS).

What properties should this registry have?

What properties should this registry have?

Scalable	Billions of endpoints, ~400 million registered names, trillions of lookups a day.

What properties should this registry have?

Scalable	Billions of endpoints, ~400 million registered names, trillions of lookups a day.
Answer quickly	The application cannot start a connection without an answer from the registry.

What properties should this registry have?

Scalable	Billions of endpoints, ~400 million registered names, trillions of lookups a day.
Answer quickly	The application cannot start a connection without an answer from the registry.
High availability	If it does not answer, the Internet is down for its users. No single point of failure; keep answering under failures, load spikes and attacks.

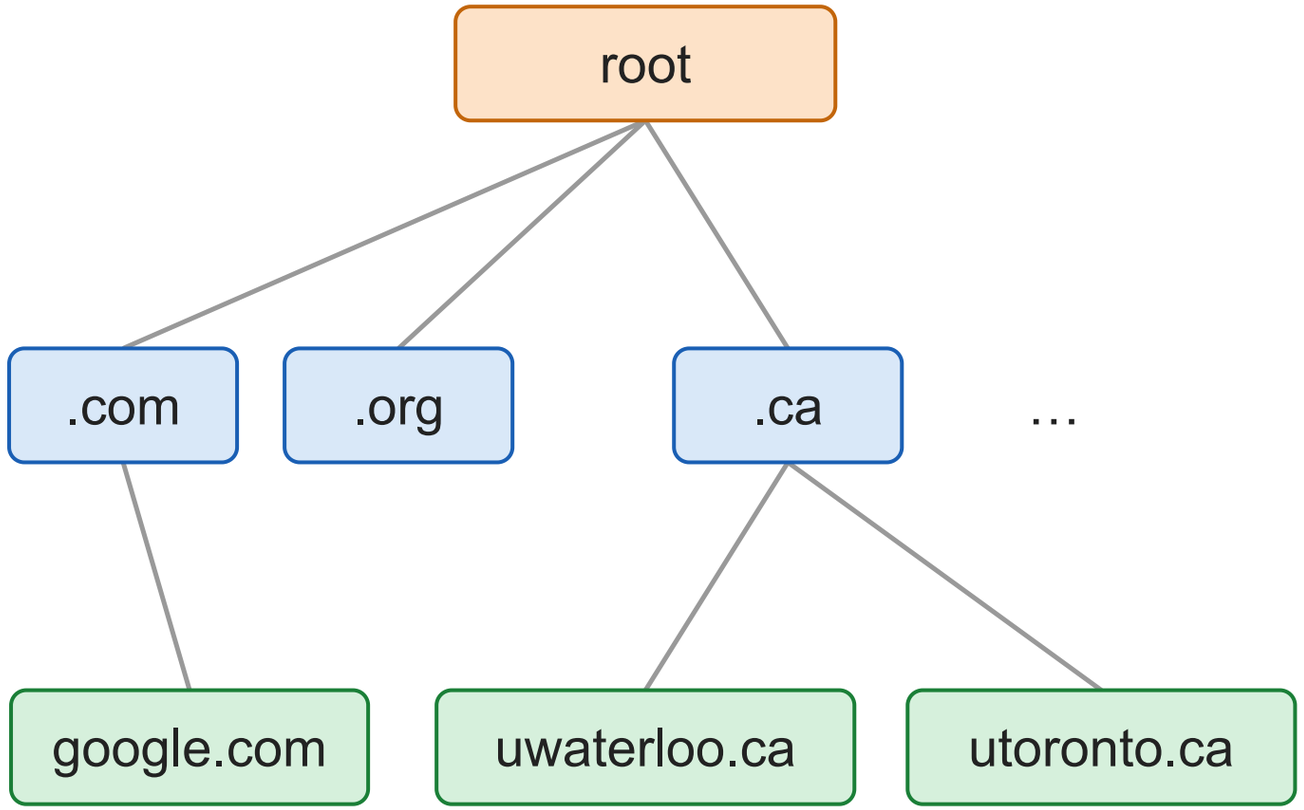
What properties should this registry have?

Scalable	Billions of endpoints, ~400 million registered names, trillions of lookups a day.
Answer quickly	The application cannot start a connection without an answer from the registry.
High availability	If it does not answer, the Internet is down for its users. No single point of failure; keep answering under failures, load spikes and attacks.
Decentralized	No single organization runs it. Each owner manages its own names: only the university decides what is under uwaterloo.ca.

What properties should this registry have?

Scalable	Billions of endpoints, ~400 million registered names, trillions of lookups a day.
Answer quickly	The application cannot start a connection without an answer from the registry.
High availability	If it does not answer, the Internet is down for its users. No single point of failure; keep answering under failures, load spikes and attacks.
Decentralized	No single organization runs it. Each owner manages its own names: only the university decides what is under uwaterloo.ca.
Secure	If the provided IP address is wrong, by mistake or on purpose, you will end up connecting to the wrong endpoint.

DNS: a distributed, hierarchical registry



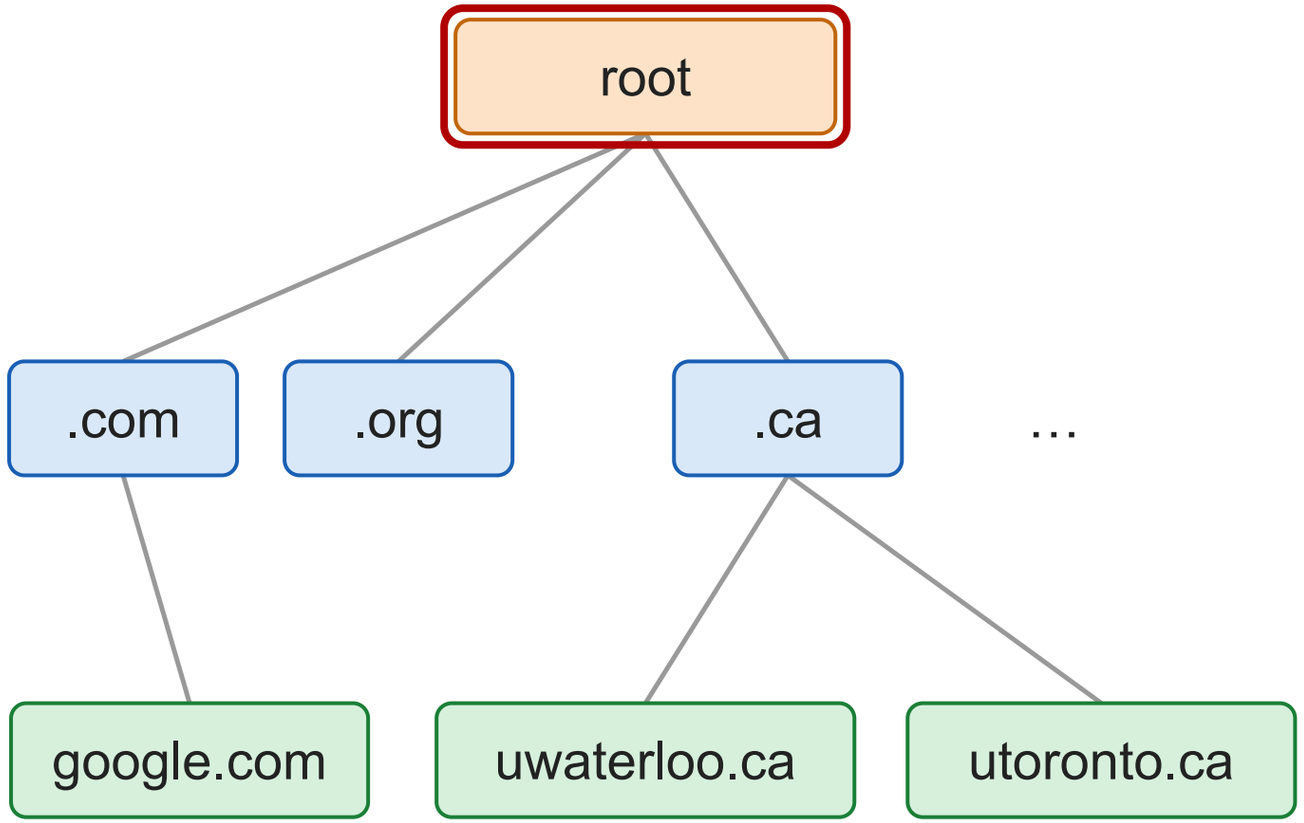
Root servers
13 root name servers, ~2,000 operational instances, run by 12 different organizations

TLD servers
~1,400 top-level domains (.com, .ca, .org, …), each with its own servers, run by one organization, its “registry” (.ca: the Canadian Internet Registration Authority, CIRA)

Authoritative servers
an organization’s actual records, run by it or by a DNS provider

Each box represents a set of servers that are part of the registry.

DNS: a distributed, hierarchical registry



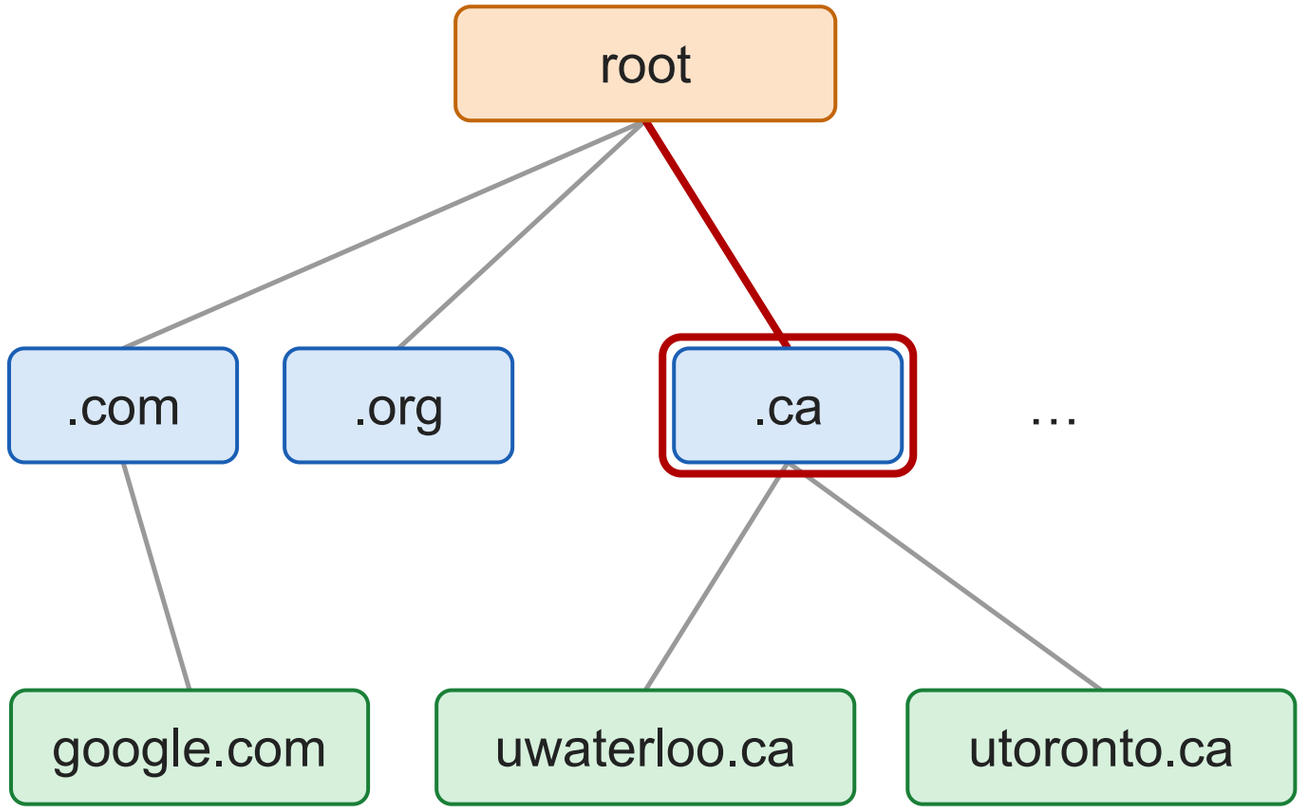
Root servers
13 root name servers, ~2,000 operational instances, run by 12 different organizations

TLD servers
~1,400 top-level domains (.com, .ca, .org, …), each with its own servers, run by one organization, its “registry” (.ca: the Canadian Internet Registration Authority, CIRA)

Authoritative servers
an organization’s actual records, run by it or by a DNS provider

Looking up cs.uwaterloo.ca. Root: “I don’t know it, but here is a .ca server.”

DNS: a distributed, hierarchical registry



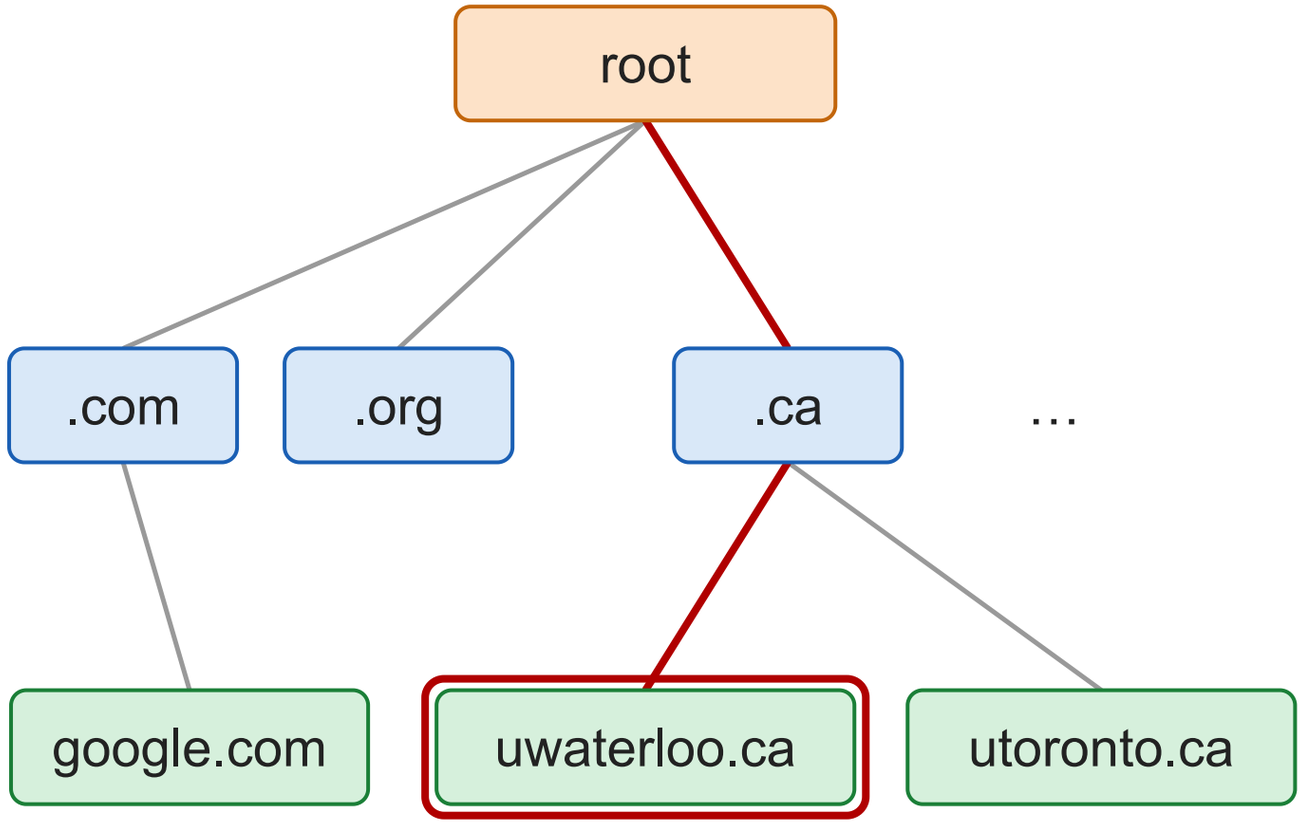
Root servers
13 root name servers, ~2,000 operational instances, run by 12 different organizations

TLD servers
~1,400 top-level domains (.com, .ca, .org, …), each with its own servers, run by one organization, its “registry” (.ca: the Canadian Internet Registration Authority, CIRA)

Authoritative servers
an organization’s actual records, run by it or by a DNS provider

The .ca server: “I don’t know it either, but here is a uwaterloo.ca server.”

DNS: a distributed, hierarchical registry



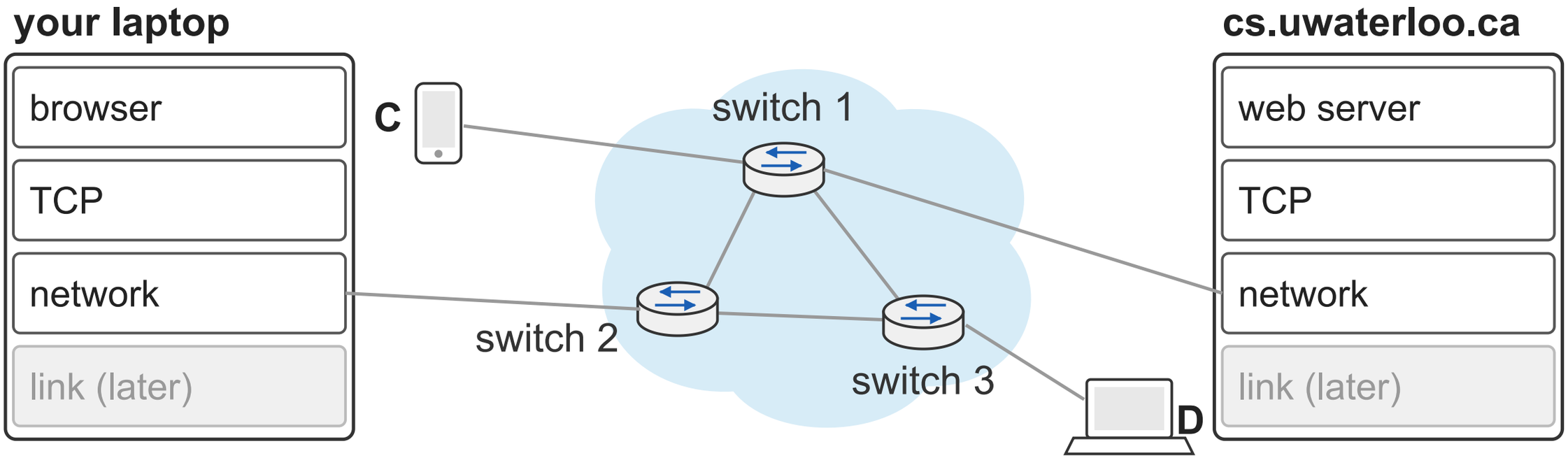
Root servers
13 root name servers, ~2,000 operational instances, run by 12 different organizations

TLD servers
~1,400 top-level domains (.com, .ca, .org, …), each with its own servers, run by one organization, its “registry” (.ca: the Canadian Internet Registration Authority, CIRA)

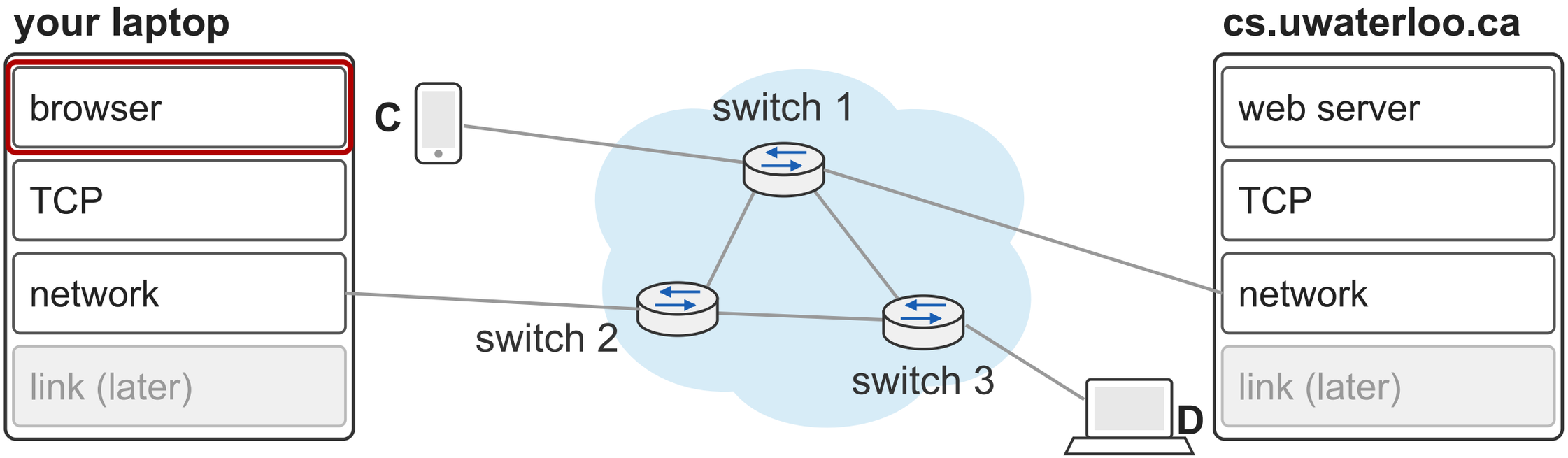
Authoritative servers
an organization’s actual records, run by it or by a DNS provider

The uwaterloo.ca server: “cs.uwaterloo.ca is 129.97.167.168.”

Where does the address come from?

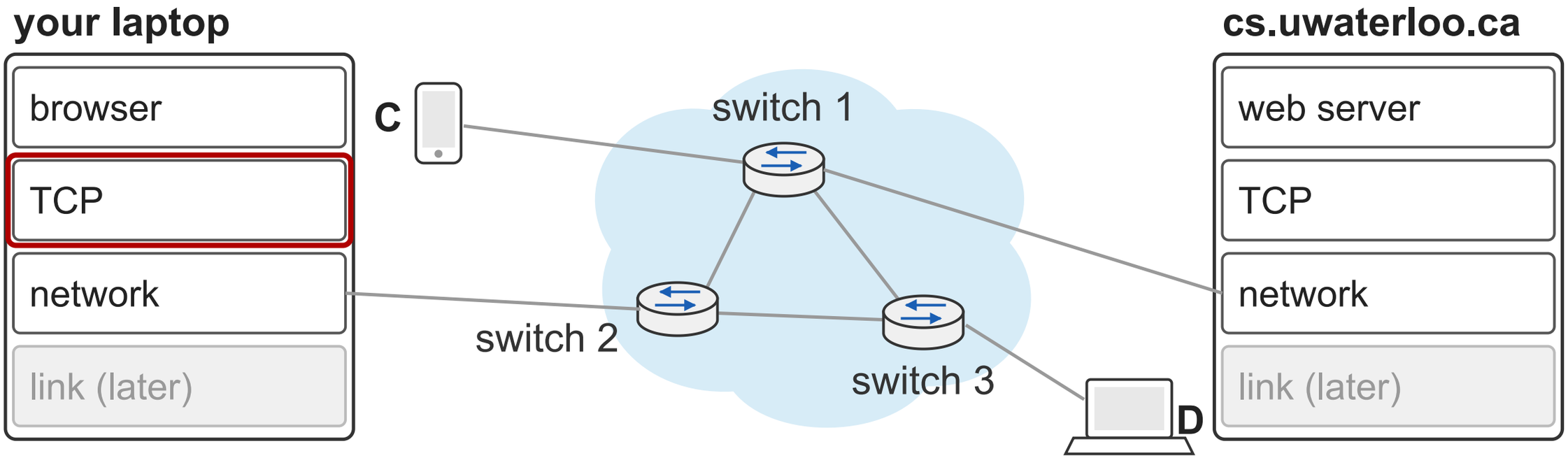


Where does the address come from?



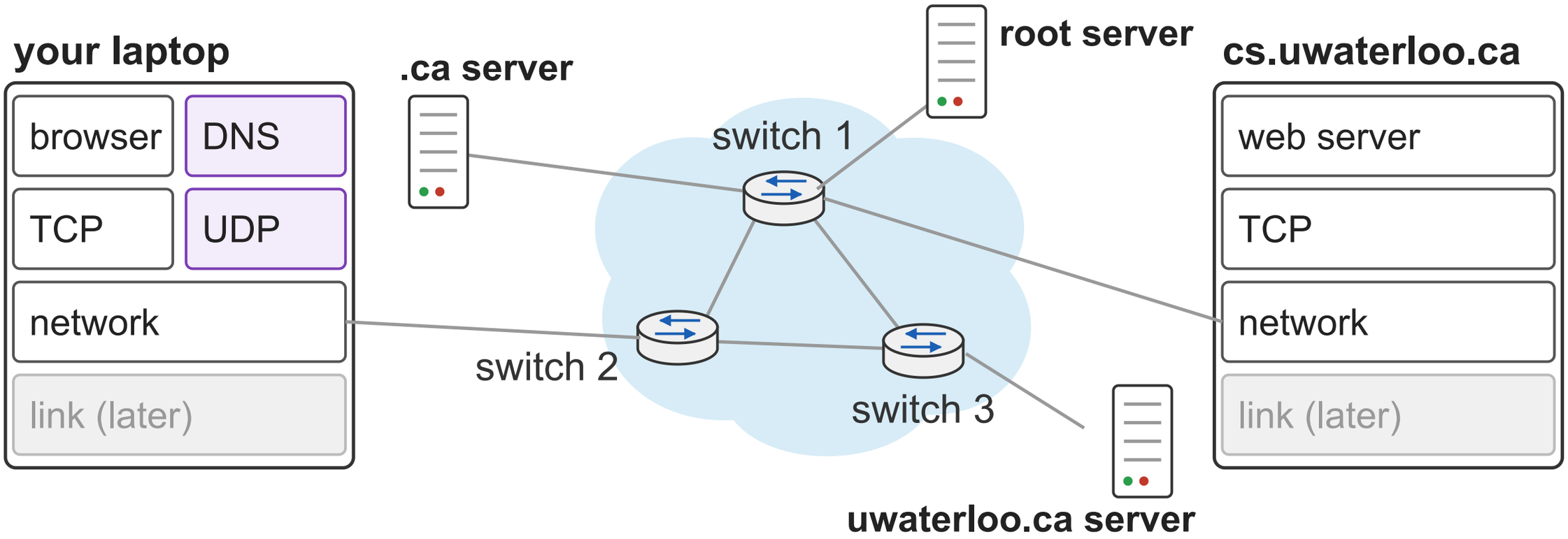
You type cs.uwaterloo.ca into the browser.

Where does the address come from?

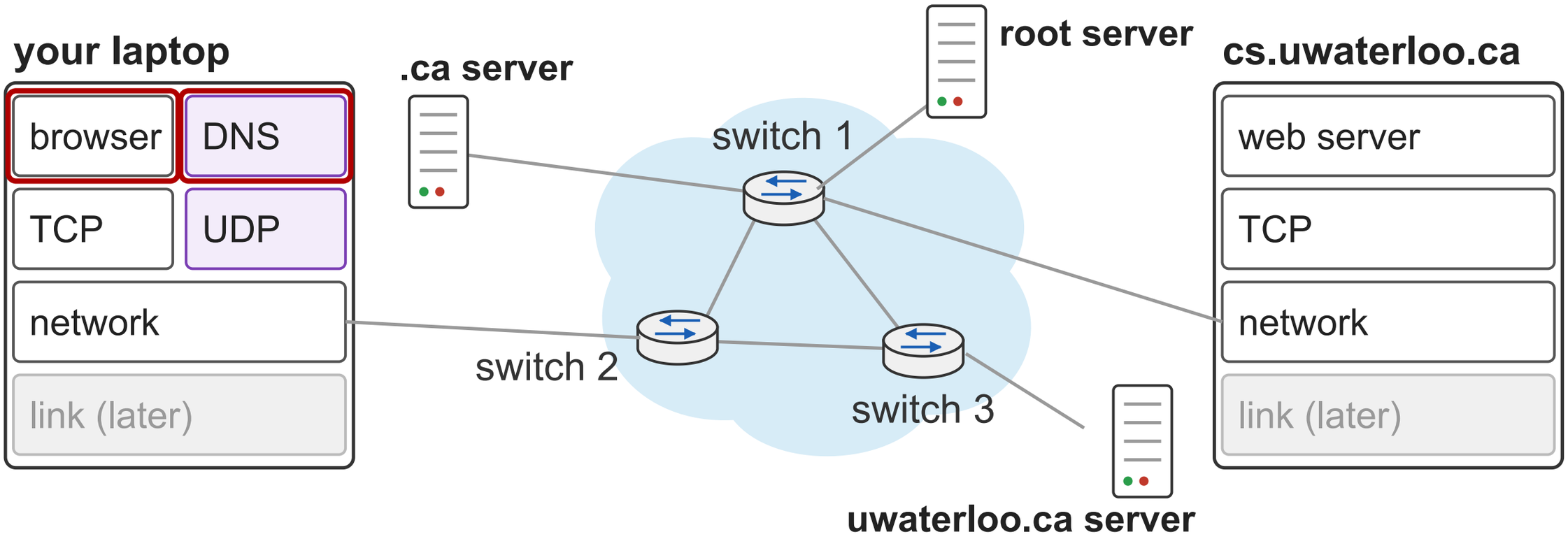


Before TCP can ask the network to deliver anything, it needs cs.uwaterloo.ca's address. Where does it get it?

DNS is an application, over UDP

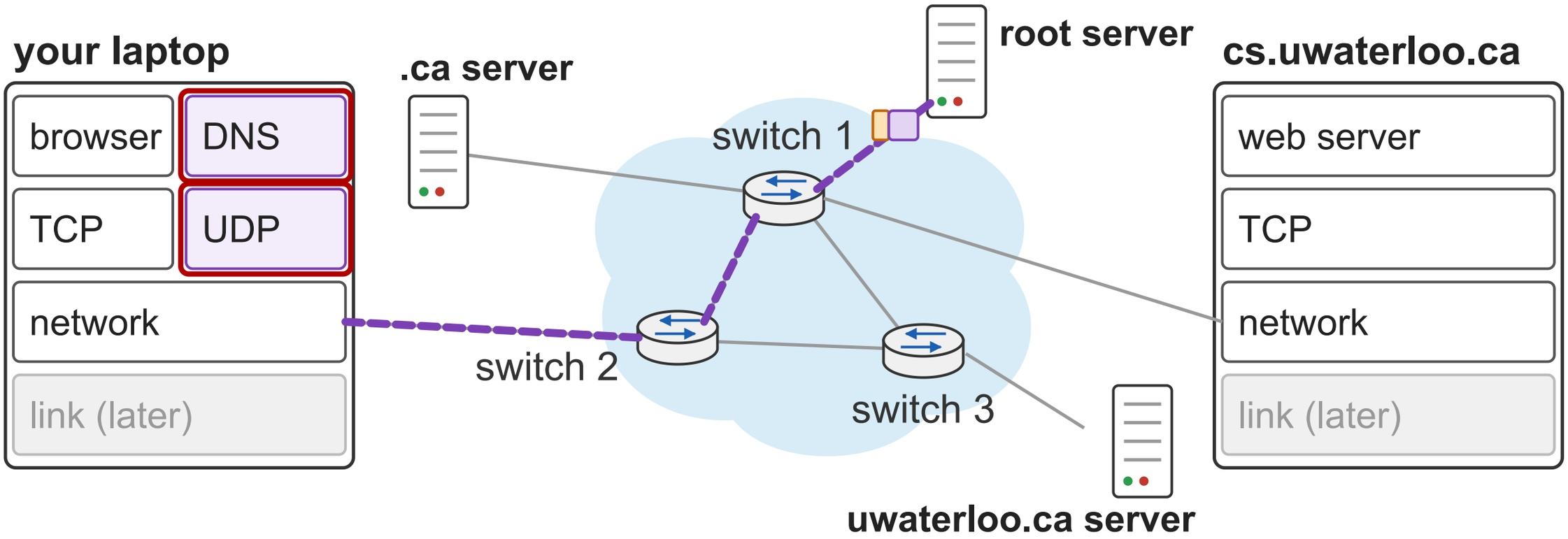


DNS is an application, over UDP



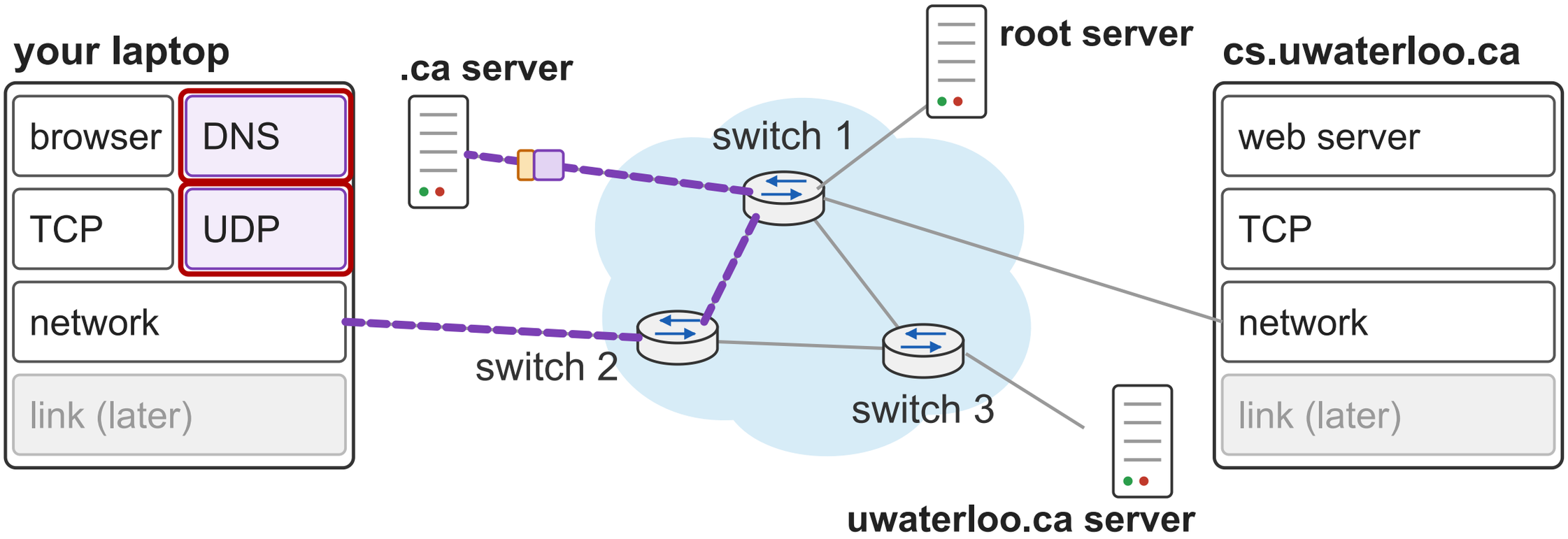
The browser asks the laptop's DNS client for cs.uwaterloo.ca's address.

DNS is an application, over UDP



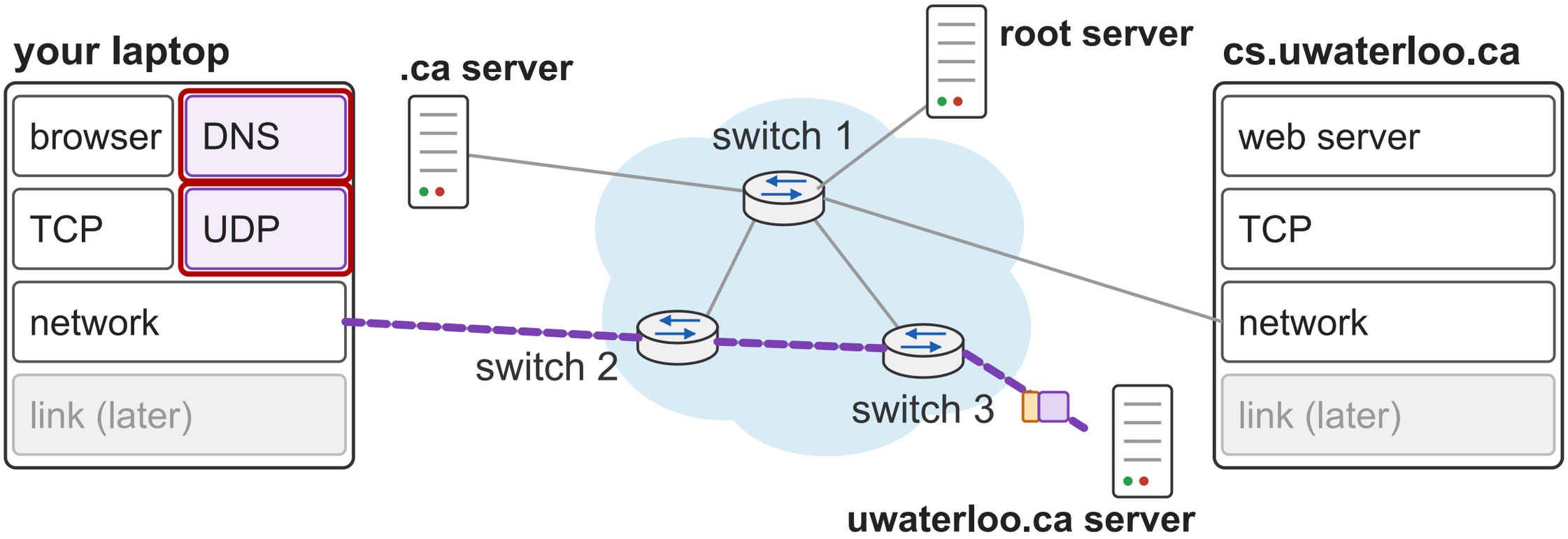
The DNS client asks a root server, in a UDP packet. The answer: “I don’t know, but here is a .ca server.”

DNS is an application, over UDP



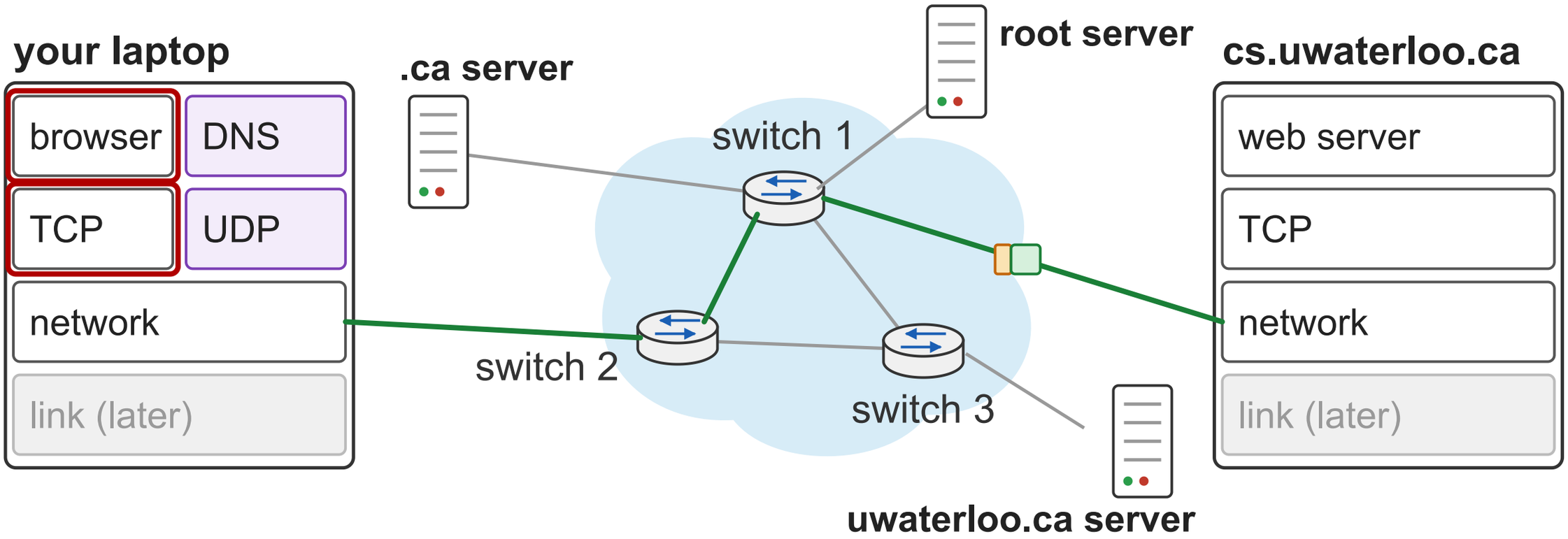
It asks the .ca server: “I don’t know either, but here is a uwaterloo.ca server.”

DNS is an application, over UDP



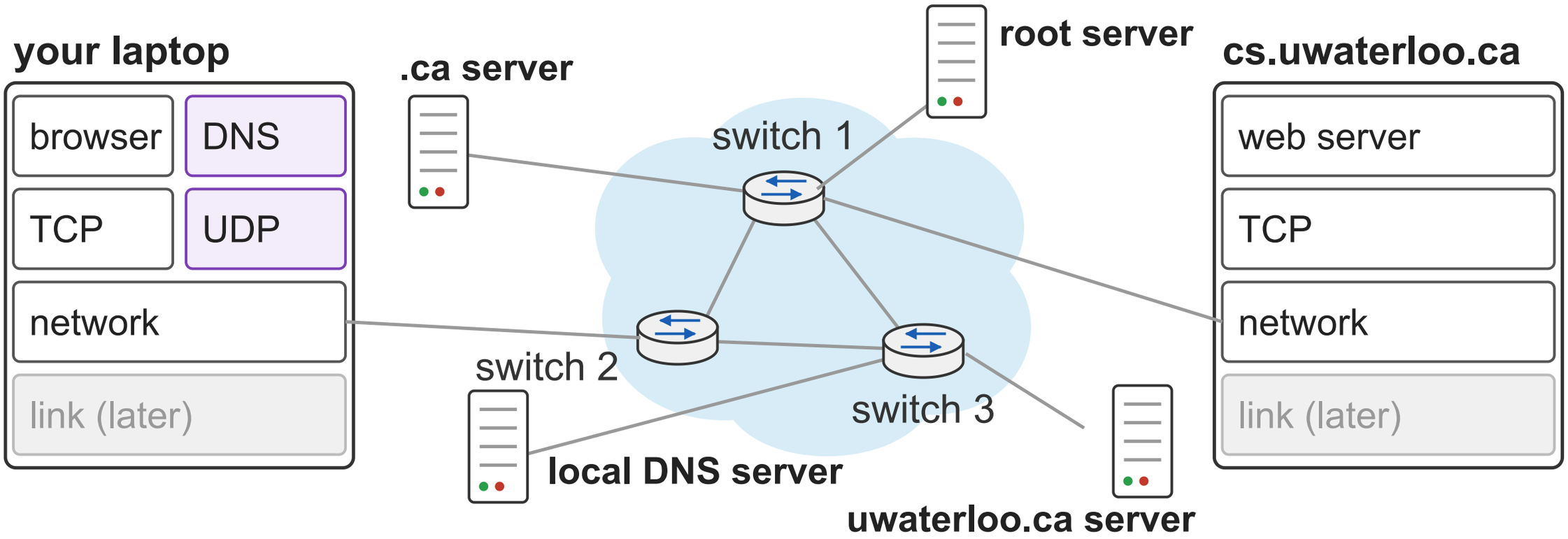
It asks the uwaterloo.ca server: “cs.uwaterloo.ca is 129.97.167.168.”

DNS is an application, over UDP



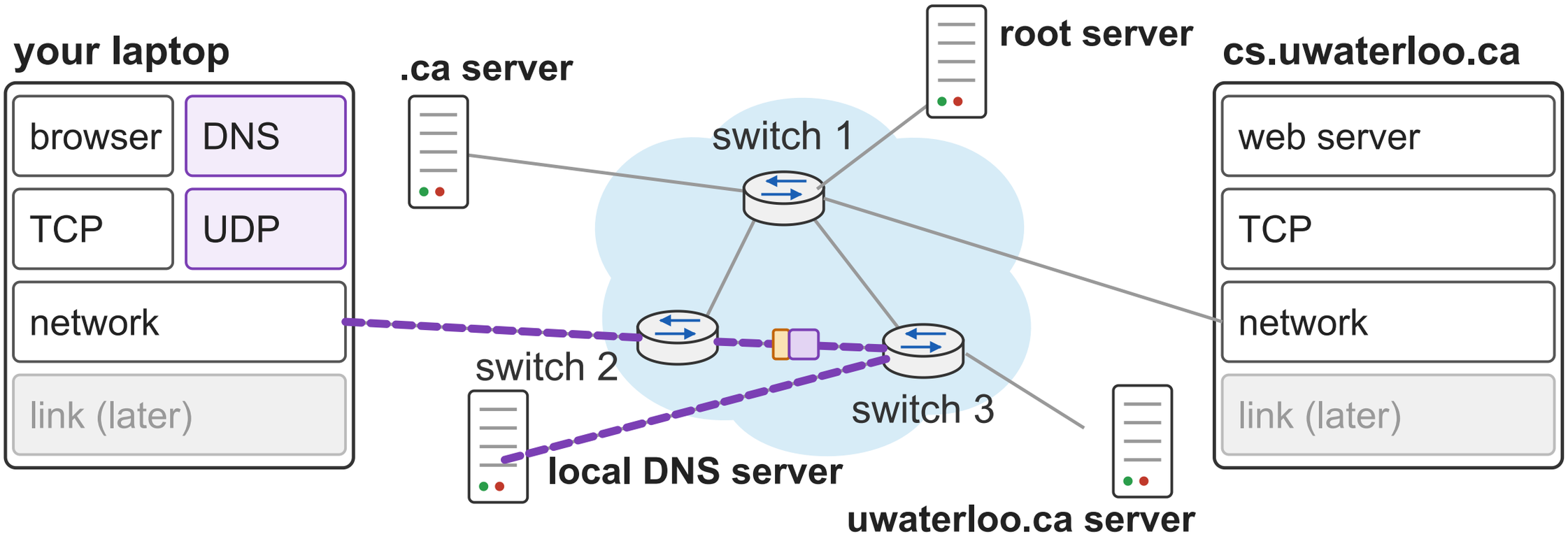
Now the browser can ask TCP for a channel to the web server at 129.97.167.168.

Local DNS servers



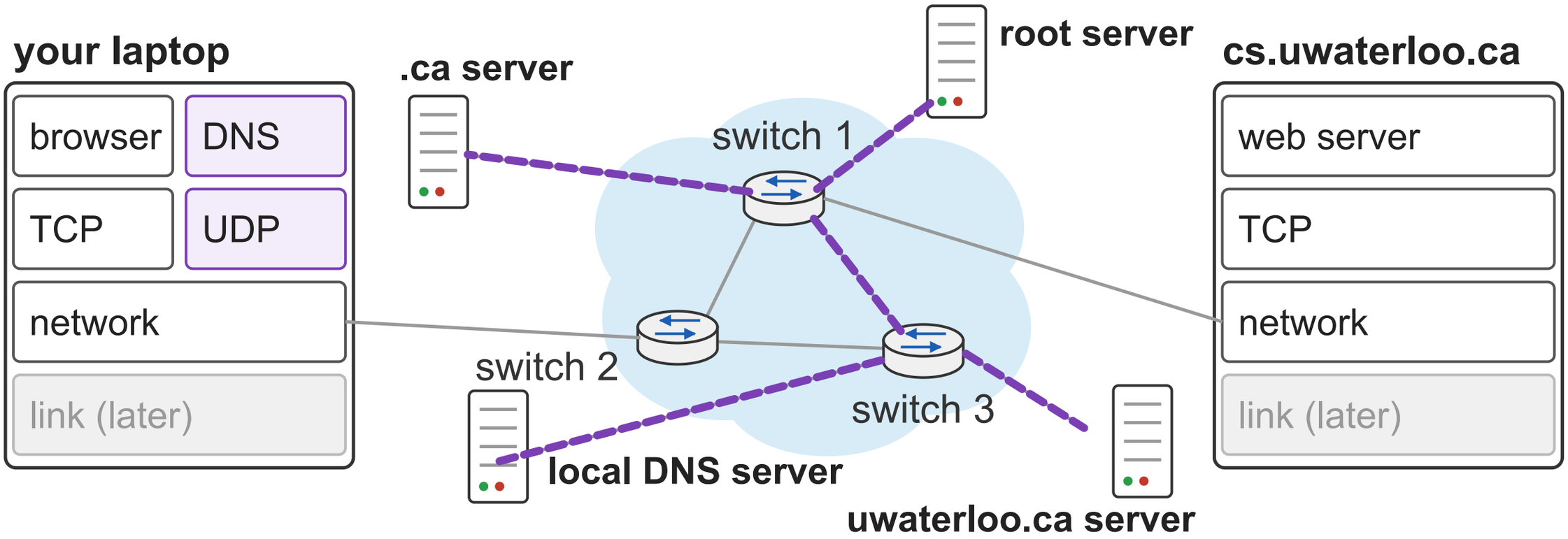
A local DNS server resolves the DNS query on the endpoint's behalf. Run by your ISP or campus network; there are even public ones (1.1.1.1, 8.8.8.8).

Local DNS servers



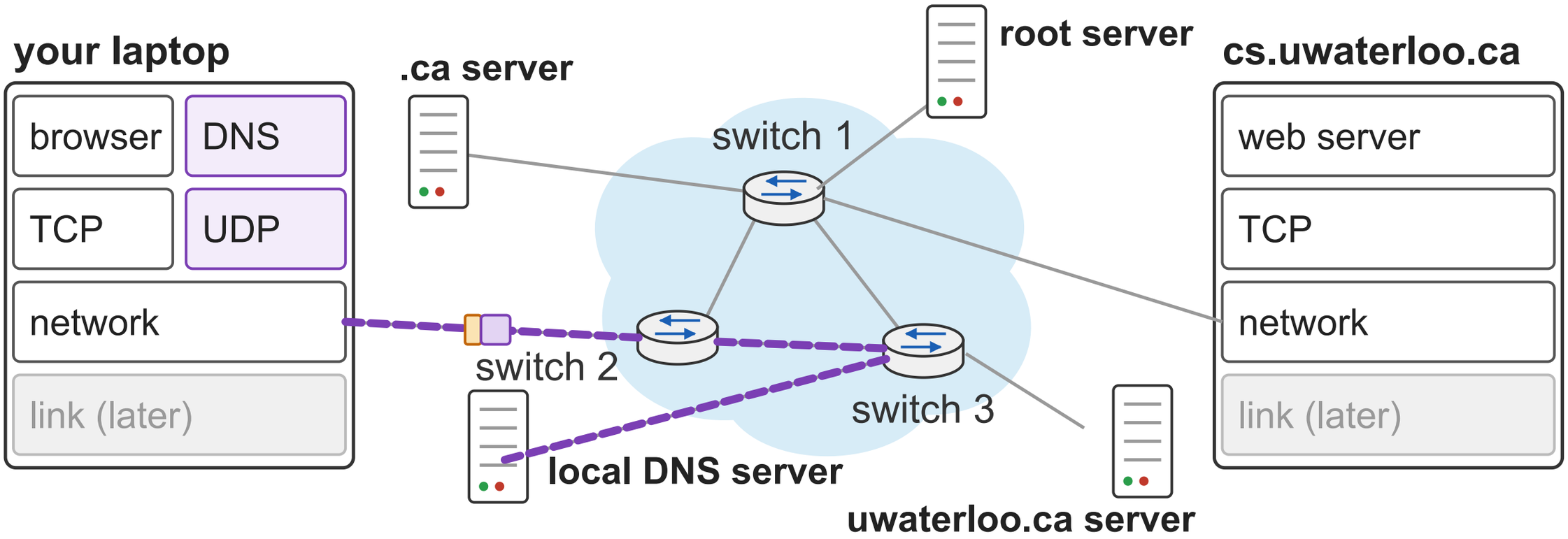
The laptop's DNS client sends one query to its local DNS server. Your endpoint finds out which one to use when it joins a network (more on that later).

Local DNS servers



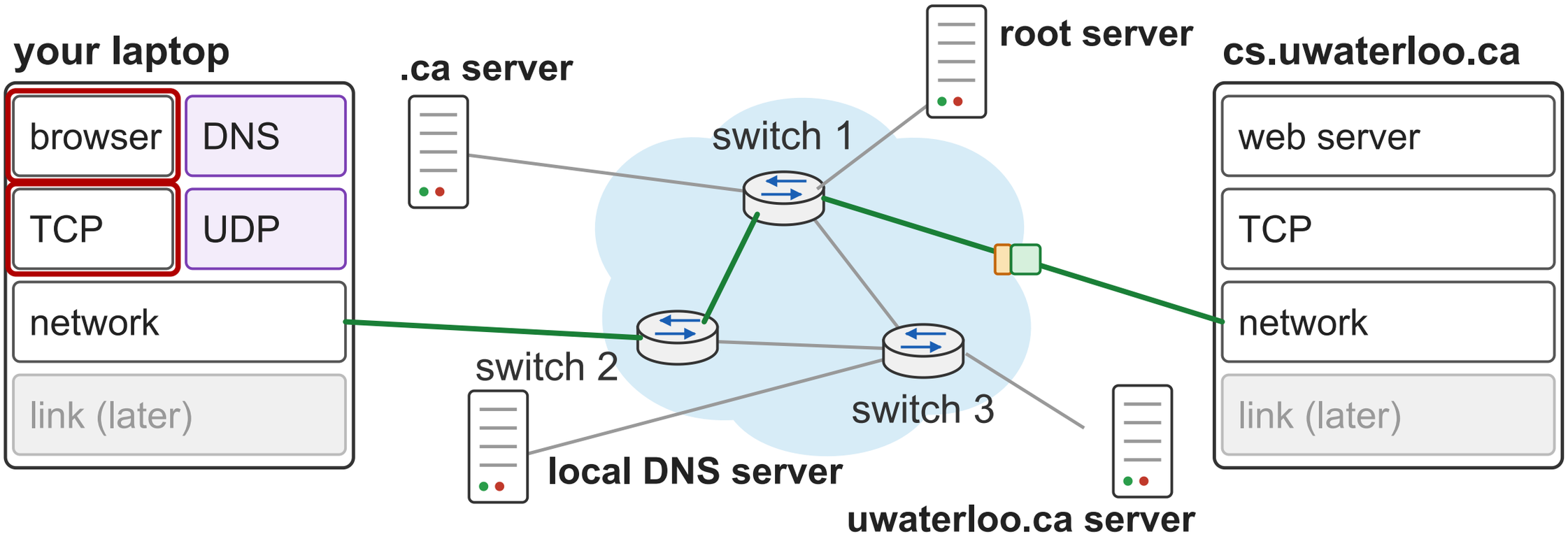
The local DNS server walks the tree for it: root, then .ca, then uwaterloo.ca.

Local DNS servers



It answers 129.97.167.168, and caches the answers it has learned. So the next lookup, from your laptop or anyone else using the same local DNS server, is resolved very quickly.

Local DNS servers



The browser asks TCP for a channel to the web server at 129.97.167.168.

Resolving a name: iterative queries

Iterative queries: each server answers with who to ask next.

a root server

.ca server



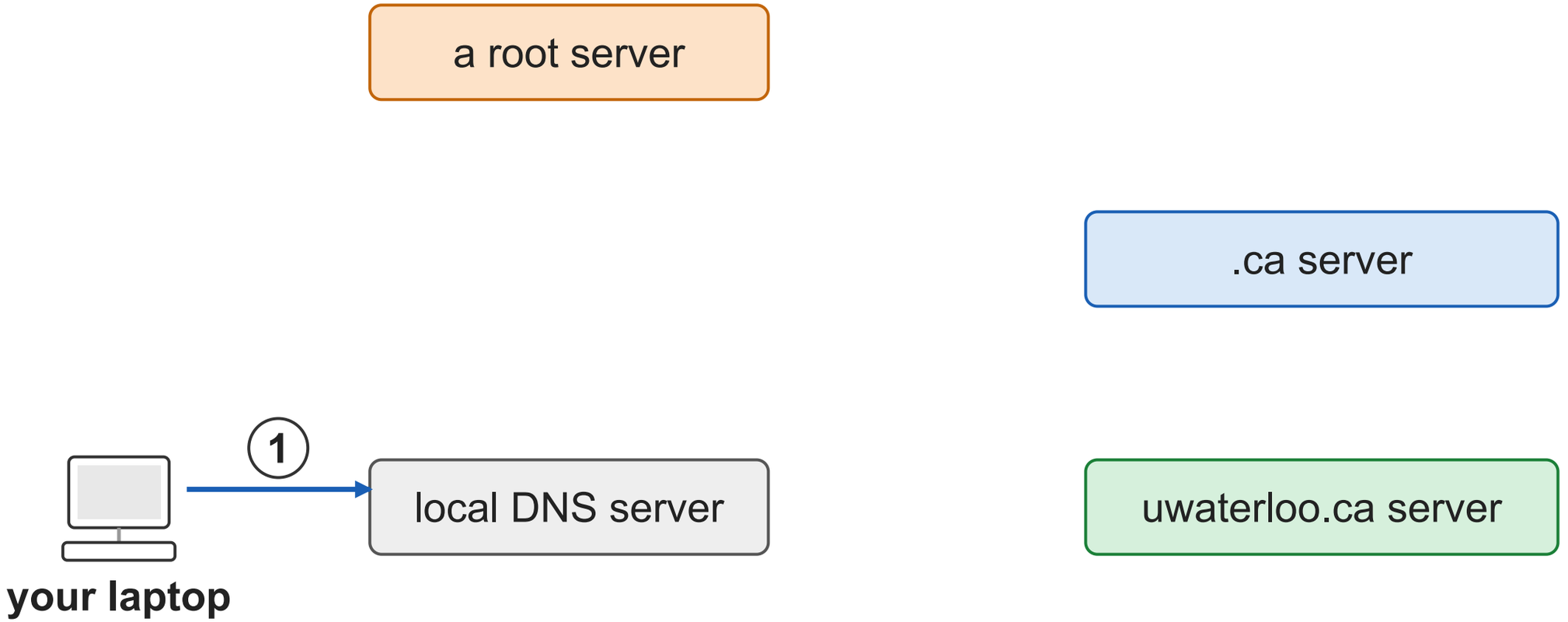
your laptop

local DNS server

uwaterloo.ca server

Resolving a name: iterative queries

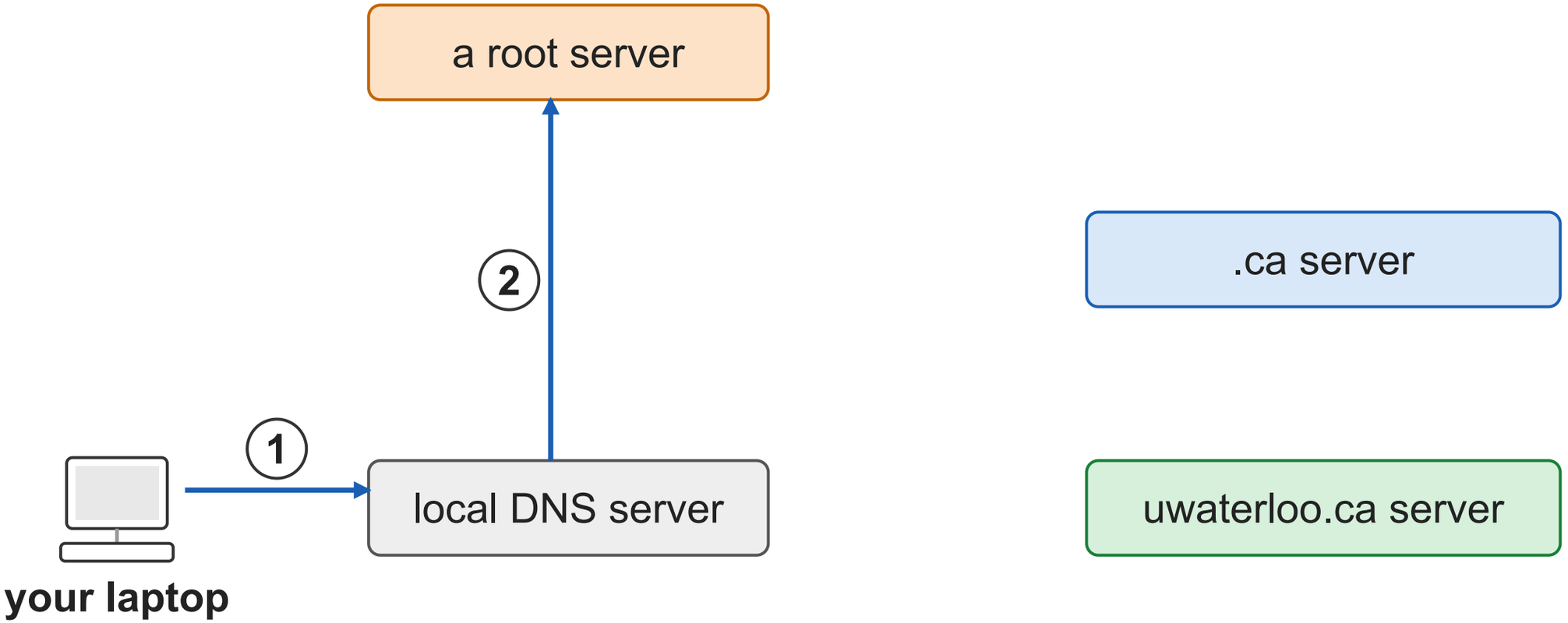
Iterative queries: each server answers with who to ask next.



Your laptop asks its local DNS server: cs.uwaterloo.ca?

Resolving a name: iterative queries

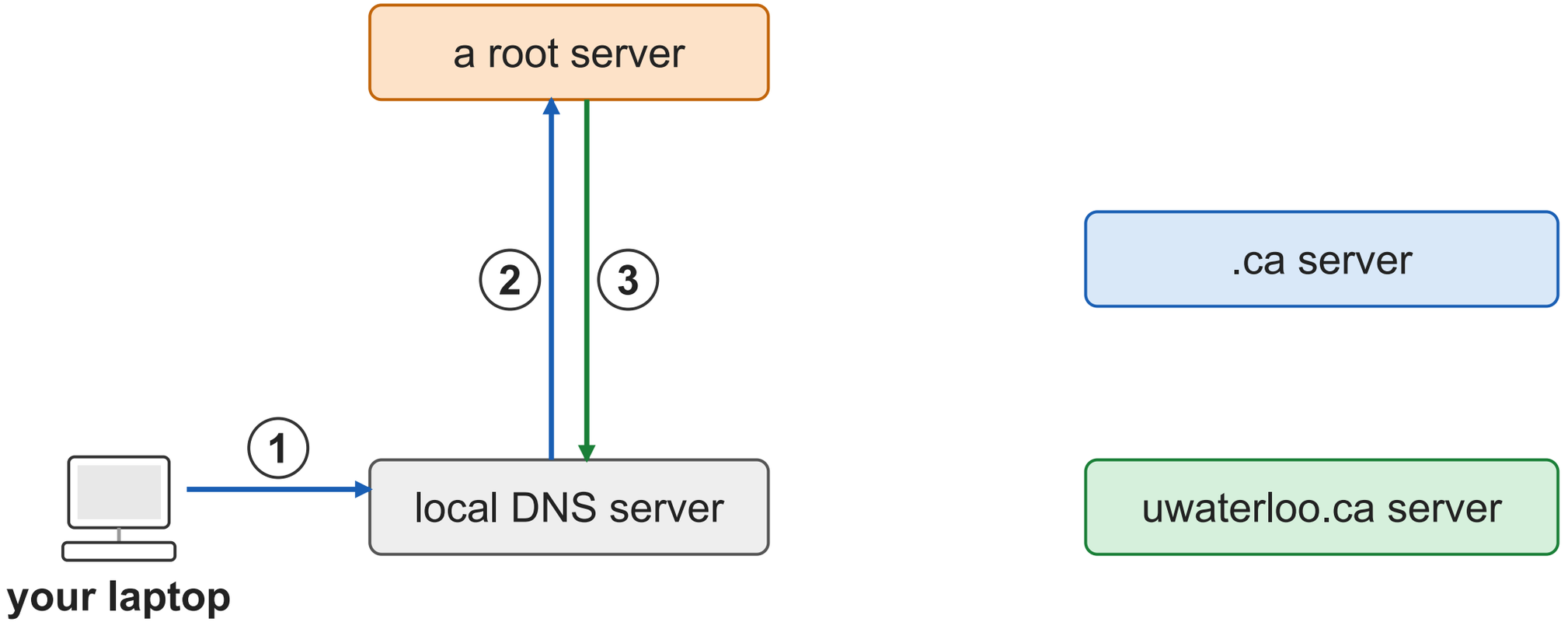
Iterative queries: each server answers with who to ask next.



The local DNS server asks a root server: cs.uwaterloo.ca?

Resolving a name: iterative queries

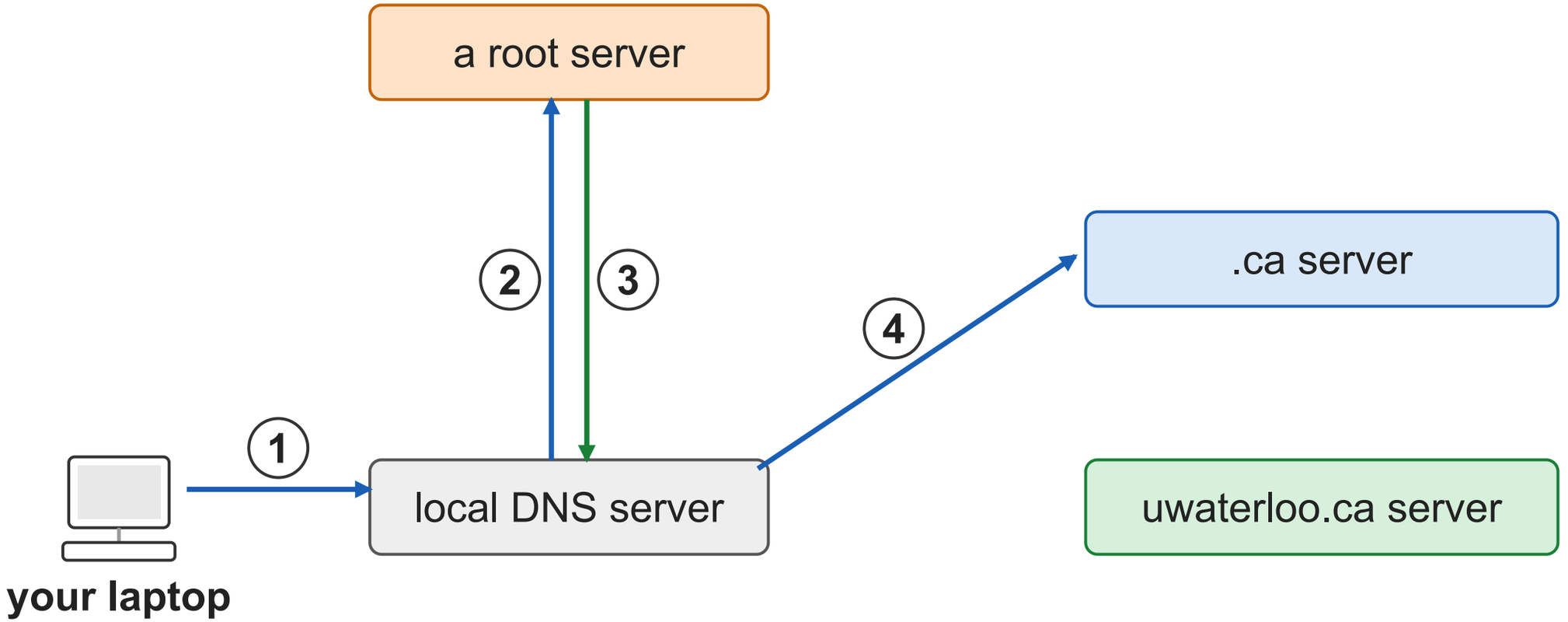
Iterative queries: each server answers with who to ask next.



Root: “Ask a .ca server, at 199.4.144.2.”

Resolving a name: iterative queries

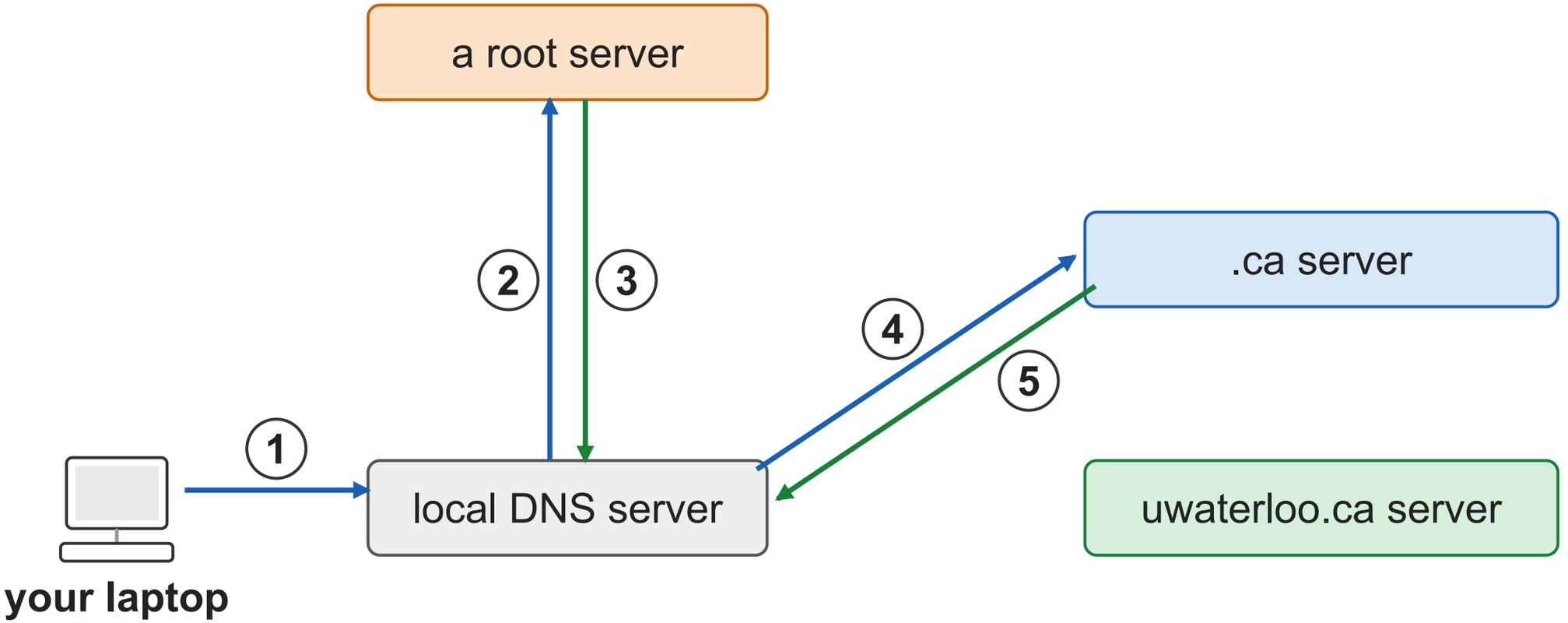
Iterative queries: each server answers with who to ask next.



The local DNS server asks the .ca server: cs.uwaterloo.ca?

Resolving a name: iterative queries

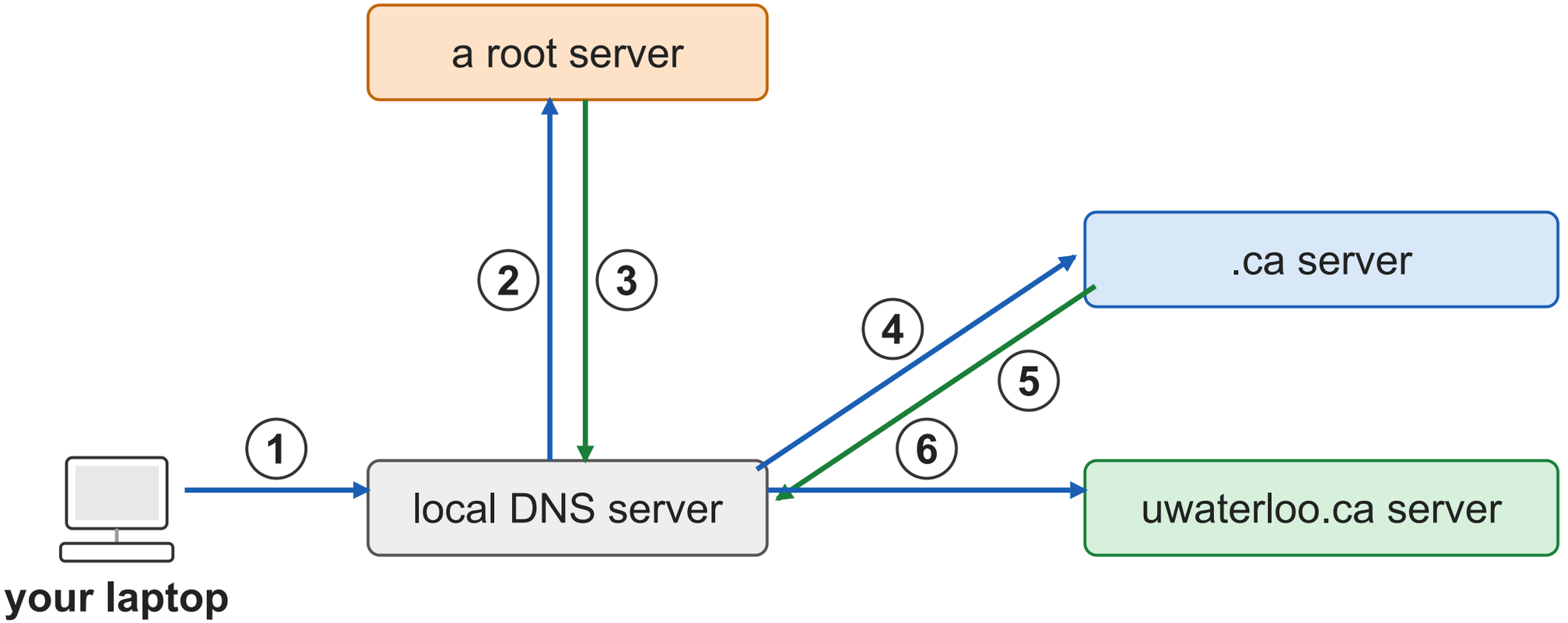
Iterative queries: each server answers with who to ask next.



.ca: “Ask a uwaterloo.ca server, at 129.97.128.229.”

Resolving a name: iterative queries

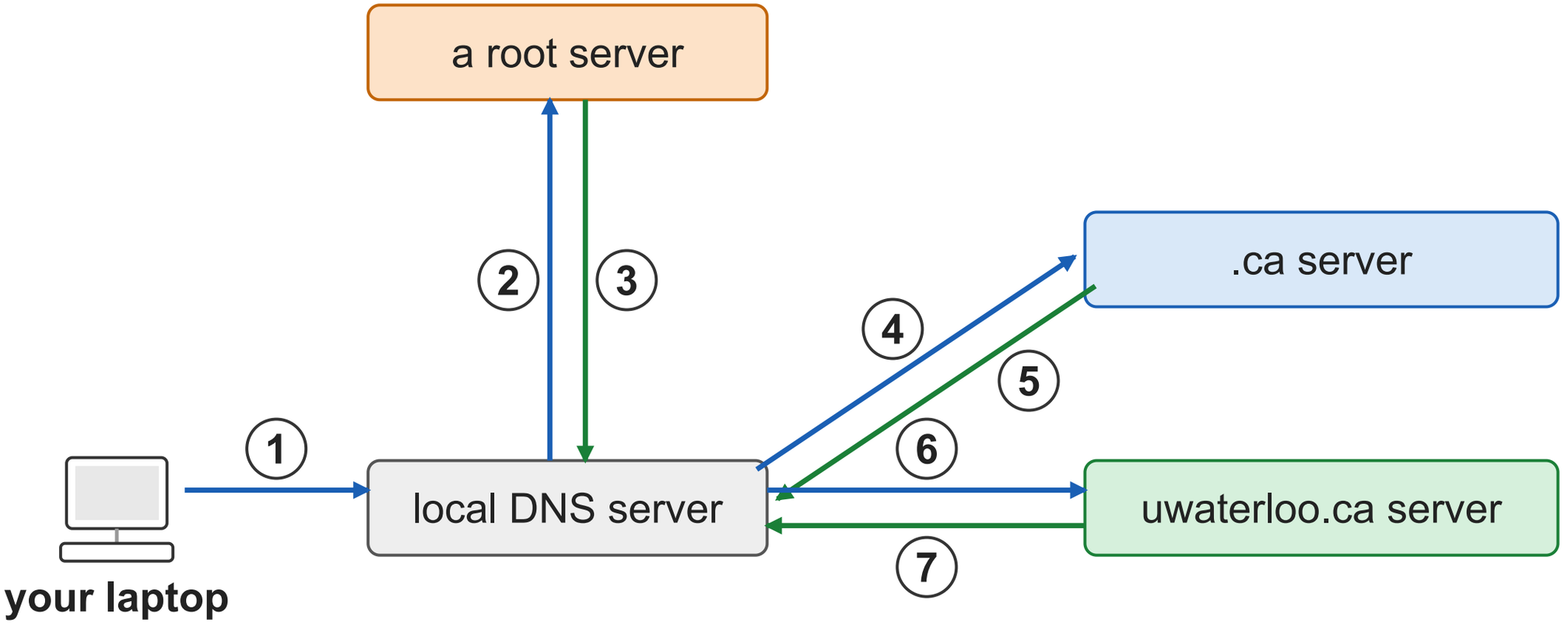
Iterative queries: each server answers with who to ask next.



The local DNS server asks the uwaterloo.ca server: cs.uwaterloo.ca?

Resolving a name: iterative queries

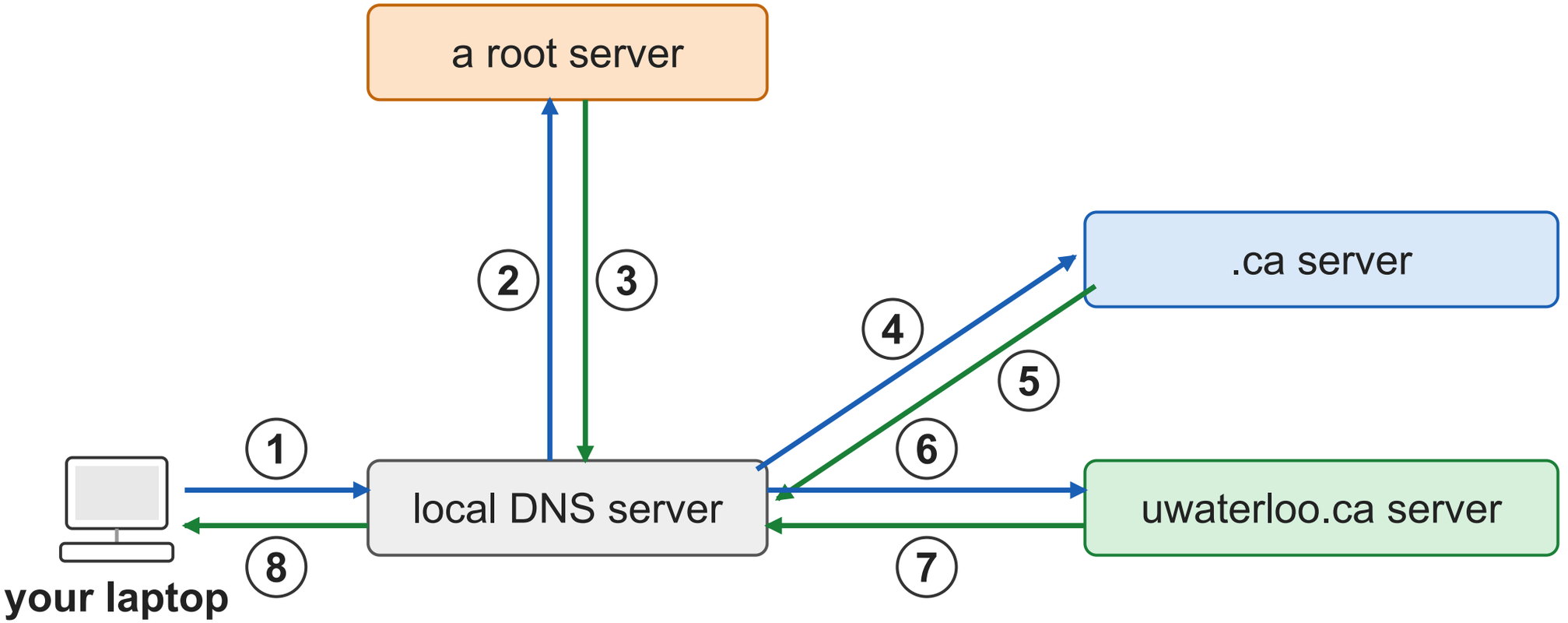
Iterative queries: each server answers with who to ask next.



uwaterloo.ca: “cs.uwaterloo.ca is 129.97.167.168.”

Resolving a name: iterative queries

Iterative queries: each server answers with who to ask next.



The local DNS server passes 129.97.167.168 on to your laptop.

Resolving a name: recursive queries

Recursive queries: each server asks the next one for you.

a root server

.ca server



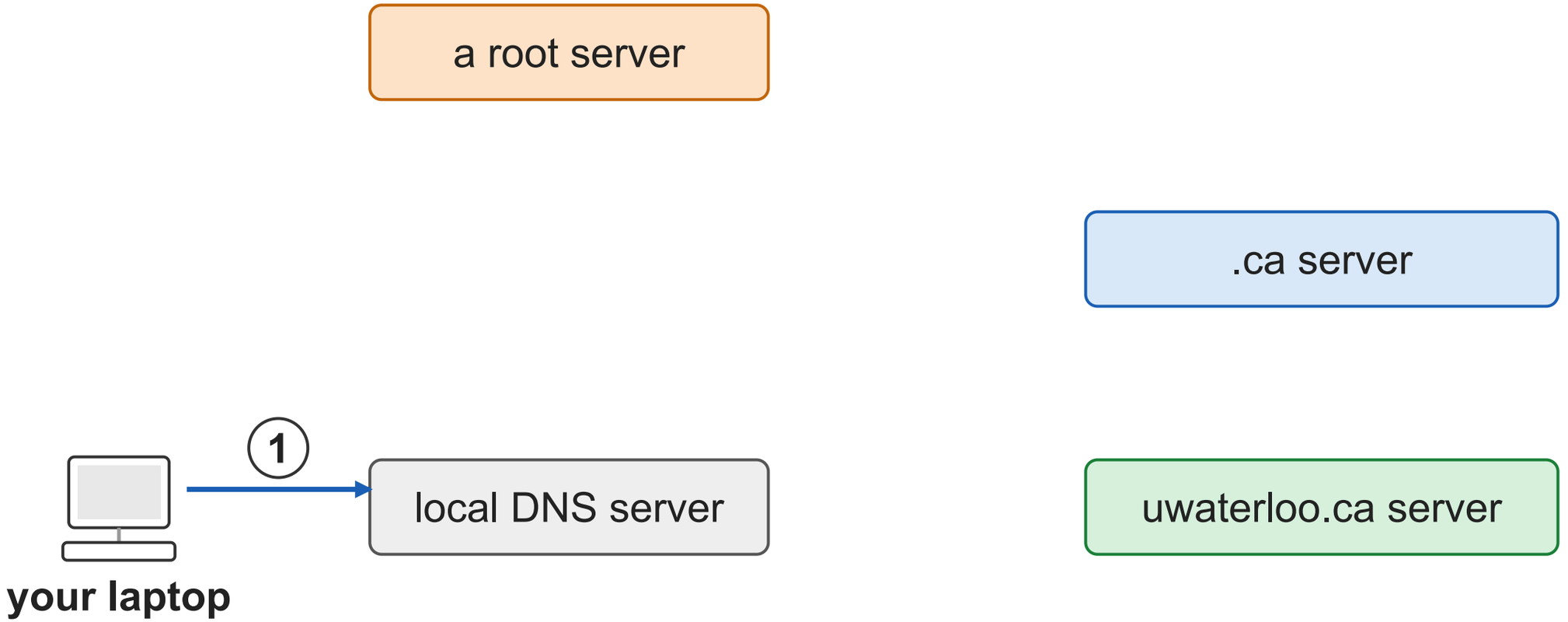
your laptop

local DNS server

uwaterloo.ca server

Resolving a name: recursive queries

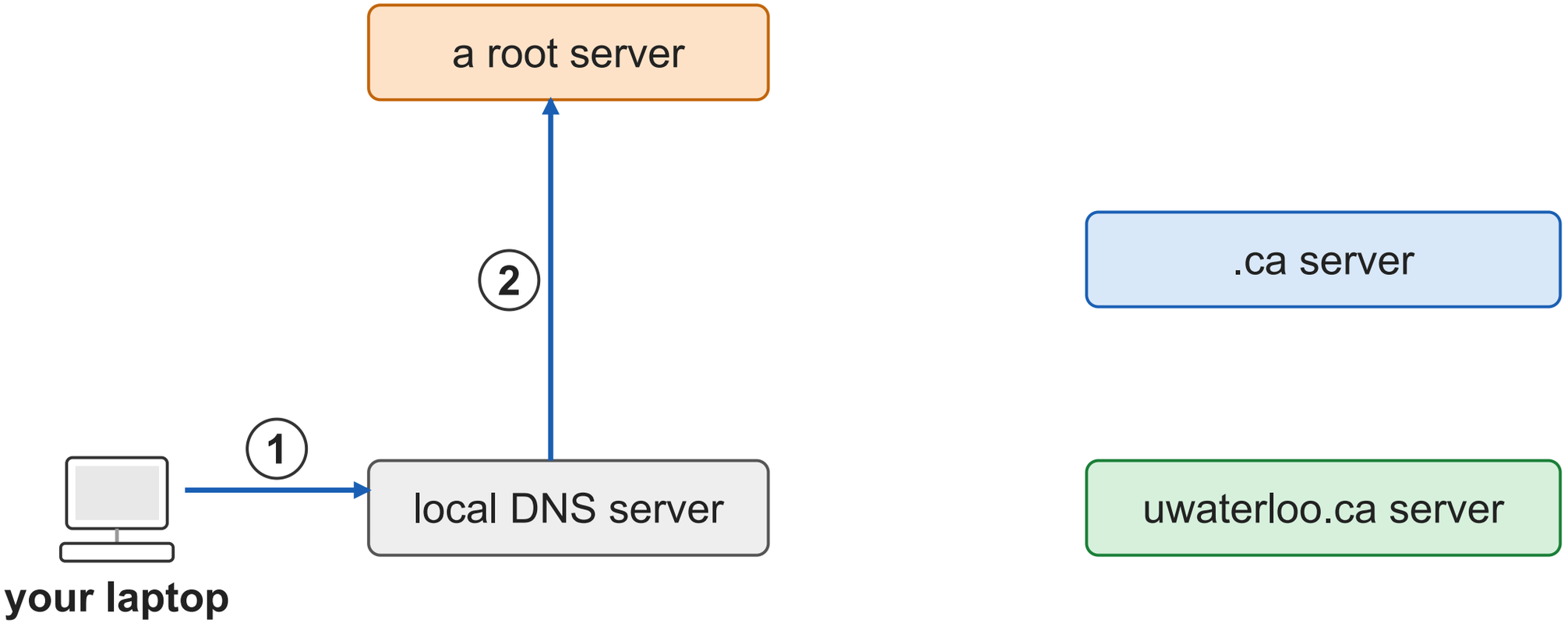
Recursive queries: each server asks the next one for you.



Your laptop asks its local DNS server: cs.uwaterloo.ca?

Resolving a name: recursive queries

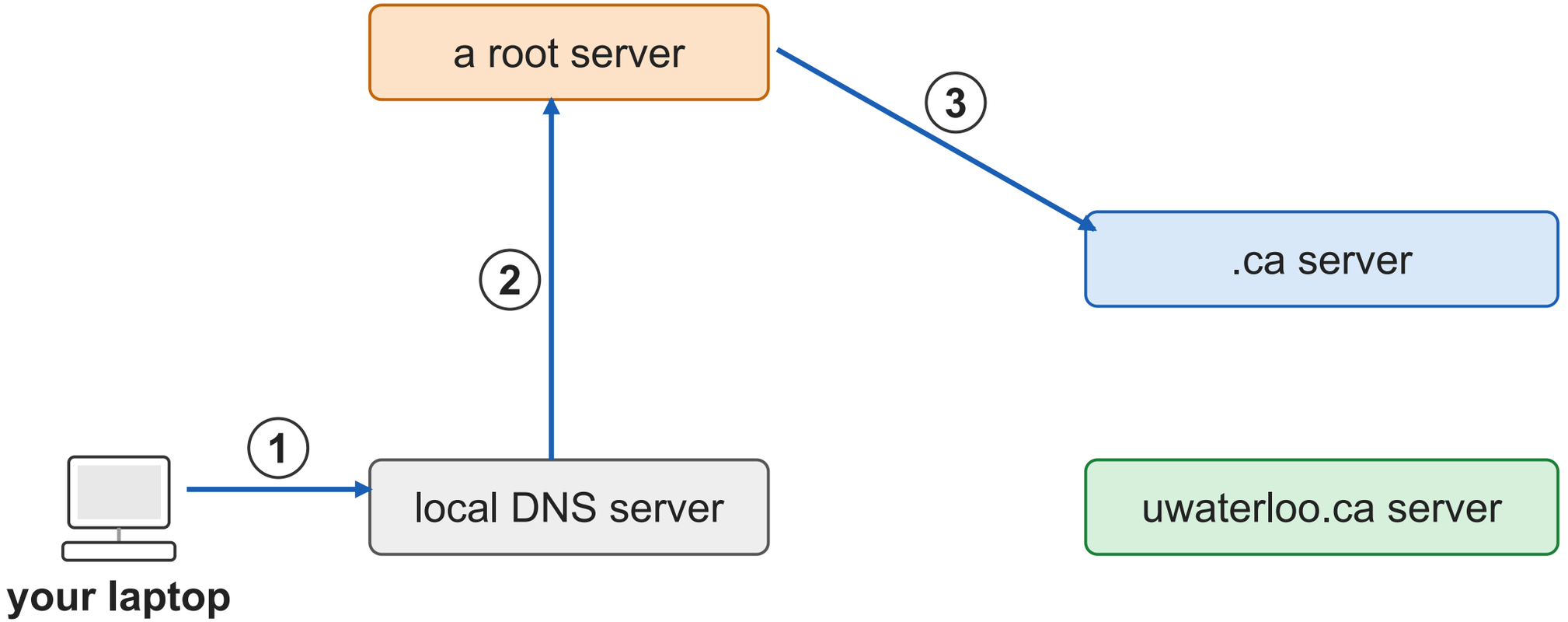
Recursive queries: each server asks the next one for you.



The local DNS server asks a root server, and leaves the rest to it.

Resolving a name: recursive queries

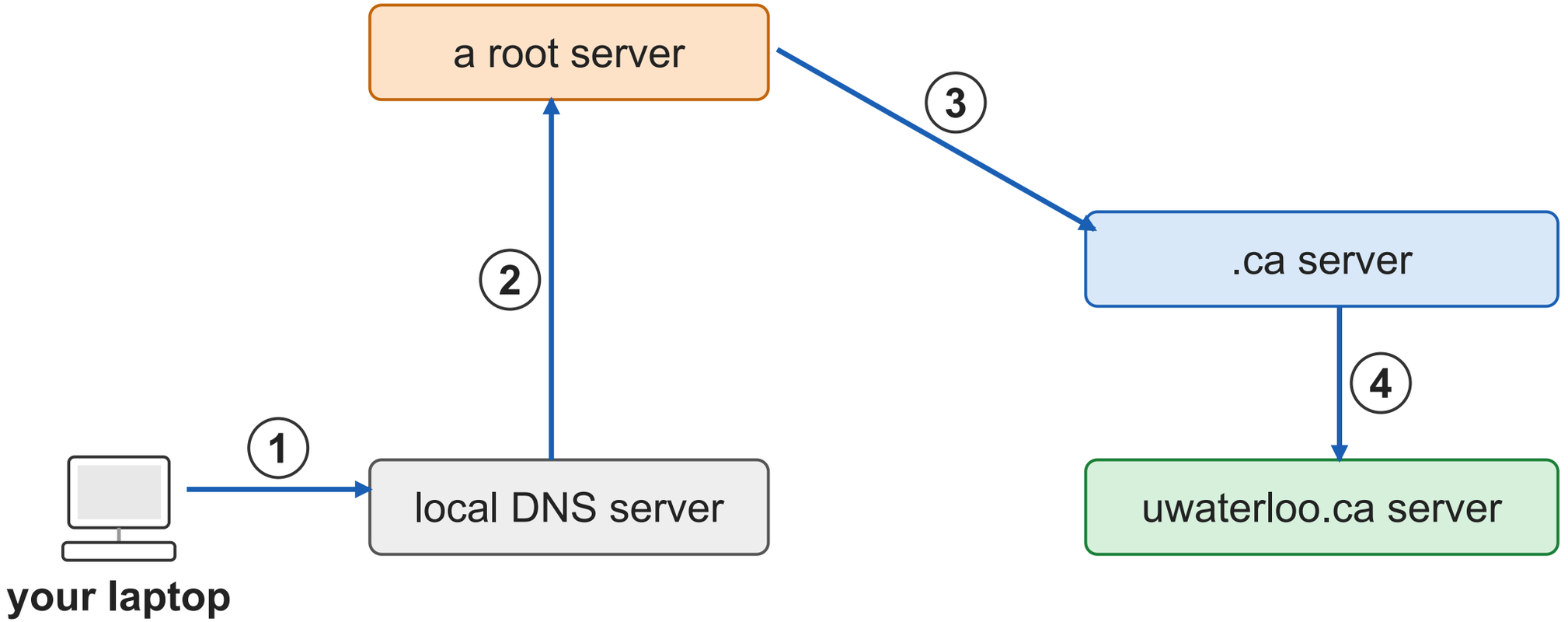
Recursive queries: each server asks the next one for you.



Root asks the .ca server.

Resolving a name: recursive queries

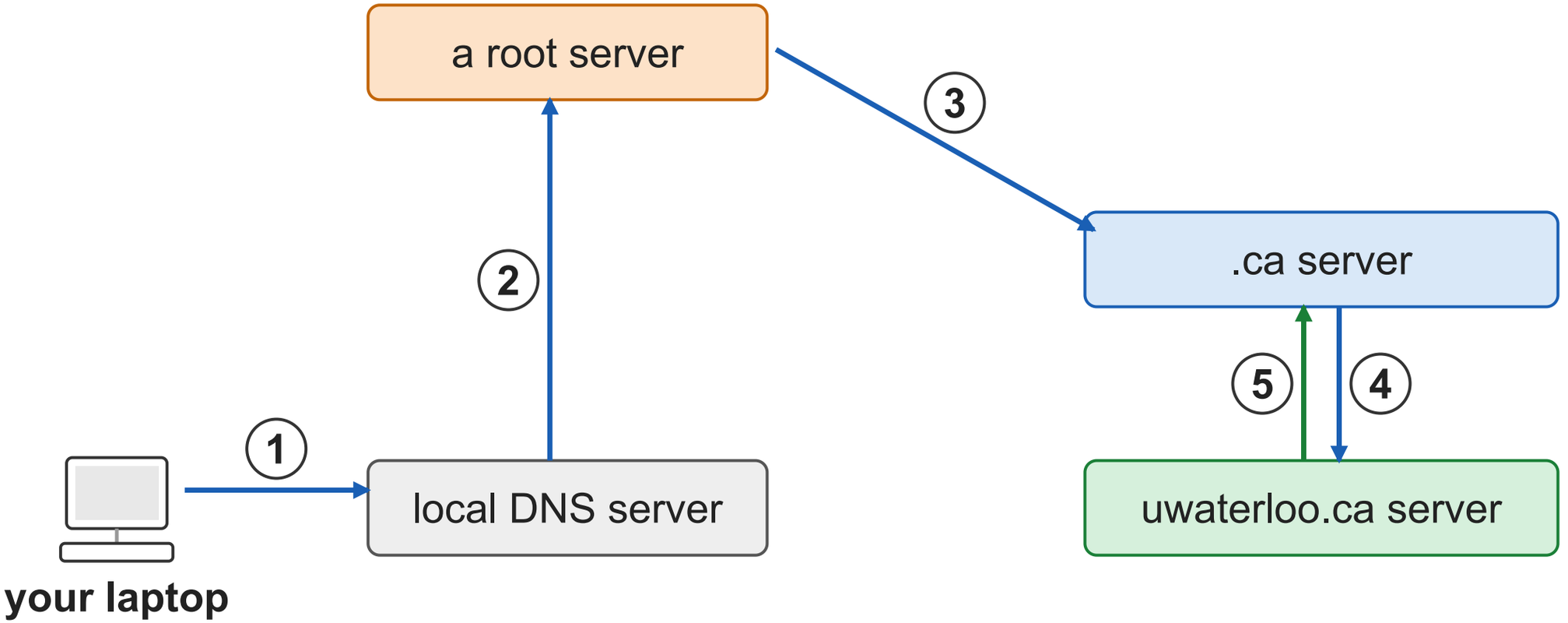
Recursive queries: each server asks the next one for you.



The .ca server asks the uwaterloo.ca server.

Resolving a name: recursive queries

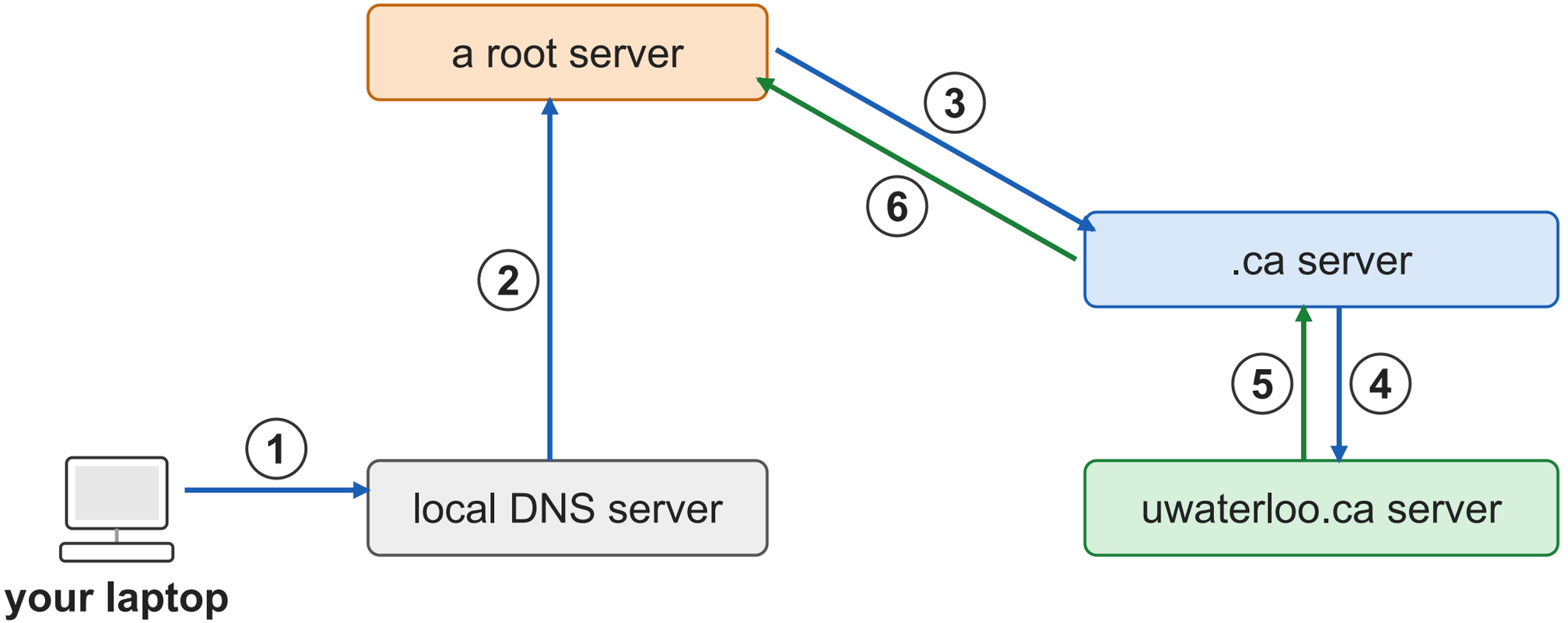
Recursive queries: each server asks the next one for you.



uwaterloo.ca: “cs.uwaterloo.ca is 129.97.167.168”, back to the .ca server.

Resolving a name: recursive queries

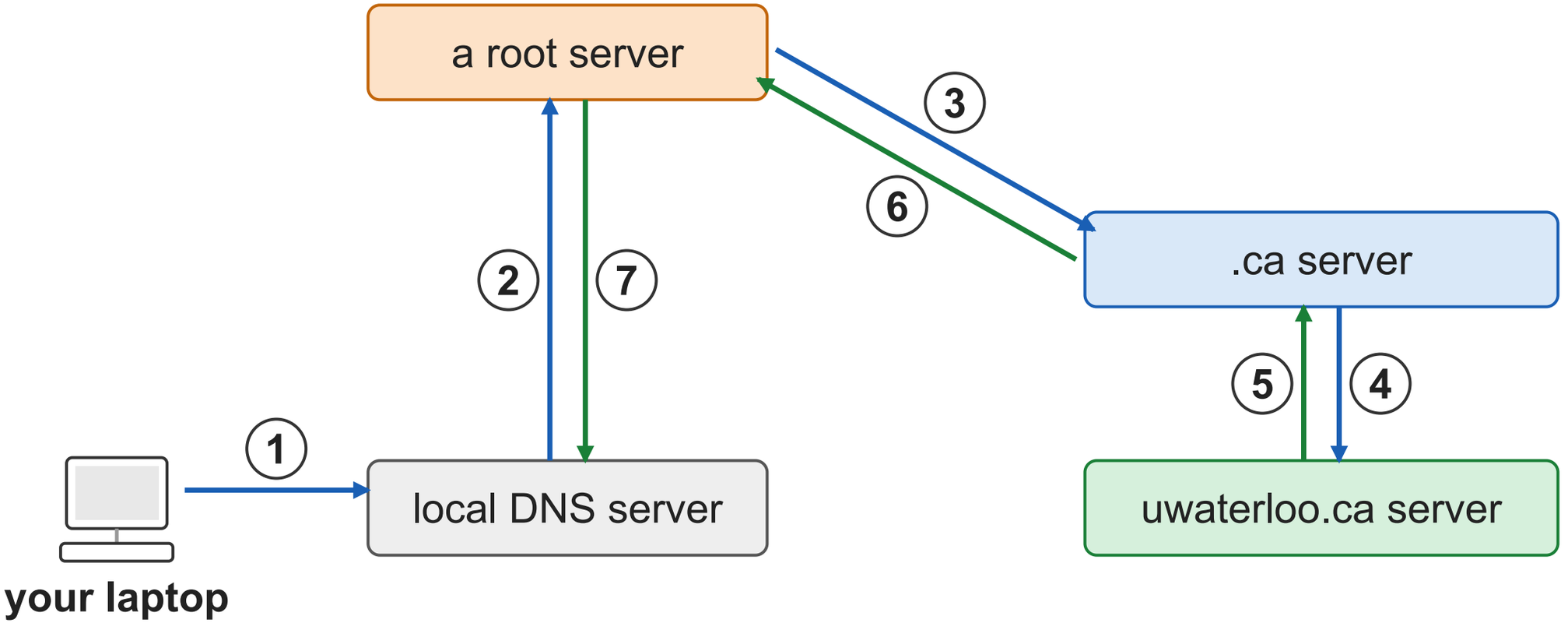
Recursive queries: each server asks the next one for you.



The .ca server passes the answer back to root.

Resolving a name: recursive queries

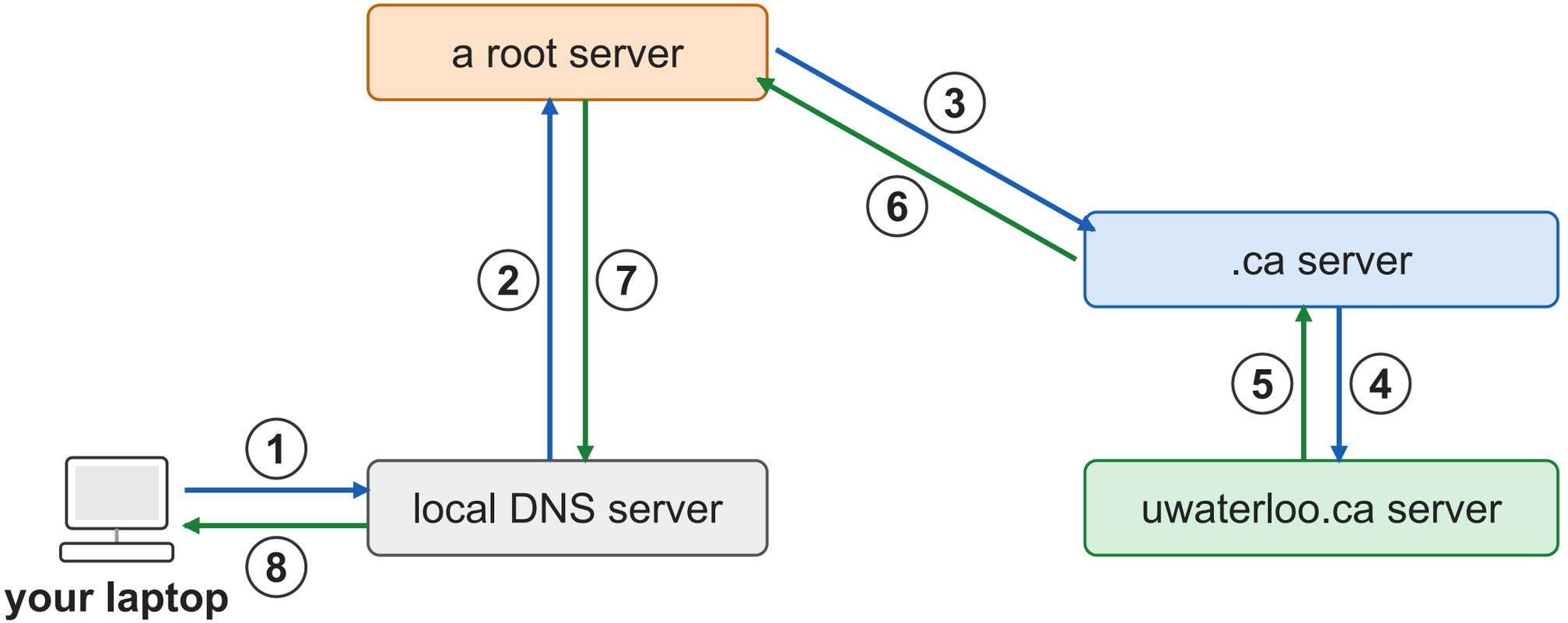
Recursive queries: each server asks the next one for you.



Root passes it back to the local DNS server.

Resolving a name: recursive queries

Recursive queries: each server asks the next one for you.

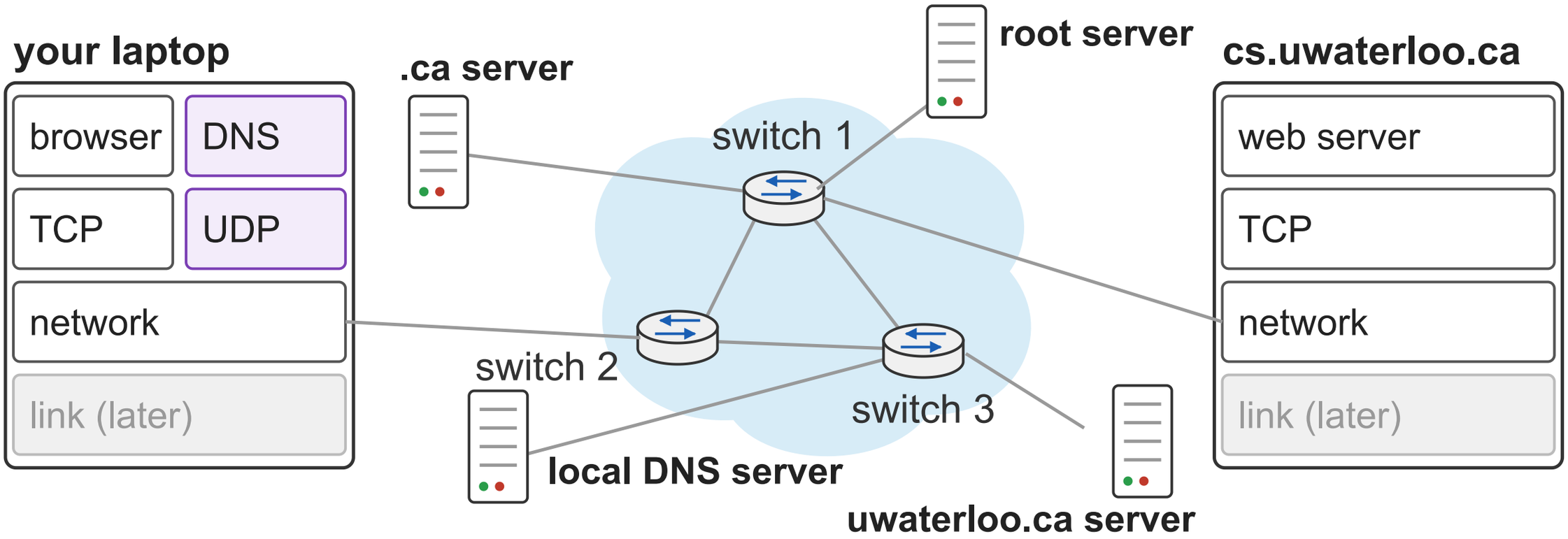


The local DNS server passes 129.97.167.168 on to your laptop.

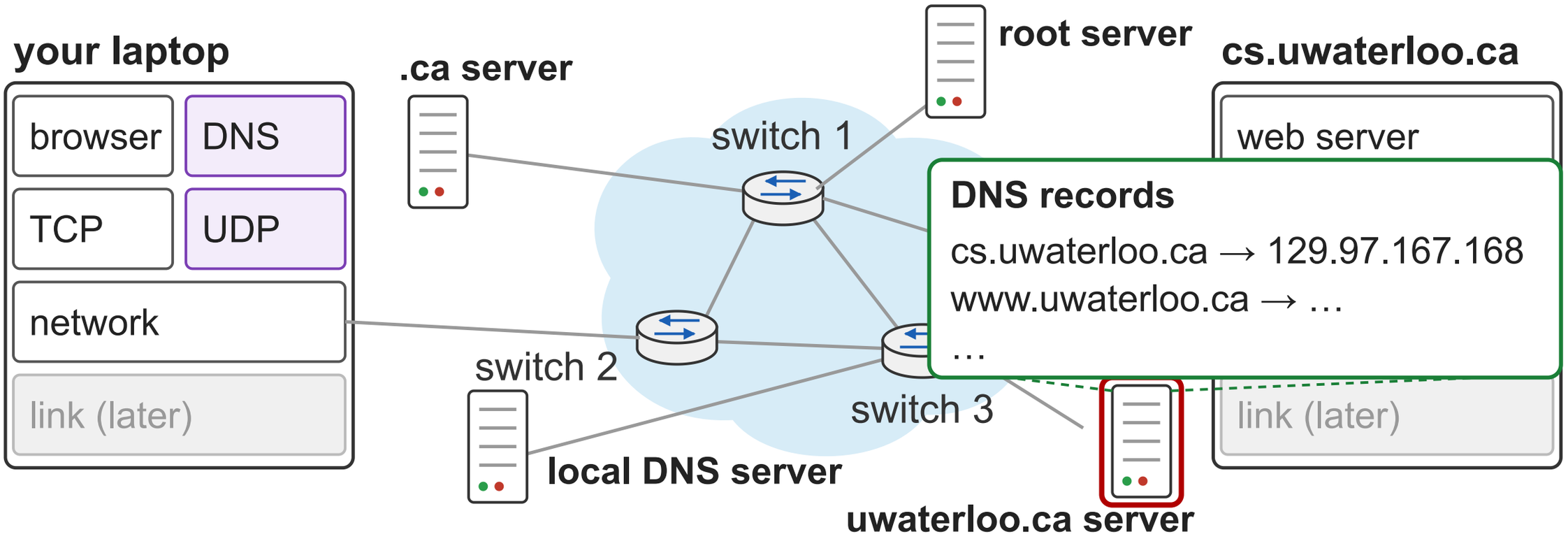
Iterative or recursive? (in-class) (in-class discussion)

Reset

Inside a DNS server

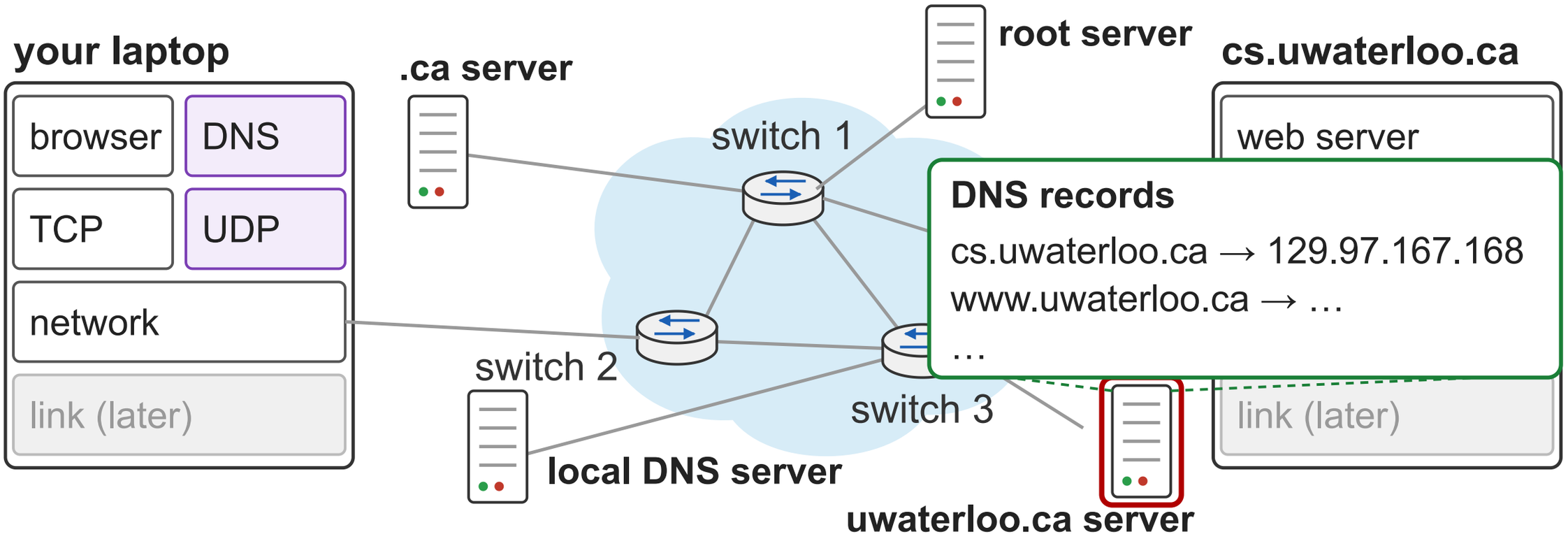


Inside a DNS server



It keeps records for the names it is responsible for.

Inside a DNS server



What does a DNS record look like?

DNS records

- Each record has the form (name, value, type, TTL).
- There are many types of DNS records. The three main ones we will discuss in this course are the following:

type	name	value
A	a hostname	its IPv4 address (AAAA: its IPv6 address)
NS	a domain	the name of a name server for that domain
CNAME	an alias	the real (canonical) name

- TTL (time to live): whoever keeps a copy of the response should only keep it for TTL seconds.

Example: A and NS records

.ca server

(uwaterloo.ca, cn-dns-mc-new.uwaterloo.ca, NS, 86400)

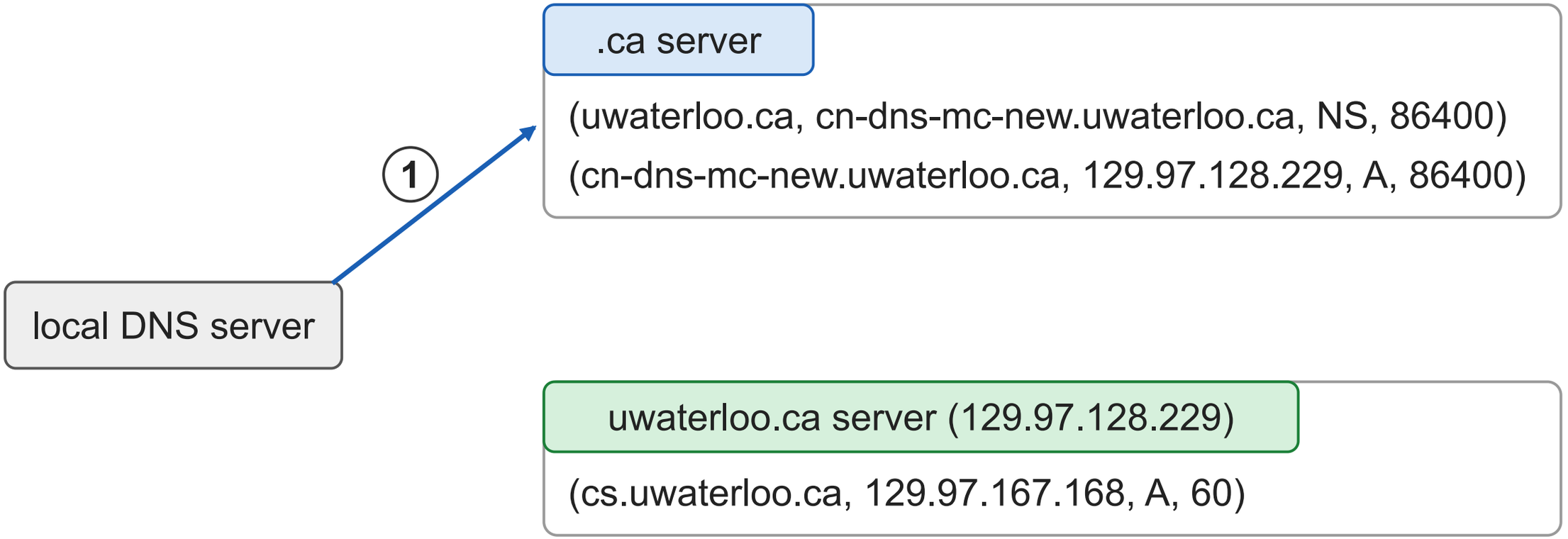
(cn-dns-mc-new.uwaterloo.ca, 129.97.128.229, A, 86400)

local DNS server

uwaterloo.ca server (129.97.128.229)

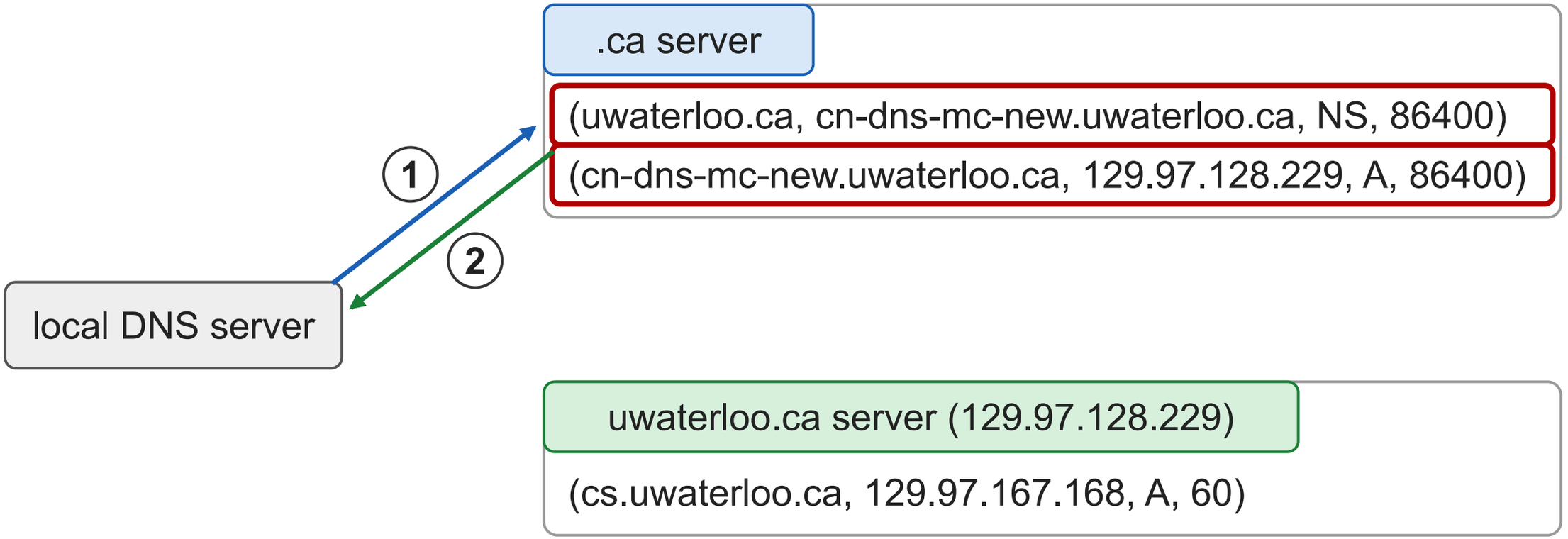
(cs.uwaterloo.ca, 129.97.167.168, A, 60)

Example: A and NS records



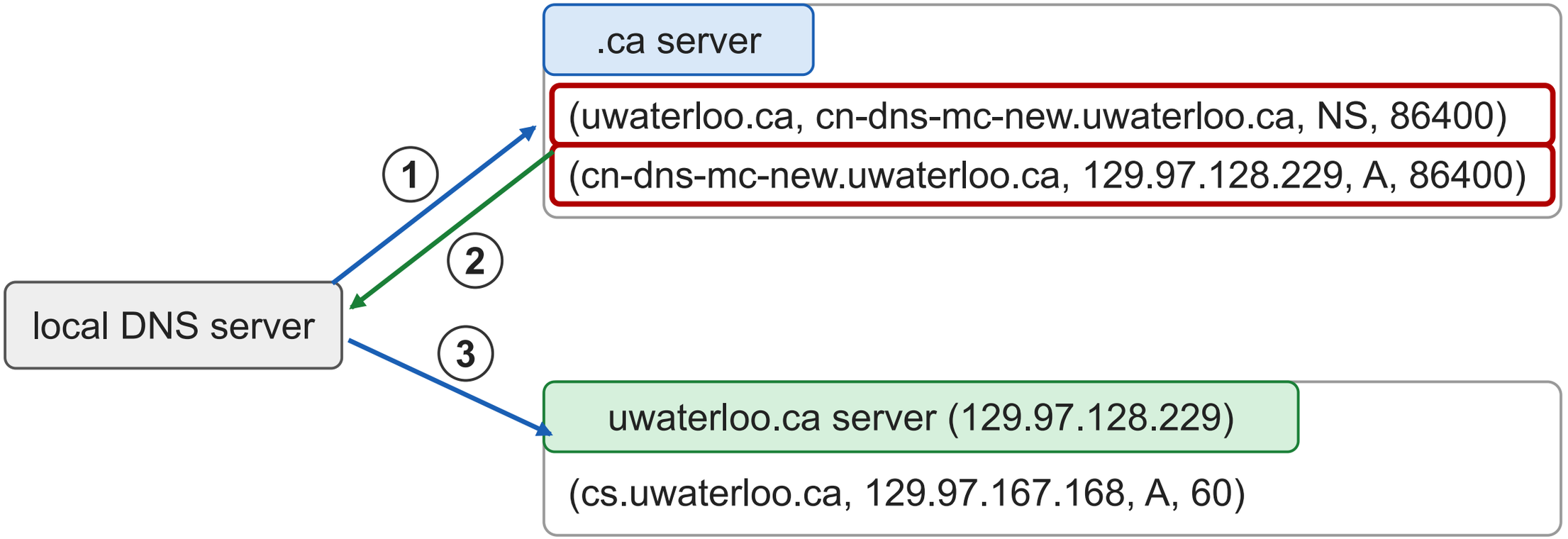
The local DNS server asks a .ca server: cs.uwaterloo.ca?

Example: A and NS records



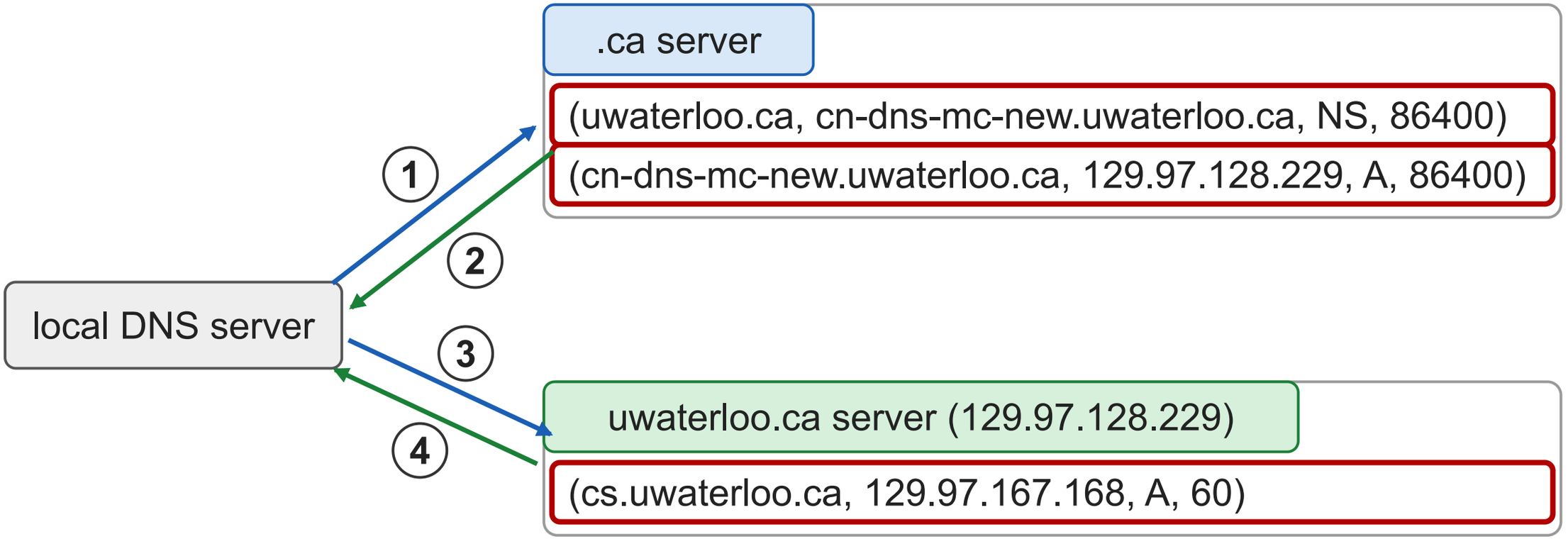
The .ca server does not know it. It replies with an NS record (the name of the name server that has the address) and an A record (the IP address of that DNS server).

Example: A and NS records



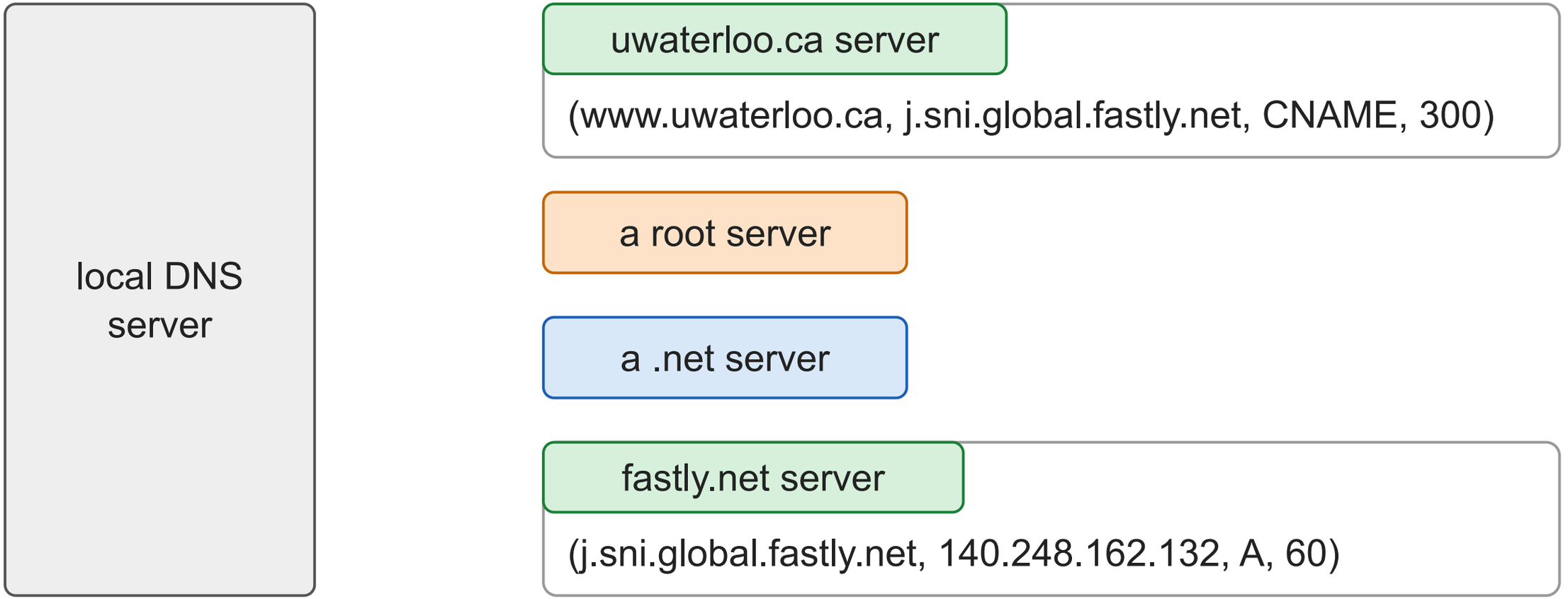
The local DNS server asks 129.97.128.229: cs.uwaterloo.ca?

Example: A and NS records

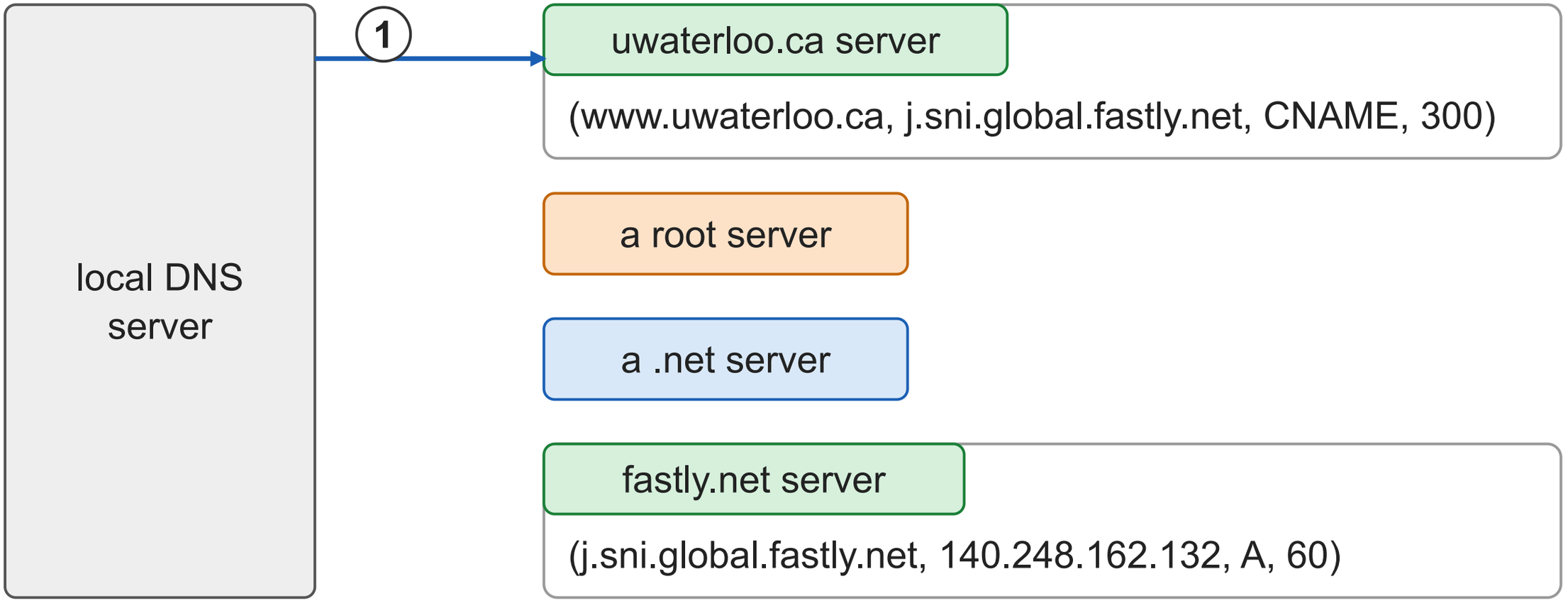


uwaterloo.ca's server replies with its A record: 129.97.167.168.

Example: CNAME records

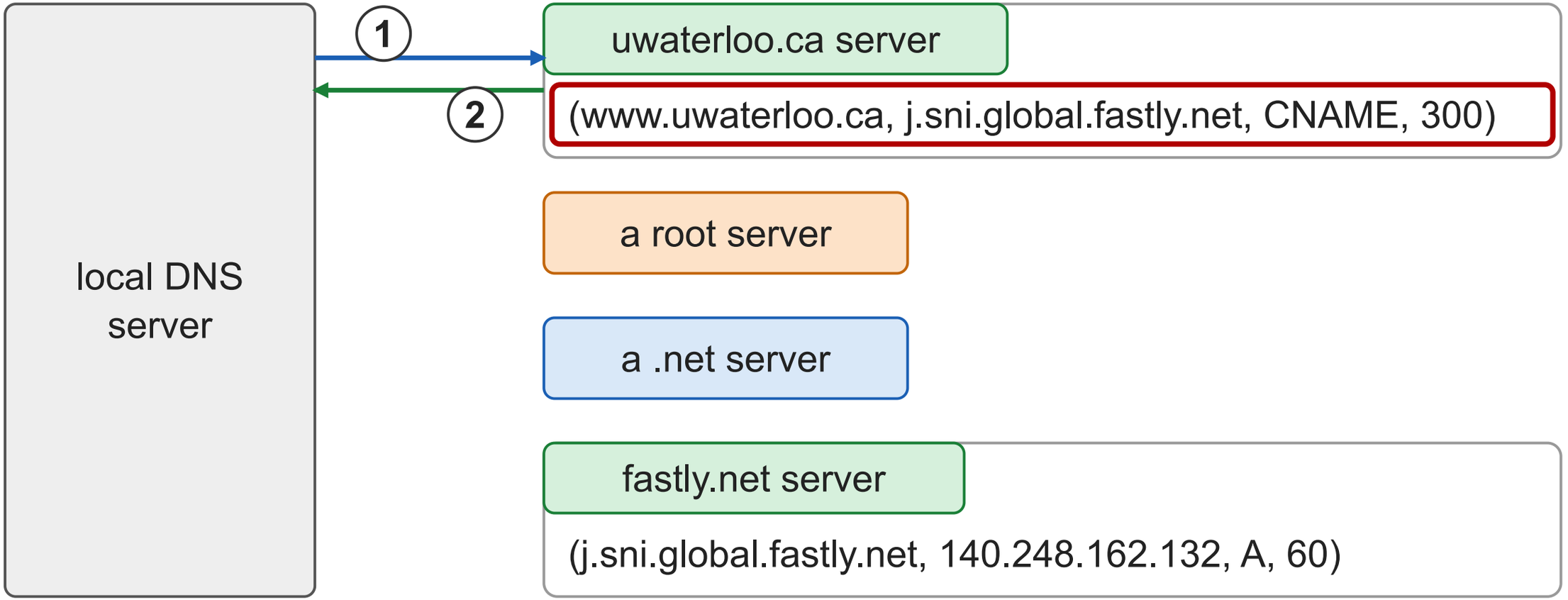


Example: CNAME records



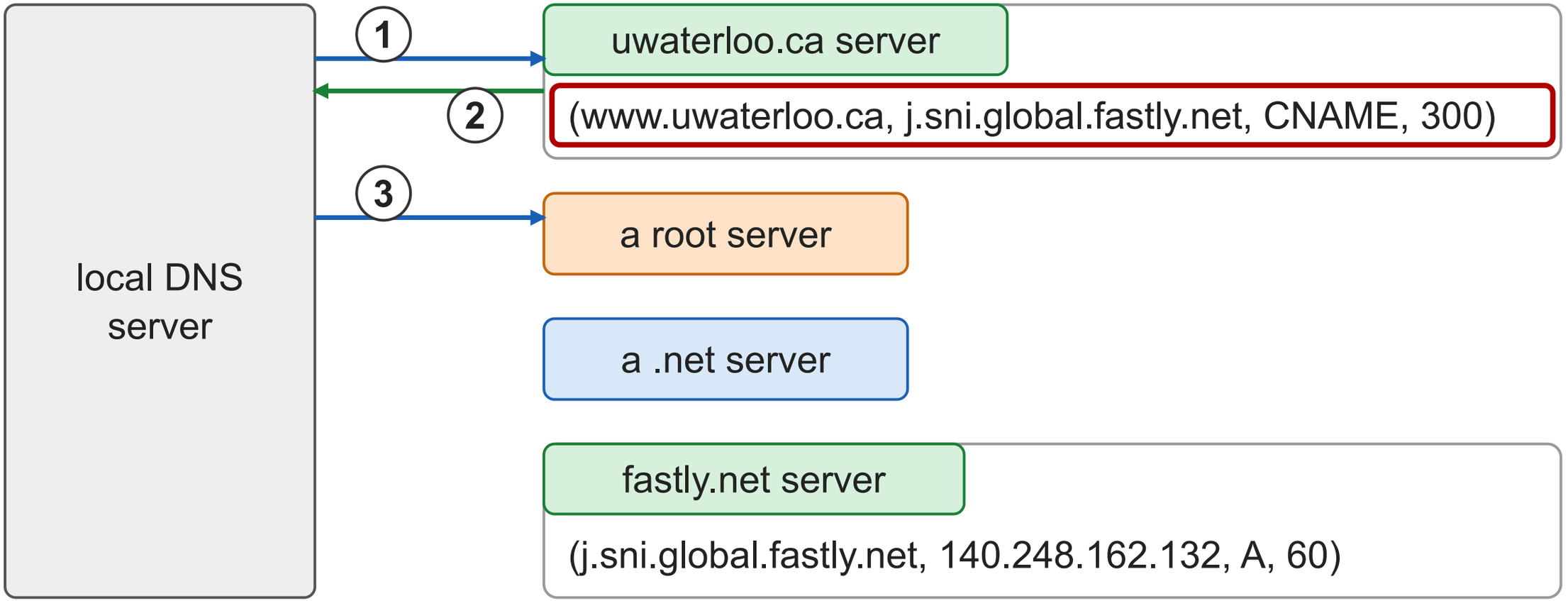
The local DNS server asks uwaterloo.ca's server: www.uwaterloo.ca?

Example: CNAME records



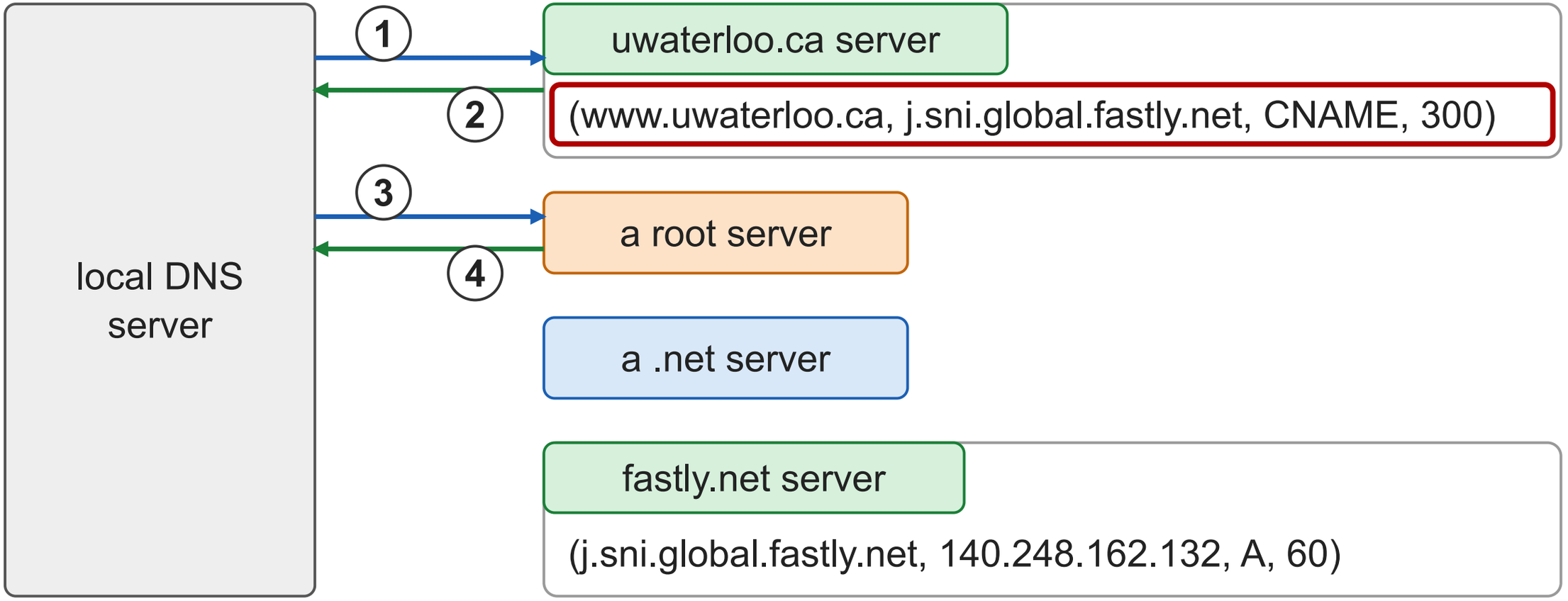
It replies with a CNAME record: www.uwaterloo.ca is an alias; its real name is j.sni.global.fastly.net.

Example: CNAME records



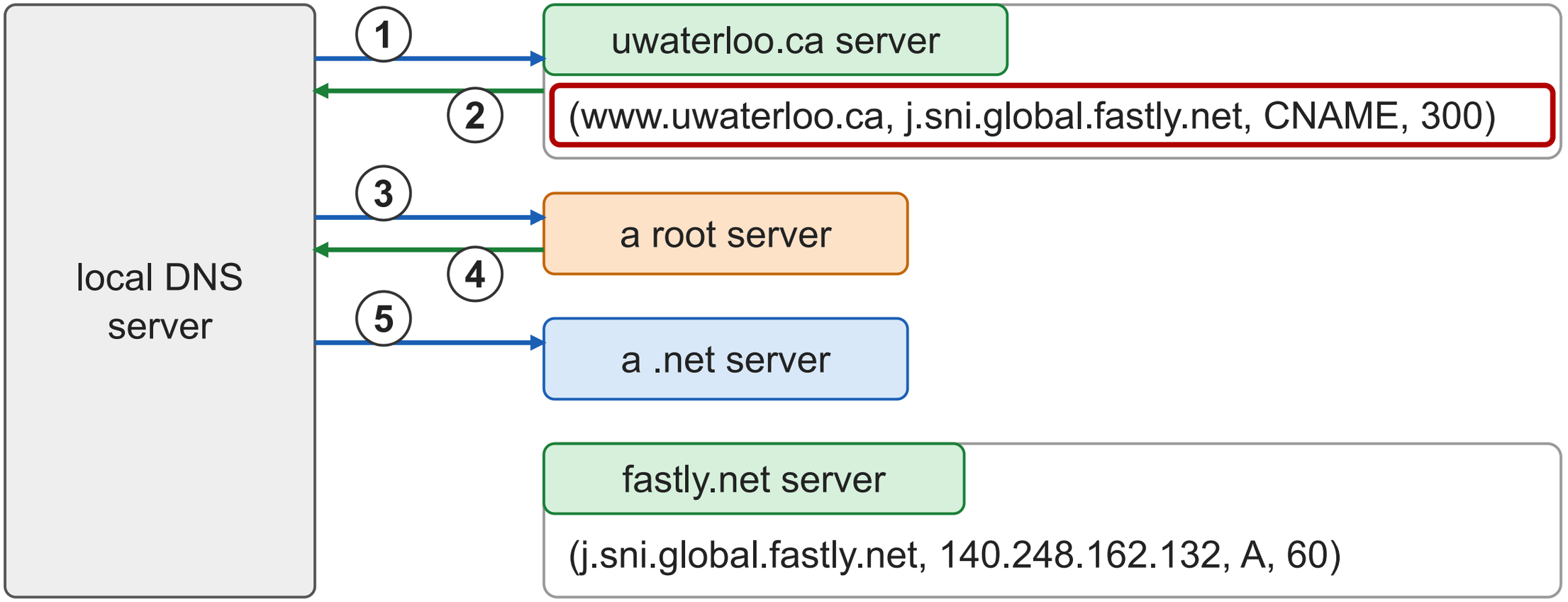
The local DNS server starts over with the real name. It asks a root server: j.sni.global.fastly.net?

Example: CNAME records



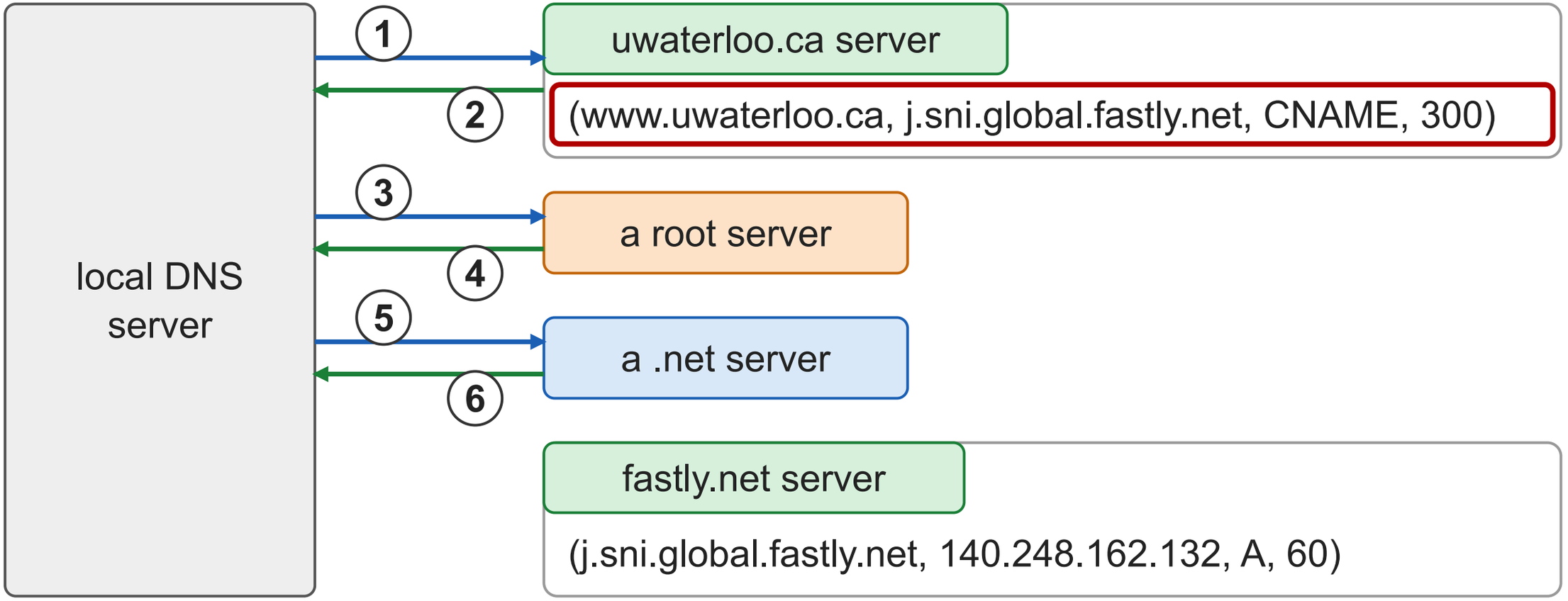
Root: “I don’t know, but here is a .net server.”

Example: CNAME records



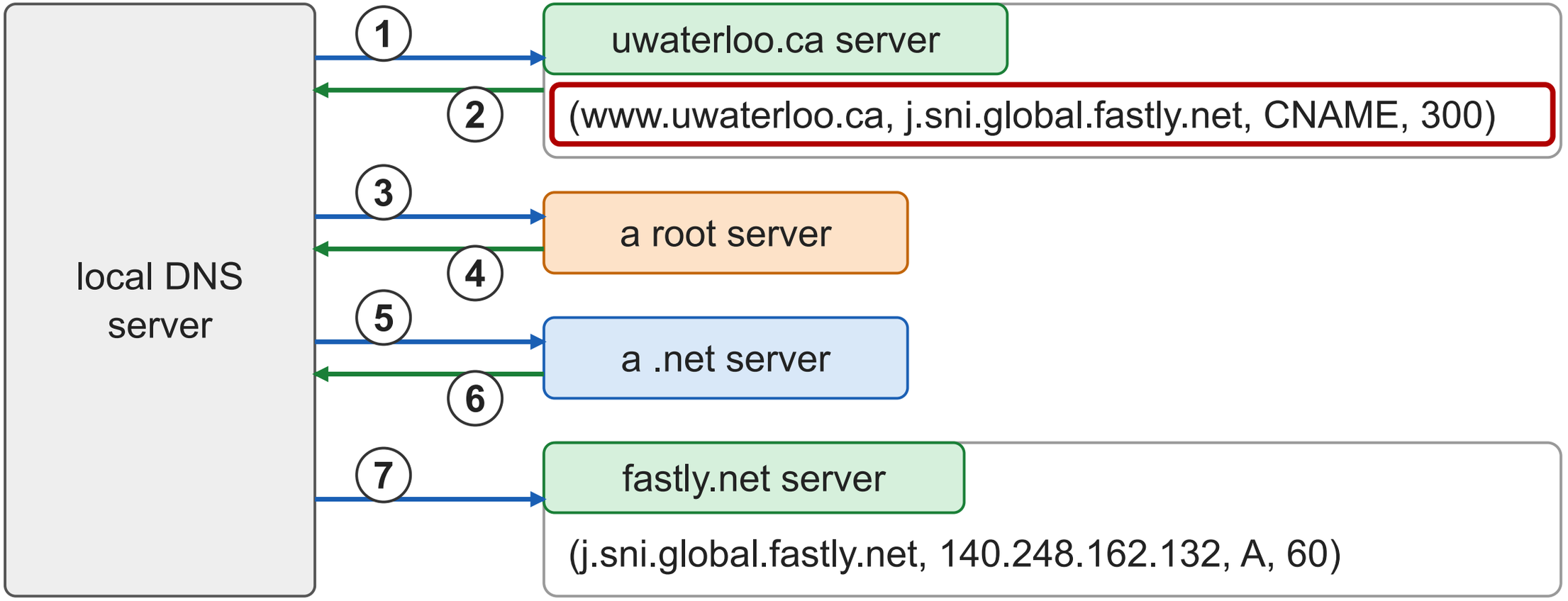
It asks the .net server: j.sni.global.fastly.net?

Example: CNAME records



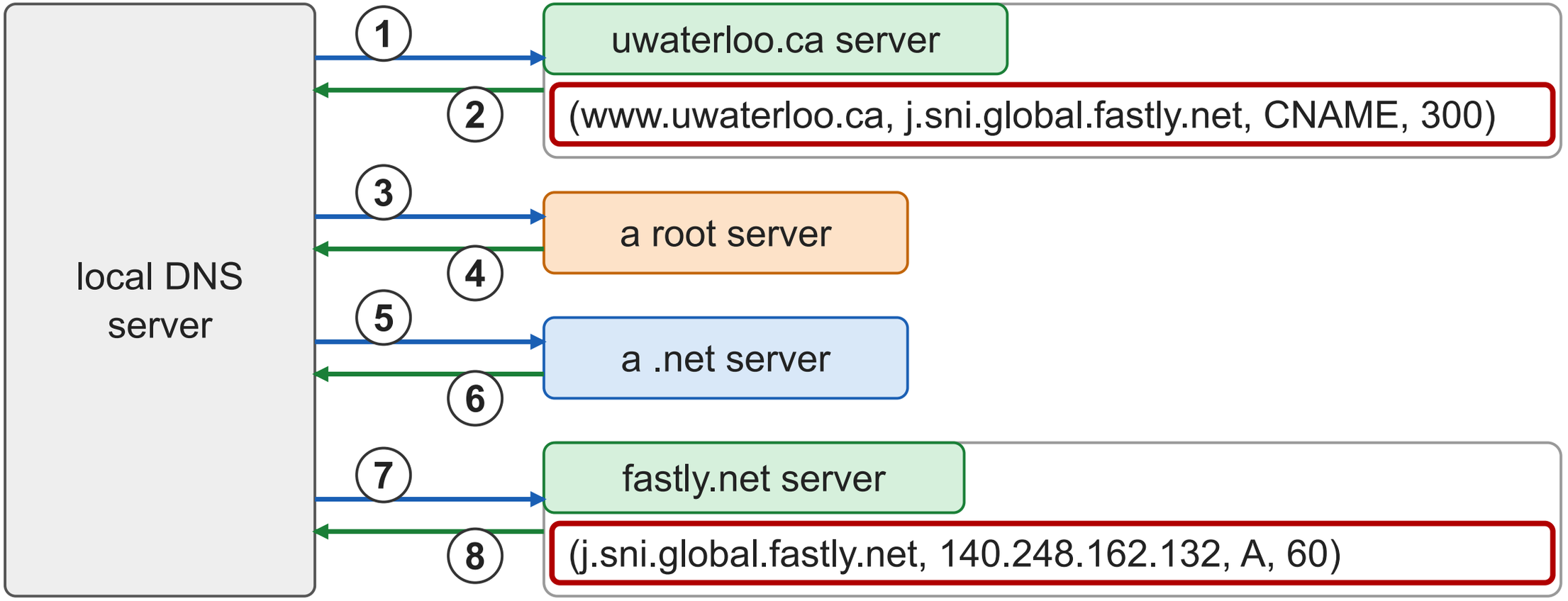
.net: “I don’t know either, but here is a fastly.net server.”

Example: CNAME records



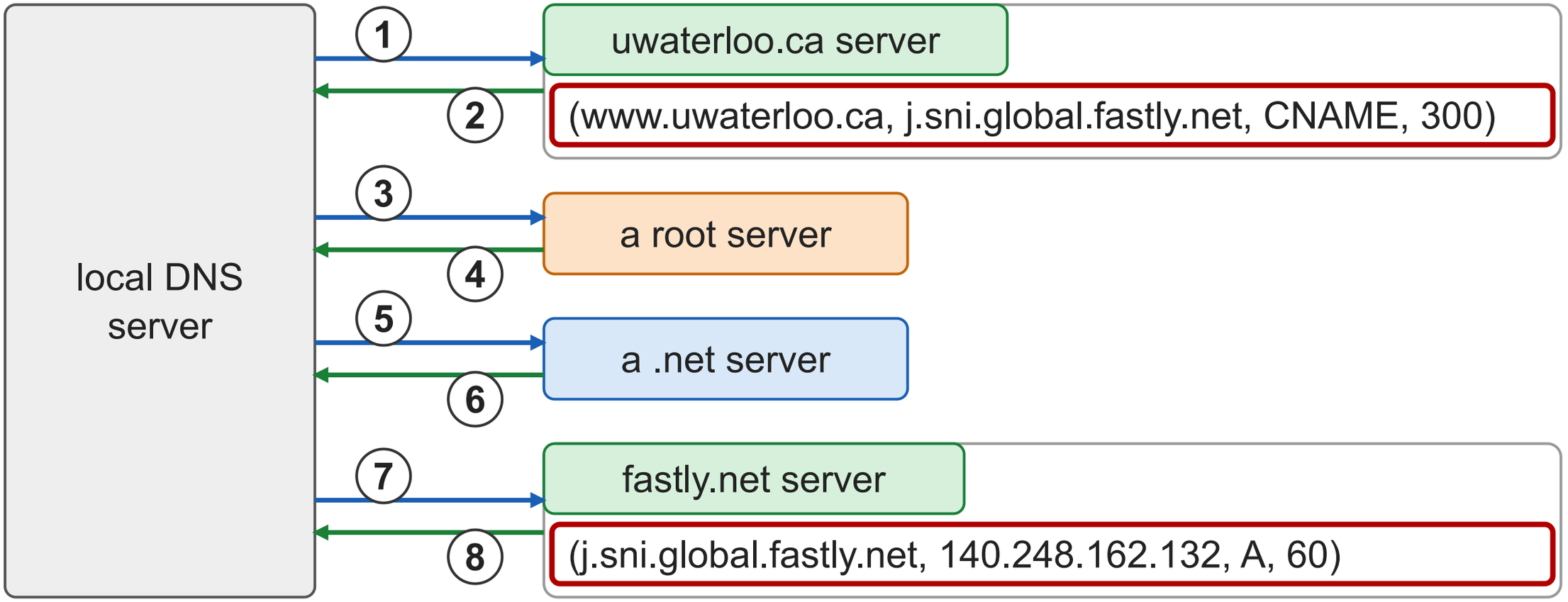
It asks the fastly.net server: j.sni.global.fastly.net?

Example: CNAME records



Fastly's server replies with an A record: 140.248.162.132. That is the address your laptop gets for www.uwaterloo.ca.

Example: CNAME records



Why an alias? Fastly, not the university, runs the web site, on addresses Fastly changes as it likes. With a CNAME, the university does not need to know them. If it switches companies later, it just changes one record.

Example: TTL



laptop 1



laptop 2

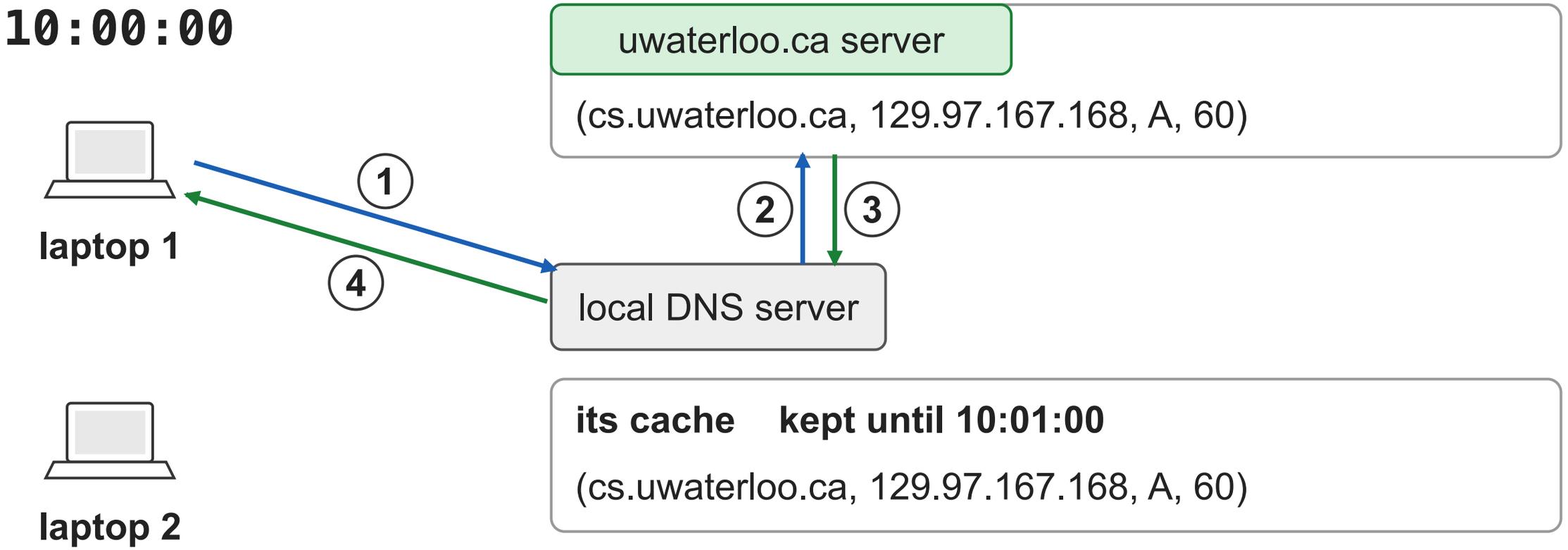
uwaterloo.ca server

(cs.uwaterloo.ca, 129.97.167.168, A, 60)

local DNS server

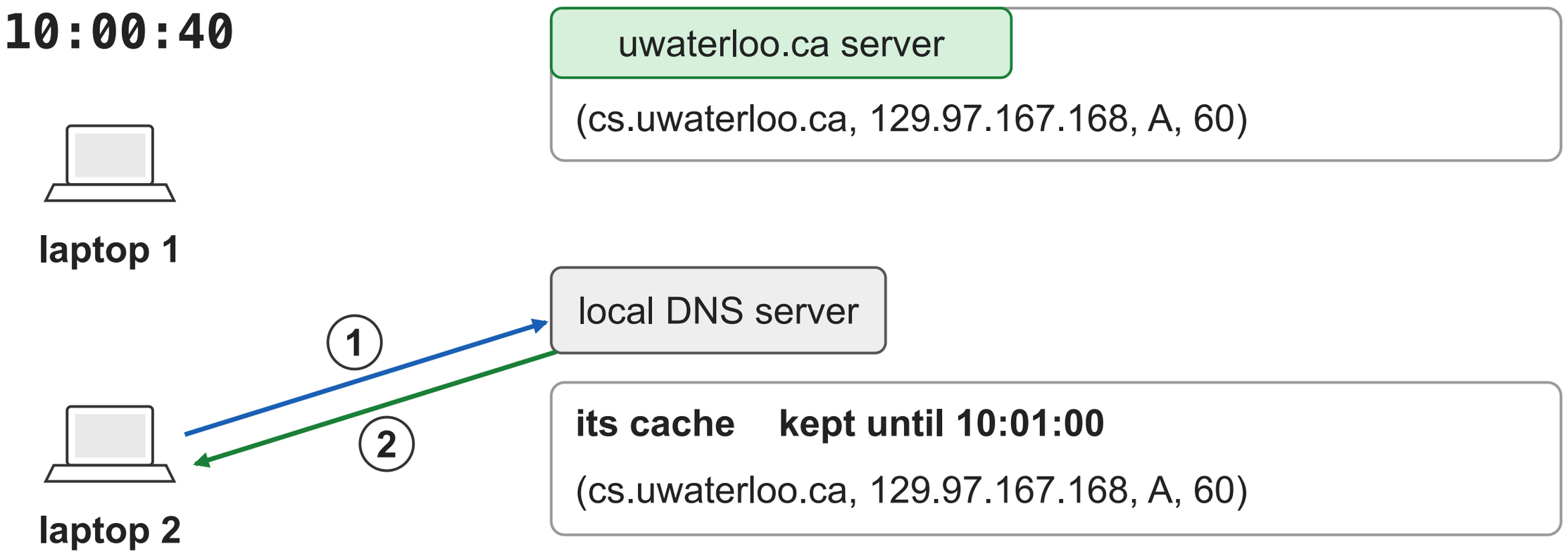
its cache

Example: TTL



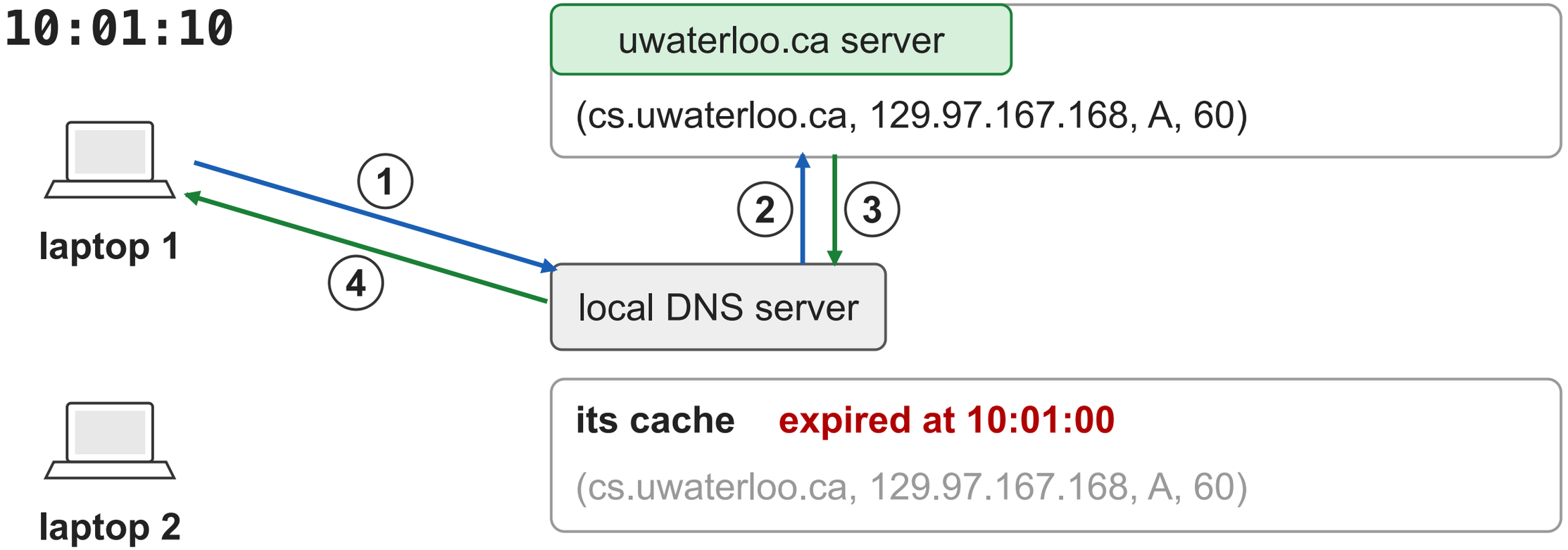
Laptop 1 asks. The cache is empty: the local DNS server asks uwaterloo.ca's server, and keeps the answer for 60 s.

Example: TTL



Laptop 2 asks. The copy is still good: the answer comes from the cache, right away.

Example: TTL

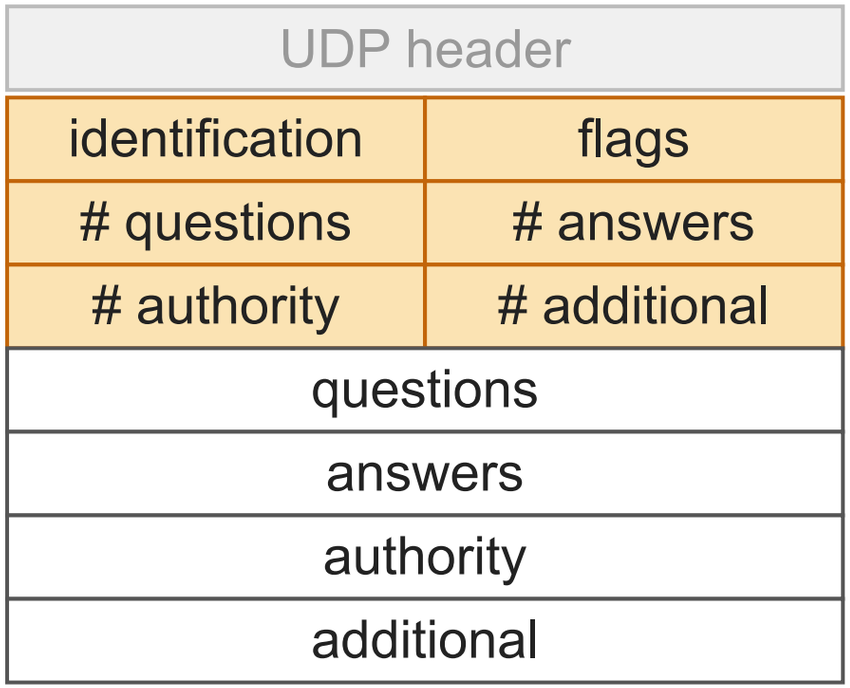


Laptop 1 asks again. The copy expired at 10:01:00, so the local DNS server asks uwaterloo.ca's server again.

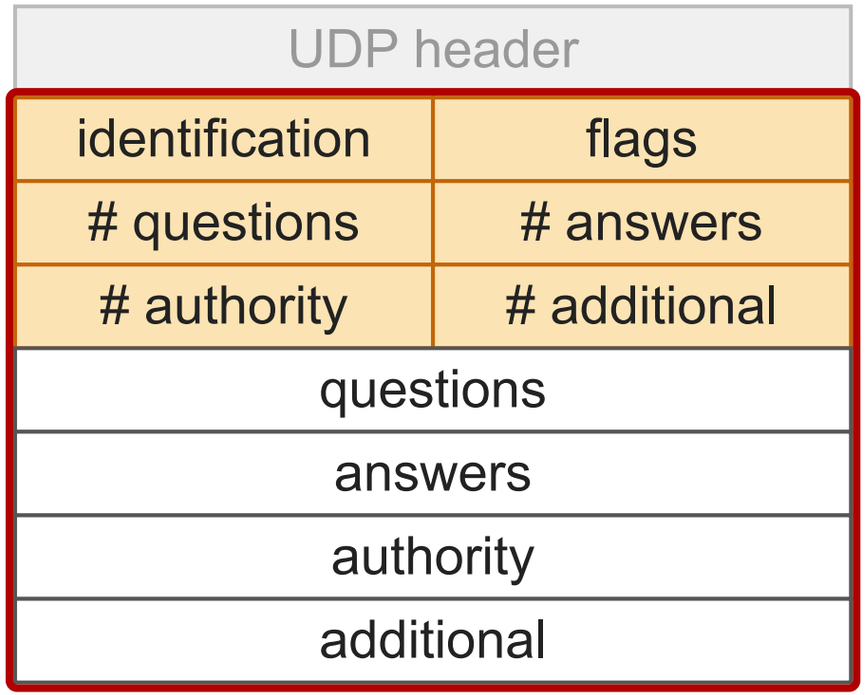
Setting up mystartup.ca (in-class) (in-class discussion)

- You register mystartup.ca with the .ca registry, the Canadian Internet Registration Authority (CIRA). Name servers: ns1.mystartup.ca (203.0.113.53, yours) and ns.dnshost.net (a DNS provider's).
- Your web site runs on two web servers, 203.0.113.10 and 203.0.113.11. Your engineering team wants its own name server, with IP address 203.0.113.99, for the domain dev.mystartup.ca.
- Which records should be added at which servers so that:
  - (1) mystartup.ca and www.mystartup.ca reach the site, on either web server?
  - (2) lookups still work when ns1.mystartup.ca is down?
  - (3) the engineering team can add names under dev.mystartup.ca, so they are accessible through DNS, without asking you?

DNS messages



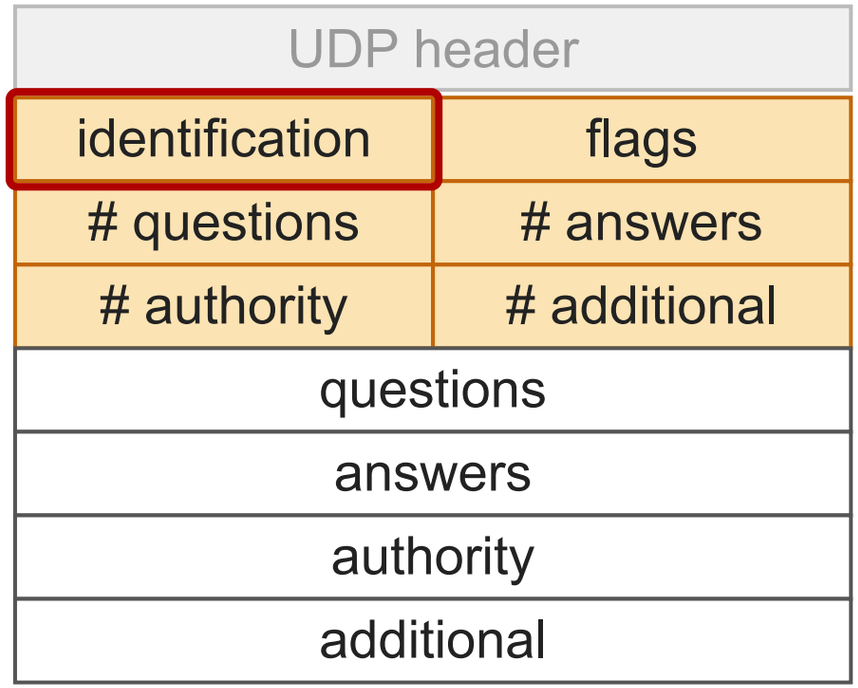
DNS messages



A DNS message

Sent in one UDP packet, to or from port 53. Queries and replies have the same format. The reply includes the question from the request as well as relevant records.

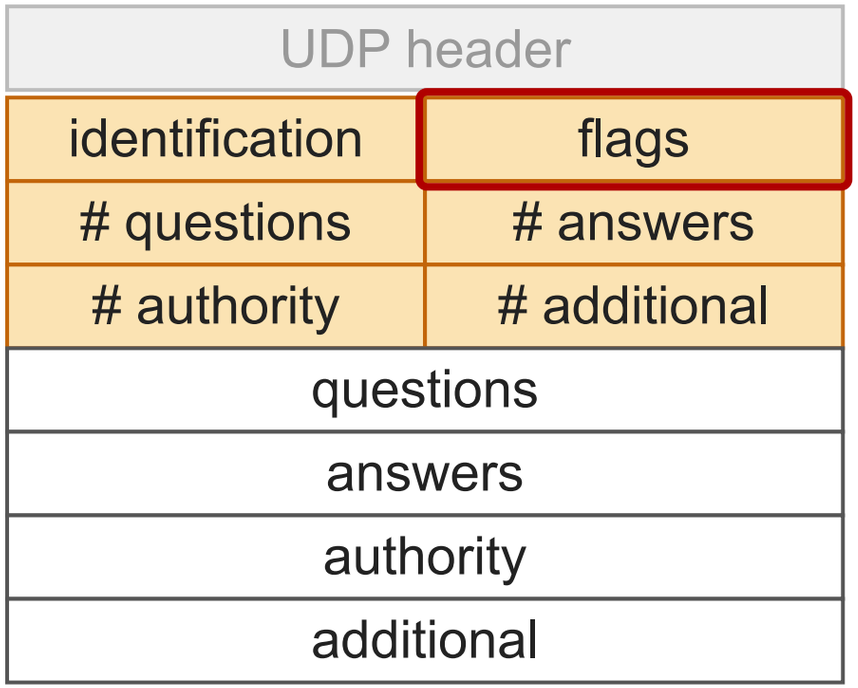
DNS messages



Identification

16 bits chosen by the asker, copied into the reply. A local DNS server has many queries out at once: this is how it matches a reply to its query.

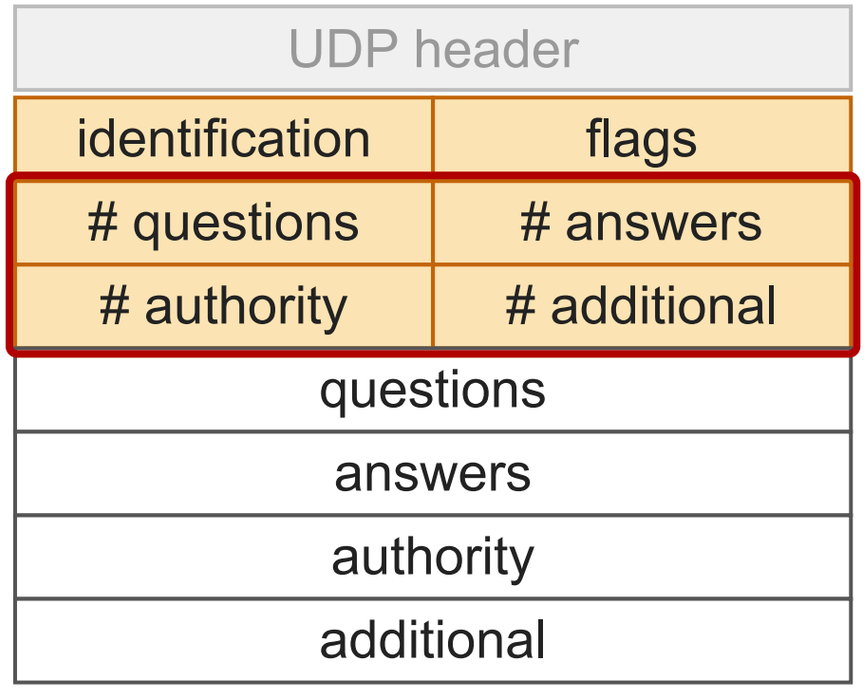
DNS messages



Flags

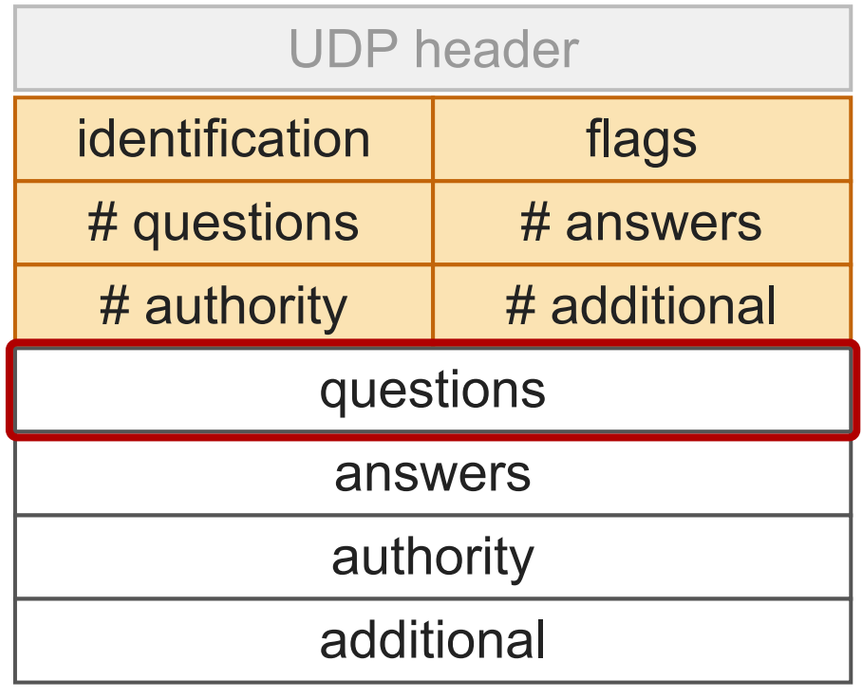
Query or reply; recursion desired (please do the whole walk for me); recursion available; authoritative (the answer comes from an authoritative DNS server that actually manages and holds the original records for that domain, rather than, say, a cache).

DNS messages



How many records are in each of the four sections below.

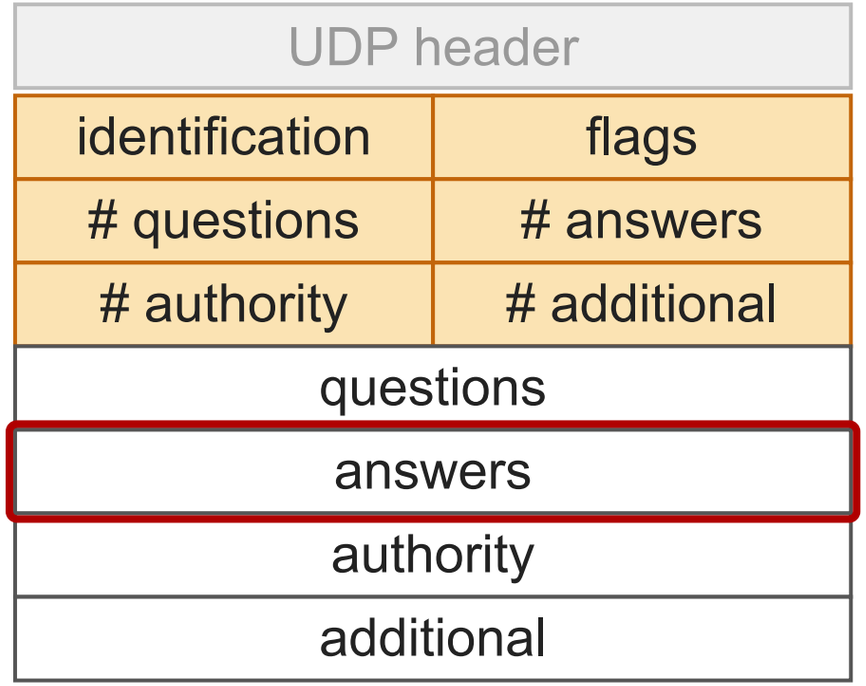
DNS messages



Questions

The name and type asked for, e.g., cs.uwaterloo.ca, type A.

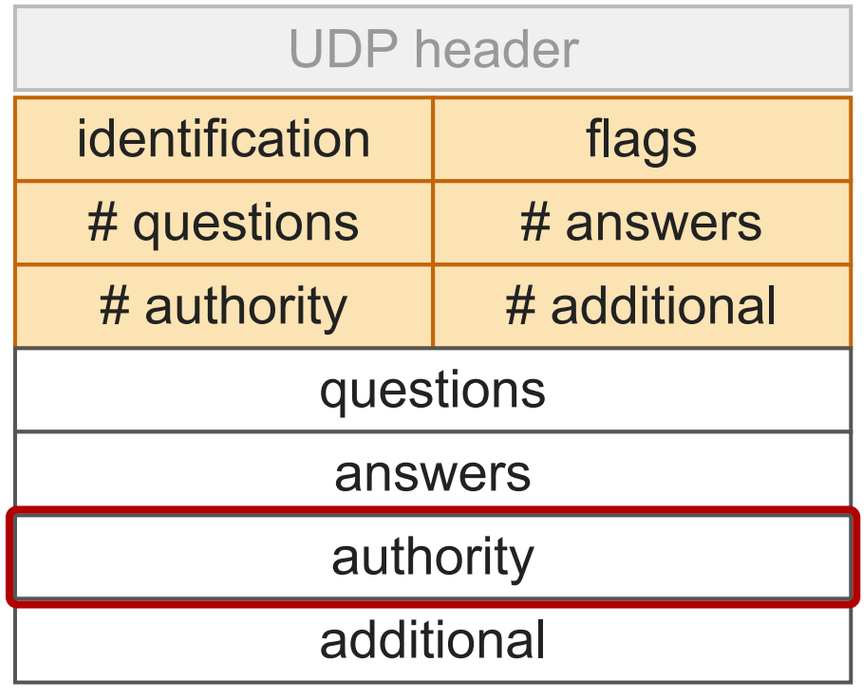
DNS messages



Answers

Records that answer the question, e.g.,
(cs.uwaterloo.ca, 129.97.167.168, A, 60).
Empty if the server does not have the answer.

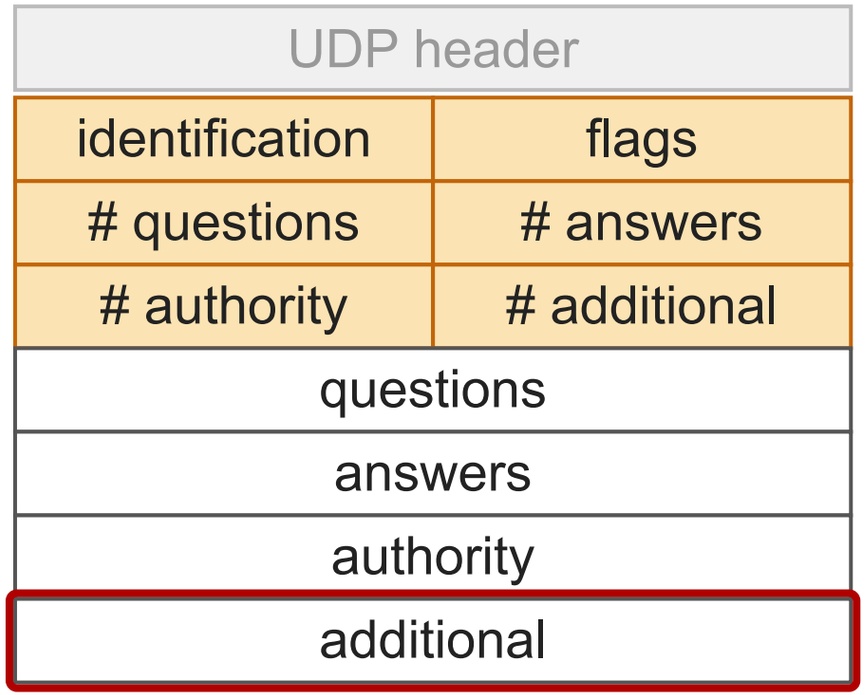
DNS messages



Authority

If the server does not have the answer: NS records for the servers that do, e.g., in .ca's reply, (uwaterloo.ca, cn-dns-mc-new.uwaterloo.ca, NS, 86400).

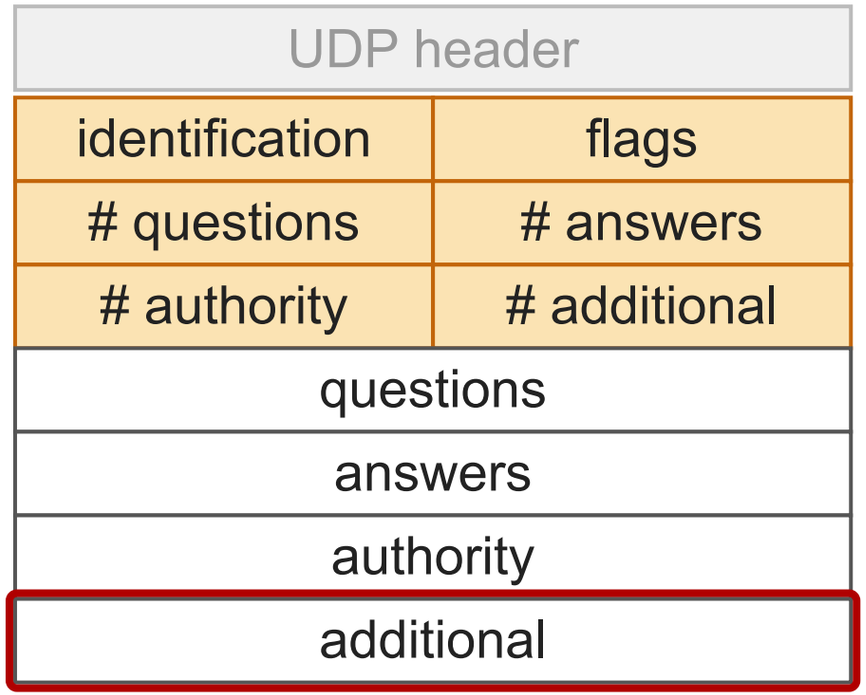
DNS messages



Additional

The A records of those name servers, so the asker can reach them without another lookup, e.g., (cn-dns-mc-new.uwaterloo.ca, 129.97.128.229, A, 86400).

DNS messages

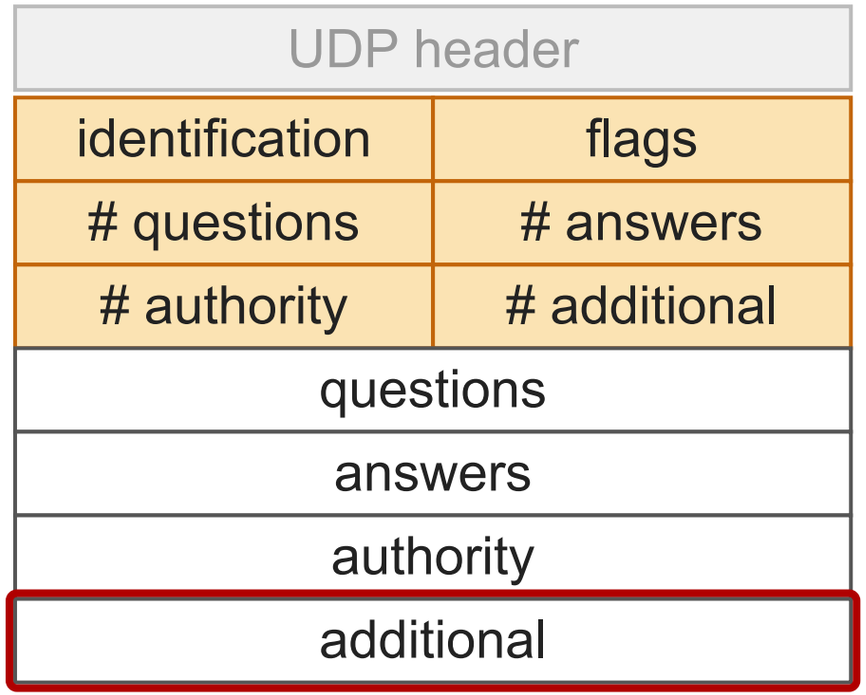


Additional

The A records of those name servers, so the asker can reach them without another lookup, e.g., (cn-dns-mc-new.uwaterloo.ca, 129.97.128.229, A, 86400).

- A reply can hold several records with the same name and type (e.g., mystartup.ca's two A records).

DNS messages



Additional

The A records of those name servers, so the asker can reach them without another lookup, e.g., (cn-dns-mc-new.uwaterloo.ca, 129.97.128.229, A, 86400).

- A reply can hold several records with the same name and type (e.g., mystartup.ca's two A records).
- Servers typically rotate the order in which they include the answers, and clients usually try the first, so this helps a bit with load balancing.

Caching

- Every DNS server (and endpoint) keeps the records it learns for TTL seconds.
  - The next lookup for the same name is answered from the cache, right away.
  - Local DNS servers almost always have the TLD servers cached. Why?
- A cached copy can be out of date. How?
- So DNS gives up strong consistency: an answer from a cache can be old, for up to its TTL.
  - The price of answering from caches, which makes DNS fast and scalable.
  - Acceptable: most records change far less often than they are looked up.
- What do you gain and lose with a long TTL? With a short one? uwaterloo.ca's NS record has a TTL of one day; cs.uwaterloo.ca's A record, 60 seconds. Why?

Does DNS meet our properties? (in-class)

property	how DNS meets it
scalable	
answer quickly	
high availability	
decentralized	
secure	not in the original design, but there are later additions: signed records (DNSSEC) and encrypted lookups (DNS over TLS or HTTPS)

Reset

- There is more to DNS: load balancing, mail servers (MX records), proving you own a domain, and attacks such as cache poisoning and floods of queries.

Try DNS on your computer

```
$ dig cs.uwaterloo.ca
…
;; QUESTION SECTION:
;cs.uwaterloo.ca. IN A

;; ANSWER SECTION:
cs.uwaterloo.ca. 60 IN A 129.97.167.168

;; Query time: 51 msec
;; SERVER: 192.168.0.1#53(192.168.0.1)
```

— you asked for an A record: an address

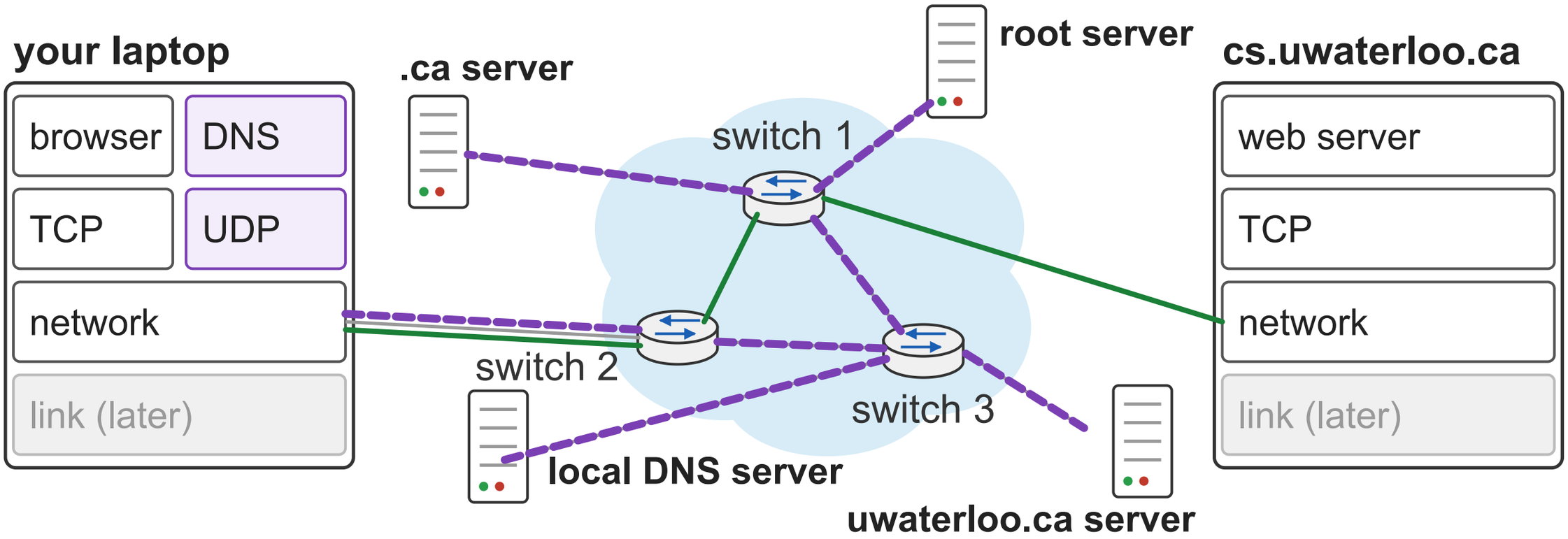
— the address; caches may keep it 60 s (the TTL)

— how long the lookup took

— your local DNS server

- Which local DNS server: `scutil --dns` (macOS), `ipconfig /all` (Windows), `resolvectl status` (Linux). The whole walk: `dig +trace cs.uwaterloo.ca`.

Recap


your laptop
browser
DNS
TCP
UDP
network
link (later)
.ca server
root server
switch 1
switch 2
switch 3
local DNS server
uwaterloo.ca server
cs.uwaterloo.ca
web server
TCP
network
link (later)